软件工程
项目管理

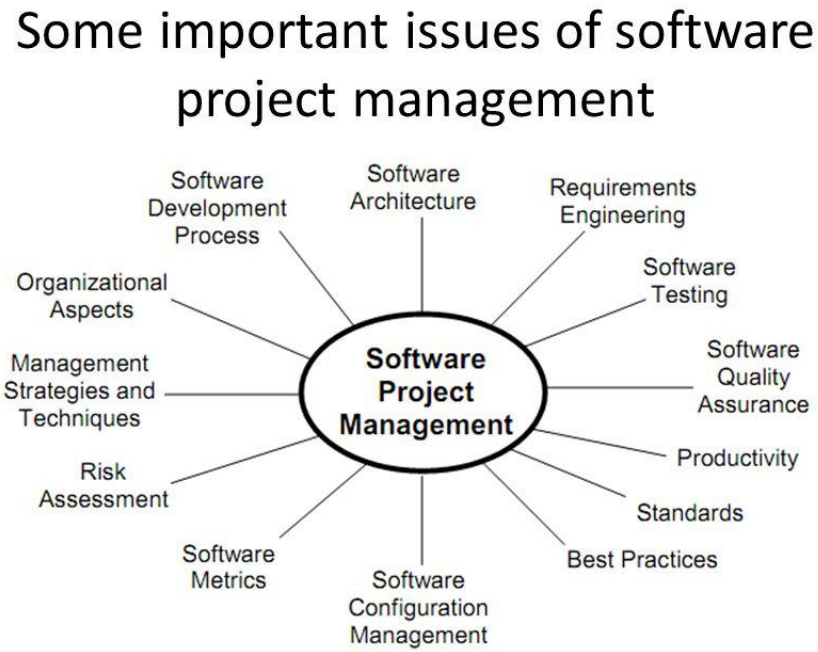
陈龙彪
厦门大学
2025年秋学期

学习内容

- 计划、组织、监控和控制软件项目所需要的管理技术
  - 在一个软件项目中为什么必须对人员、问题和过程进行管理?
  - 什么是软件度量? 如何使用它们管理软件项目和软件过程?
  - 一个软件项目组如何对工作量、成本和项目时间进行可靠的估算?
  - 采用什么技术去正式地评估影响项目成功的风险?
  - 一个软件项目管理者如何选择软件工程工作任务集?
  - 如何创建一个项目进度计划?
  - 如何定义质量使其能够被控制?
  - 什么是软件质量保证?
  - 为什么正式技术评审那么重要?
  - 在计算机软件开发之中及它被交付给客户之后如何进行变化管理?

学习内容

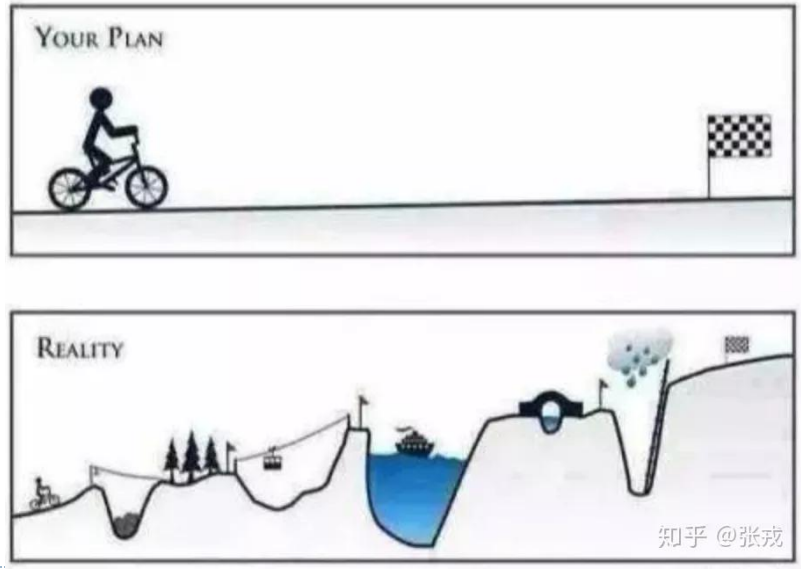
- 项目管理的概念
  - 人员、产品、过程、项目
- 软件过程和项目度量
  - 度量指标、方法、控制
- 软件项目计划
  - 估算、资源、分解、决策
- 风险分析和管理
  - 识别、预测、管理策略
- 项目进度安排及跟踪
  - 延迟、任务、进度安排
- 软件质量保证
  - 质量、评审、保证、错误防范
- 软件配置管理
  - 版本、变更、审计、SCM



第3章：项目管理的概念

项目管理太弱

- 软件危机后的普遍性结论：软件项目成功率非常 低的原因可能是项目管理能力太弱
- 软件项目管理是指软件生存周期中软件管理者所进行的一系列活动，其目的是在一定的时间和预设范围内，有效地利用人力、资源、技术和工具， 使软件系统或软件产品按原定计划和质量要求如期完成



How to lead?

How to organize?

How to collaborate?



How to motivate?

How to create good ideas?

ow to organize?

How to collabora



How to r

e good ideas?

软件项目管理的关注点（4P）

- 人员（People）
  - 人员是软件工程项目的基本要素和关键因素
  - 在对人员进行组织时，有必要考虑参与软件过程（及每一个软件项目）的人员类型
- 产品（Product）
  - 定义项目范围，其中包括建立产品的目的和范围、可选的解决方案、技术或管理的约束等
- 过程（Process）
  - 通常将项目分解为任务—子任务等，其分解准则是基于 软件工程的过程
- 项目（Project）
  - 采用科学的方法及工具对项目基本内容进行管理



软件项目管理的关注点（4P）

- 人员（People）
  - 人员是软件工程项目的基本要素和关键因素
  - 在对人员进行组织时，有必要考虑参与软件过程（及每一个软件项目）的人员类型
- 产品（Product）
  - 定义项目范围，其中包括建立产品的目的和范围、可选的解决方案、技术或管理的约束等
- 过程（Process）
  - 通常将项目分解为任务—子任务等，其分解准则是基于 软件工程的过程
- 项目（Project）
  - 采用科学的方法及工具对项目基本内容进行管理



软件项目管理中的共利益者（Stakeholder）

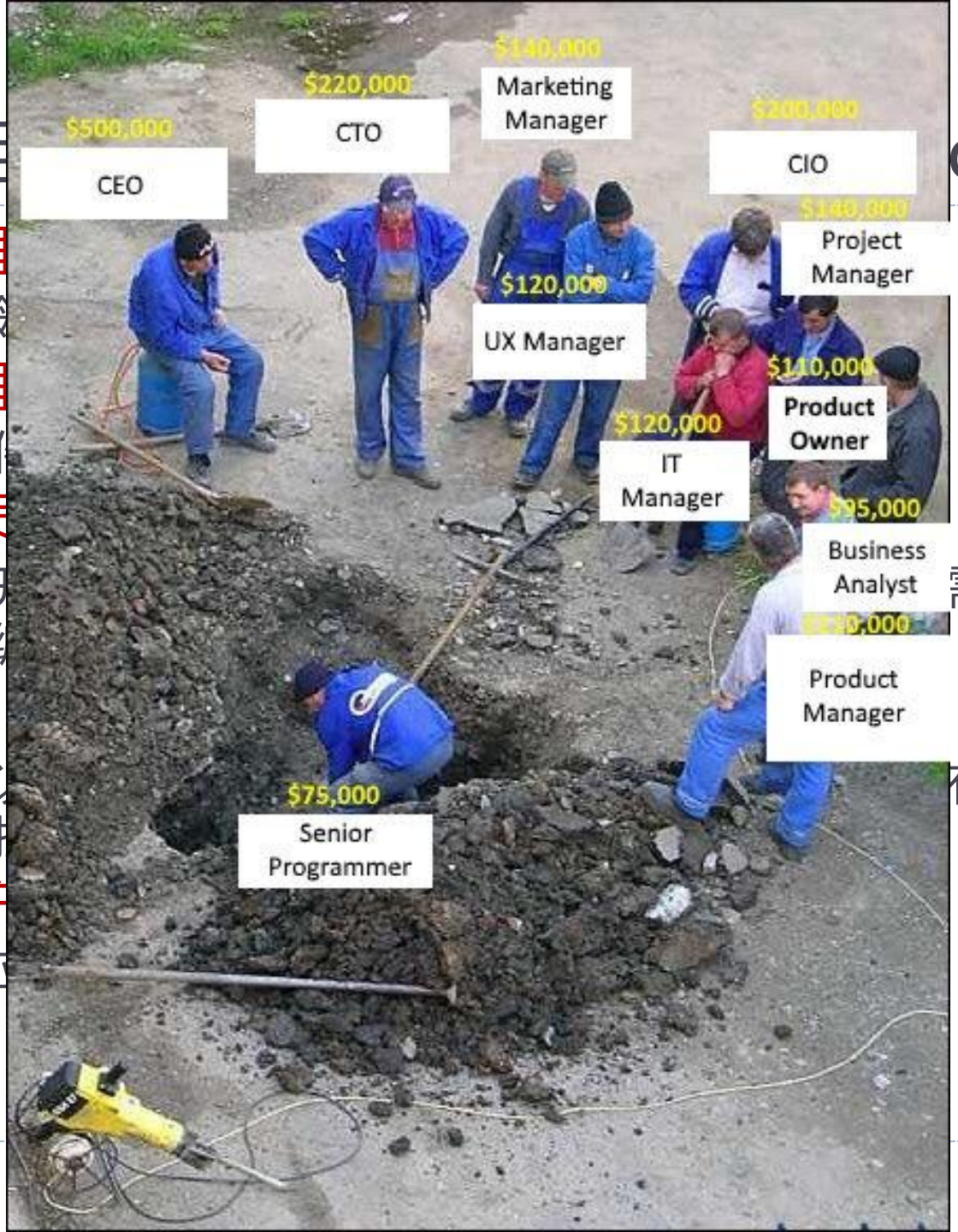
- 高级管理人员
  - 可以是领域专家，负责提出项目的目标并定义业务问题
- 项目管理人员
  - 负责软件项目的管理工作，其负责人通常称为项目经理
- 开发人员
  - 掌握了开发一个产品或应用所需的专门技术，胜任包括需求分析、设计、编码、测试、发布等各种相关的开发岗位
- 客户
  - 一组可说明待开发软件的需求的人，也包括与项目目标有关 的其它风险承担者
- 最终用户
  - 产品或应用提交后与产品/应用进行交互的

软件项目管理中的共利益者（Stakeholder）

- 高级管理人员
  - 可以是领域专家，负责提出项目的目标并定义业务问题
- 项目管理人员
  - 负责软件项目的管理工作，其负责人通常称为项目经理
- 开发人员
  - 掌握了开发一个产品或应用所需的专门技术，胜任包括需求分析、设计、编码、测试、发布等各种相关的开发岗位
- 客户
  - 一组可说明待开发软件的需求的人，也包括与项目目标有关 的其它风险承担者
- 最终用户
  - 产品或应用提交后与产品/应用进行交互的

软件项目 older)

- 高级管理
  - 可以是
- 项目管理
  - 负责软
- 开发人员
  - 掌握了
    设计、
- 客户
  - 一组可
    风险承
- 最终用户
  - 产品或



需求分析、

有关 的其它

软件项目 holder)

- 高级管理人
  - 可以是领 项
- 项目管理人
  - 负责软件 里
- 开发人员
  - 掌握了开 括需求分析、设计、编
- 客户
  - 一组可说 示有关 的其它风险承担
- 最终用户
  - 产品或应



知乎 @光头刘长发

软件项目组

- 为项目分配人力资源，该项目需要n个人工作k年
  - n个人被分配到m个不同的任务（m>n）相对而言几乎没有合作的情况发生；协调是软件管理者的责任
  - n个人被分配到m个不同的任务（m<n）需要建立非正式的“小组”；需指定专门的小组负责人；小组之间的协调由软件管理者负责
  - n个人被分成t个小组，每一个小组分配一个或多个任务每个小组有一个特定的结构，该结构是为同一个项目的所有小组定义的；协调由小组和项目管理者共同控制

软件项目组结构的七个因素

- 待解决问题的难度
- 产生的程序的规模、以代码行或者功能点来衡量
- 小组成员需要共同工作的时间（小组生存期）
- 问题能够被模块化的程度
- 待建造系统所要求的质量和可靠性
- 交付日期的严格程度
- 项目所需要的社交性（通信）的程度



软件项目组组织的四个范型

- 封闭式范型
  - 按照传统的权利层次来组织小组
- 随机式范型
  - 松散地组织小组，并依赖于小组成员个人的主动性
- 开放式范型
  - 具有封闭式范型的控制性，又包含随机式范型的创新性
- 同步式范型
  - 依赖于问题的自然划分，组织小组成员各自解决问题的 片段，他们之间没有什么主动的通信

软件项目组的人员协调

- 正式的、与人无关的方法
  - 软件工程文档和交付物（含源代码）、技术备忘录……
- 正式的、人员间的规程
  - 状态评审会议、设计和代码检查
- 非正式的、人员间的规程
  - 信息传播、问题解决及“需求和开发人员配置”的小组会议
- 人员间的网络
  - 与项目组之外的人进行的非正式的讨论



避免“小组毒性”

- 狂乱的工作环境：成员浪费时间，失去对工作目标的关注
- 个人、业务或技术因素：成员之间产生摩擦、高挫败率
- 碎片式的规程、不适当的过程：变成完成任务的障碍
- 角色定义不清楚：导致责任缺乏、相互指责
- 连续和重复的失败：信心缺乏、士气下降



案例分析

- 唐僧团队（Tangseng's Team）
  - 《西游记》中的唐僧团队，虽然是虚拟的，但是师徒历经百险求取真经的故事，不仅家喻户晓，而且是中国文化的集中代表
  - 这个团队最大的好处是互补性：领导有权威、有目标，但能力差点；员工有能力，但自我约束力差，目标不够明确，有时还会开小差
  - 但总的来看，这个团队是个非常成功的团队，虽然历经九九八十一磨难，但最后修成了正果



案例分析

- 唐僧团队（Tangseng's Team）
  - 一个成功的团队，基本上要有四种人：德者、能者、智者、劳者。德者领导团队，能者攻克难关，智者出谋划策，劳者执行有力
  - 德者唐僧：唐僧是一个目标坚定、品德高尚的人，他受唐王之命，去西天求取真经，以普渡众生，广播善缘
  - 能者悟空：悟空可称得上是职业经理人。他有个性、有想法、执行力强，也很敬业、重感情，懂得知恩图报，是个非常优秀的人才
  - 智者八戒：八戒虽然干活不积极，总是开小差，但在大是大非上立场比较坚定，打起妖怪来也不心慈手软；生活上能够随遇而安，有些脏累差的活，他虽有怨言，但也能完成；悟空有不对的地方，他都直言不讳，从某种程度上发挥了团队协调和管理作用
  - 劳者悟净：悟净是团队组织者。他工作任劳任怨，心细如丝。他经常站在悟空的一面说服唐僧，但当悟空有了不敬的言语，他又马上跳出来斥责悟空，护卫师傅。团队中这样的人，一定要给予恰当的位置，如行政、人事、质量管理、客户服务等方面

案例分析

- 唐僧团队（Tangseng's Team）
  - 一个成功的团队，基本上要有四种人：德者、能者、智者、劳者。德者领导团队，能者攻克难关，智者出谋划策，劳者执行有力
  - 德
  - 能
  - 智
  - 劳

孙悟空只能是一个好员工，不能成为一个好领导，为什么呢？

孙悟空最大的乐趣是降妖伏魔，常说“抓几个妖怪玩玩”，是一种工作狂的表现。他不近女色、不恋钱财、不惧劳苦，在降妖伏魔中找到了无限的乐趣。但是他天性顽皮、直言不讳，经常把玉皇大帝、各大神仙都不放到眼里，注定他无法成为一个卓越的领导。

王之命，
去、执行
的人才
大非上立
而安，有
地方，他
如丝。他经常站在悟空的一面说服唐僧，但当悟空有了不敬的言语，他又马上跳出来斥责悟空，护卫师傅。团队中这样的人，一定要给予恰当的位置，如行政、人事、质量管理、客户服务等方面

软件项目管理的关注点（4P）

- 人员（People）
  - 人员是软件工程项目的基本要素和关键因素
  - 在对人员进行组织时，有必要考虑参与软件过程（及每一个软件项目）的人员类型
- 产品（Product）
  - 定义项目范围，其中包括建立产品的目的和范围、可选的解决方案、技术或管理的约束等
- 过程（Process）
  - 通常将项目分解为任务—子任务等，其分解准则是基于 软件工程的过程
- 项目（Project）
  - 采用科学的方法及工具对项目基本内容进行管理



产品

- 确定软件范围
  - 信息目标、功能和性能
  - 软件范围在管理层和技术层都必须是无二义性和可理解的
- 问题分解：软件需求分析的核心活动，主要解决以下两个方面的问题
  - 必须交付的功能
  - 用于交付功能的过程



产品

- 在制定项目计划之前，应该首先确定产品的目标和范围，考虑可选的解决方案，识别技术和管理上的限制。如果没能这些信息，就不可能进行合理的（准确的）成本估算，也不可能进行有效的风险评估和适当的项目任务划分，更不可能制定可管理的项目进度计划来给出意义明确的项目进展标志。
- 软件开发者和客户必须一同定义产品的目标和范围。确定产品的目标只是标识出产品的总体目标（从客户的角度），而不用考虑这些目标如何实现。而确定产品的范围，要标识出产品的主要数据、功能和行为特性，而且更为重要的是，应以量化的方式界定这些特性。
- 了解了产品的目标和范围之后，就要开始考虑备选的解决方案了。虽然这一步并不讨论细节，但可以使管理者和参与开发人员根据给定的约束条件选择“最好”的方案，其中约束条件包括产品交付的期限、预算的限制，可用的人员、技术接口以及其他各种因素。

迷惑产品图鉴：智能无人便利店



软件项目管理的关注点（4P）

- 人员（People）
  - 人员是软件工程项目的基本要素和关键因素
  - 在对人员进行组织时，有必要考虑参与软件过程（及每一个软件项目）的人员类型
- 产品（Product）
  - 定义项目范围，其中包括建立产品的目的和范围、可选的解决方案、技术或管理的约束等
- 过程（Process）
  - 通常将项目分解为任务—子任务等，其分解准则是基于 软件工程的过程
- 项目（Project）
  - 采用科学的方法及工具对项目基本内容进行管理

过程

- 软件过程：定义、开发、支持
- 软件工程范型
  - 线性顺序模型、原型实现模型……
- 项目管理者必须决定哪一个过程模型最适合于
  - 需要该产品的客户和将做此工作的人员
  - 产品本身的特征
  - 软件项目组工作的项目环境



过程

- 软件过程提供了一个框架，在该框架下可以制定软件开发的综合计划
- 一小部分框架活动适用于所有软件项目，不用考虑其规模和复杂性
- 多种不同的任务集合——每一种集合都由任务、里程碑、工作产品以及质量保证点组成——使得框架活动适合于不同软件项目的特性和项目团队的需求
- 最后的普适合性活动——如软件质量保证、软件配置管理和测量，这些活动覆盖了过程模型
- 普适性活动独立于任何一个框架活动，且贯穿于整个过程之中



软件项目管理的关注点（4P）

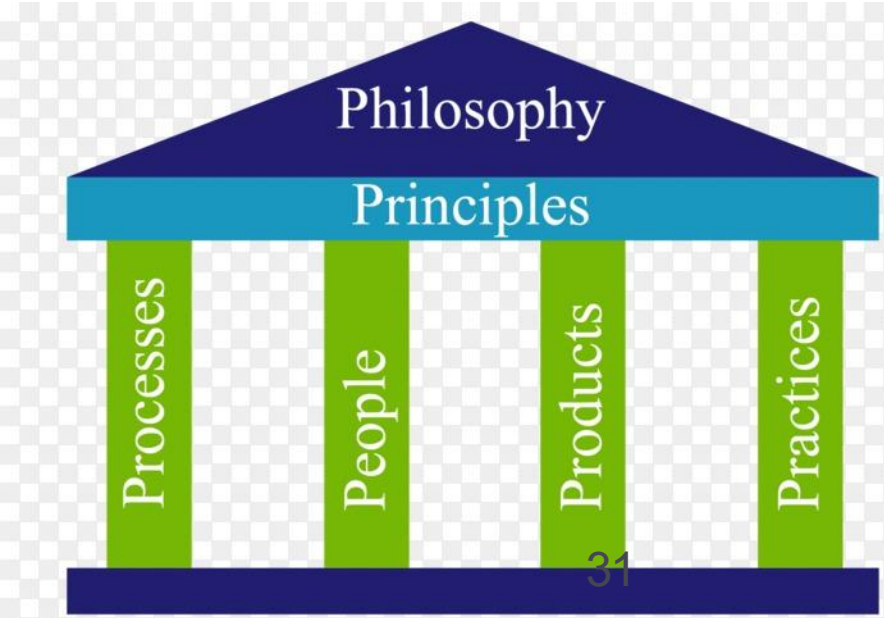
- 人员（People）
  - 人员是软件工程项目的基本要素和关键因素
  - 在对人员进行组织时，有必要考虑参与软件过程（及每一个软件项目）的人员类型
- 产品（Product）
  - 定义项目范围，其中包括建立产品的目的和范围、可选的解决方案、技术或管理的约束等
- 过程（Process）
  - 通常将项目分解为任务—子任务等，其分解准则是基于 软件工程的过程
- 项目（Project）
  - 采用科学的方法及工具对项目基本内容进行管理

项目

- 一个项目正处于危险状态的信号
  - 软件人员不了解他们的客户的需求
  - 产品范围定义糟糕
  - 没有很好地管理变化
  - 选择的技术发生变化
  - 业务需求发生变化（或未被很好地定义）
  - 时限是不现实的
  - 用户抵制
  - 失去赞助（或从来没有真正地得到过）
  - 项目组缺乏具有合适技能的人员
  - 管理者和开发者躲避已学到的最好的实践经验和教训

软件项目管理原则

- 在正确的基础上开始工作
- 保持动力
- 跟踪进展
- 做出聪明的决策
- 进行事后的分析
- 关键软件管理实践
  - 正式的风险管理
  - 经验成本和进度估算
  - 基于度量的项目管理
  - 获得值跟踪
  - 依据质量目标跟踪缺陷
  - 人员计划管理



W5HH原则

- 为什么（Why）该系统被开发?
- 将做什么（What）？什么时候（When）做?
- 某功能由谁（Who）负责?
- 他们的机构组织位于何处（Where）？
- 工作将如何（How）被技术和管理地进行?
- 每种资源需要多少（How much）？

软件项目管理的关注点

- 人是第一位的：任何管理者如果忘记了软件工程是人的智力密集的劳动，他就永远不可能在项目管理上得到成功
- 产品来自需求：任何管理者如果在项目开发早期没有进行全面的客户沟通，他有可能为错误的问题建造无用的解决方案
- 过程需要重视：对过程的不在意的管理者可能空有技术方法和工具却无法形成产品
- 项目要有计划：没有一个可靠的项目计划就开始工作的管理者将危及产品的成功

Week 2



第4章：软件过程和项目度量

软件度量

- 软件度量是指对计算机软件的范围广泛的测度
  - 测度可以应用于软件过程中，目的是在一个连续的基础上改进它
  - 测度也可以用于整个软件项目中，辅助估算、质量控制、生产率评估及项目控制
  - 测度能够被软件工程师使用以帮助评估技术工作产品的质量且在项目进行中辅助进行战术决策
- 在软件项目管理的语境中，我们主要关心生产率和质量的度量
- 有4个测度软件过程、产品和资源的理由
  - 为了刻画、评价、预测或改善

测量、度量和指标

- 测量（measure）
  - 对一个产品或过程的某个属性的范围、数量、维数、容量或大小提供了一个定量的指标
    - 每个数据点的数据采集
- 测度（measurement）
  - 确定一个测量的行为
    - 若干个点的数据采集
- 度量（metric）
  - 对一个系统、部件或过程具有的某个给定属性的度的一个定量测量
    - 由数据采集得到的评审结论



过程和项目领域中的度量

- 过程度量的收集涉及所有的项目，而且要经历相当长的时间，目的是提供能够引导长期的软件过程改进的一组过程指标
- 项目指标使得软件项目管理者能够
  - 评估正在进行的项目的状态
  - 跟踪潜在的风险
  - 在问题造成不良影响之前发现它们
  - 调整工作流程或任务
  - 评估项目组控制软件工程工作产品的质量的能力

过程度量和软件过程改善

- 改善过程的唯一合理的方法是测量过程的特定属性
  - 人员的技能和动因被认为是对质量和性能最有影响的因素
  - 产品的复杂性对软件质量和小组性能产生相当大的影响
  - 过程中采用的技术也有影响

软件度量礼仪

- 解释度量数据时使用通用的观念，并考虑组织的敏感性
- 对收集测量和度量的个人及小组提供定期的反馈
- 不要使用度量去评价个人
- 与开发小组一起设定清晰的目标以及达到这些目标的度量
- 不要用度量去威胁个人或小组
- 指出某个问题域的度量数据不应被赋予“否定的”含义；这些数据仅仅是过程改善的指标
- 不要迷惑于某一单个度量而排除其他重要度量

统计软件过程改善

- 故障分析
  - 根据来源分类所有的错误和缺陷
  - 记录修改每个错误和缺陷的成本
  - 统计每一类错误和缺陷的数目，并按降序排列
  - 计算每一类错误和缺陷的总成本
  - 分析数据，找出造成组织最高成本的错误和缺陷类型
  - 制定修改过程的计划，目的是消除成本最高的错误和缺陷类型

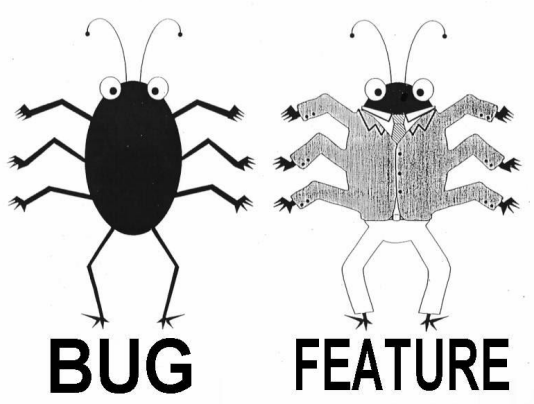


错误、缺陷

- 错误是软件工程产品或交付物中的某个瑕疵，在该软件交付给最终用户之前已被软件工程师发现
- 缺陷则是交付给最终用户之后才发现的瑕疵



据说是玩家在育碧的办公室里偷拍的！



项目度量

- 软件过程度量用于战略目的，软件项目度量则是战术的
  - 项目度量及从中导出的指标由项目管理者和软件项目组使用，以改进项目工作流程和技术活动
- 软件项目度量建议每个项目都应该测量
  - 输入：完成工作所需的资源的测量
  - 输出：软件工程过程中产生的交付物或工作产品的测量
  - 结果：表明交付物的有效性的测量
- 软件过程的直接测量包括花费的成本和工作量
  - 产品的直接测量包括产生的代码行、执行速度、内存大小及某段时间内报告的缺陷
  - 产品的间接测量包括功能、质量、复杂性、有效性、可 靠性、可维护性及许多其它“能力”

面向规模的度量

- 通过规范化质量和/或生产率的测量而得到
  - 每千行代码的错误数
  - 每千行代码的缺陷数
  - 每个代码行的花费
  - 每千行代码的文档页数
  - LOC (Lines of Codes) 测量依赖于程序设计语言

```
public void GetOnlineInfo()
{
    HttpBrowserCapabilities bc = Request.Browser;
    int hbcWidth = bc.ScreenPixelsWidth;
    //string hbcHeight = bc.ScreenPixelsHeight.ToString();

    //项目经理要求这里运行缓慢,好让客户给钱优化，并得到明显的速度提升
    Thread.Sleep(2000);

    HttpContext cont = System.Web.HttpContext.Current;
```

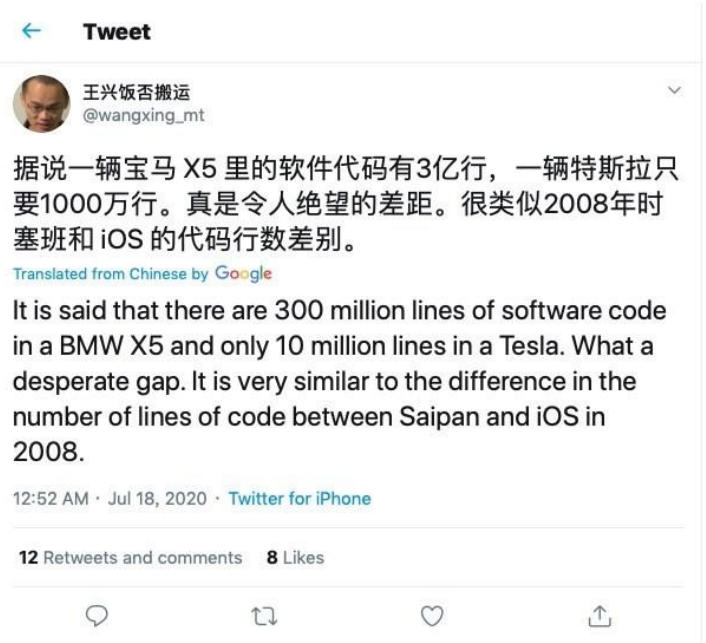
面向功能的度量

- 功能点（FP）是基于软件信息域的可计算的测量及软件复杂性的评估而导出的
  - 主观性强，加权因子是经验值
- 特征点
  - 功能点测量的超集，能够被用于系统及工程软件应用
  - 特征点度量适用于算法复杂性较高的应用
  - 实时系统、过程控制软件及嵌入式软件应用都有较高的 算法复杂性，因此适合特征点度量
- 3D功能点
  - 数据维、功能维、行为维
  - 指这三个软件维的特性被“计算、量化及变换”成测量， 以提供软件所传递的功能的指标

调和不同的度量方法

- 代码行和功能点度量之间的关系依赖于实现软件所采用的程序设计语言及设计的质量
  - 调和LOC和FP的关系

Programming Language	LOC per Function point avg.	median	low	high
Ada	154	-	104	205
Assembler	337	315	91	694
C	162	109	33	704
C++	66	53	29	178
COBOL	77	77	14	400
Java	63	53	77	-
JavaScript	58	63	42	75
Perl	60	-	-	-
PL/1	78	67	22	263
Powerbuilder	32	31	11	105
SAS	40	41	33	49
Smalltalk	26	19	10	55
SQL	40	37	7	110
Visual Basic	47	42	16	158



软件质量度量

- 软件工程的最高目标就是产生高质量的系统、应用或产品
  - 必须在成熟的软件过程背景下应用有效的方法及现代化的工具
  - 必须测量是否能够实现高质量系统、应用或产品



影响质量的因素概述

- 产品的操作（使用它）
- 产品的修改（改变它）
- 产品的变迁（修改它使之能够用于另一个环境，即移植）

- 质量测量
  - 正确性
  - 可维护性
  - 完整性
  - 可用性

缺陷排除效率

- 缺陷排除效率 DRE = E / (E + D)
  - E是软件交付给最终用户之前所发现的错误数
  - D是软件交付之后所发现的缺陷数
  - DRE鼓励软件项目组采用先进技术在交付之前发现尽可能多的错误

在软件工程过程中集成度量

- 大多数软件开发者仍然没有进行测量
- 更可悲的是，他们中大多数根本没有开始测量的愿望



支持软件度量的论点

- 如果我们不进行测量，就没法确定我们是否在改进。如果我们没有改进，将会导致失败
- 软件设计完成后，多数开发者都急于知道以下问题的答案
  - 哪些用户需求最有可能发生改变?
  - 本系统中哪些部分最易于出错?
  - 对于每一个模块应该设计多少测试?
  - 当测试开始时，我们能够预测多少错误?



建立基线

- 通过建立度量基线，可以在过程、项目以及产品（技术）级别上获得收益
  - 数据必须是合理精确的，应该避免关于过去项目的“推测”
  - 数据应该从尽可能多的项目中收集
  - 测量必须一致
  - 应用应该和将被估算的工作类似



度量收集、计算和评估

- 理想情况下，建立基线所需的数据已经在项目进行中被收集起来，但遗憾的是，很少能做到这样

- 软件度量收集过程：



管理变化：统计过程控制

- 相同的过程度量将随项目不同而变化
  - 如何能够告知是否作为改善活动的改善的度量值具有定量的影响？
  - 如何知道是否我们正在观察一个统计有效的趋势，或者是否该趋势仅仅是统计噪声的结果
  - 什么时候对特殊软件度量的改变是有意义的
- 控制图
  - 移动范围控制图
  - 个体控制图



小型组织的度量

- 大多数软件开发组织的软件人员不超过20，期望这样的组织将开发出全面的软件度量计划是不合理的，在多数情况下也是不现实的
- 一种实现任何软件过程相关活动的常识性方法
  - 保持简单
  - 剪裁以满足局部需要
  - 确定它可以增值

对于小的小组，收集测量和计算度量的成本范围在学习阶段是项目预算的3%到8%，熟悉后降到1%以下

建立软件度量计划

1. 识别你的业务目标
2. 识别你想知道或学习的内容
3. 识别你的子目标
4. 识别和你的子目标相关的实体和属性
5. 定型你的测度目标
6. 标识可量化的问题和相关的指标，以帮助达到测度目标
7. 标识你将收集的指标的数据元素
8. 定义将被使用的测量，并使这些定义可操作
9. 标识你将用于实现测量的动作
10. 准备实现测量的计划



第5章：软件项目计划

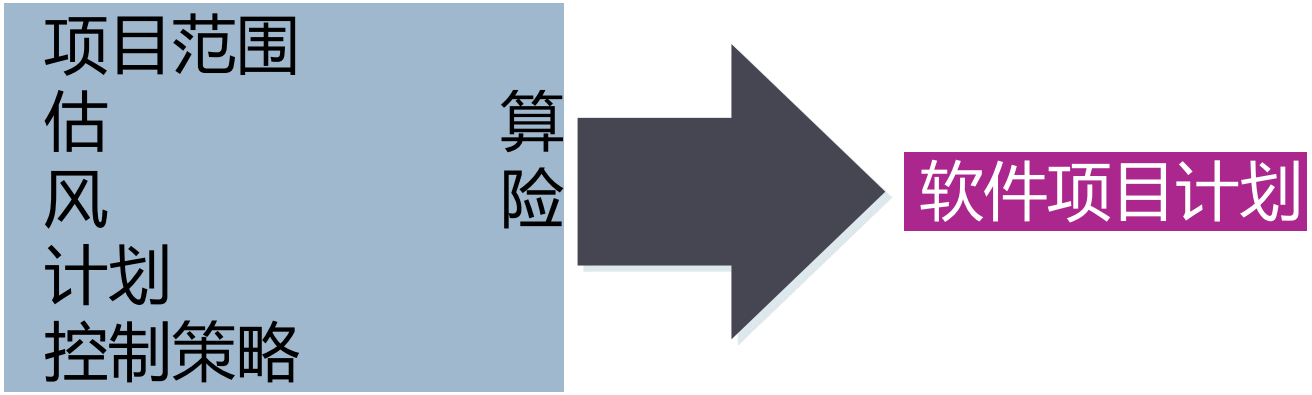
项目计划任务

- 规定项目范围
- 确定可行性
- 分析风险
- 确定需要的资源
- 估算成本和工作量
- 制定项目进度计划



项目计划目标

- 软件项目计划的目标是提供一个框架，使得管理者能够对资源、成本及进度进行合理的估算
- So the end result gets done on time, with quality!



软件范围

软件范围

- 软件范围描述了
  - 将要交付给最终用户的功能和特性
  - 输入和输出的数据
  - 使用软件时要呈现给用户的内容
  - 界定系统的性能、约束条件、接口和可靠性
- 定义范围的两种方法
  - 在与所有共同利益者交流之后，写出对软件范围的叙述性描述
  - 与最终用户开发一组用例



可行性

- 一旦确定了软件范围（并取得了用户的同意），人们自然会问：我们能够开发出满足范围要求的软件吗？这个项目可行吗？软件工程师常常匆忙越过这些问题，不料竟会一开始就注定要陷入这个项目的泥潭中







可行性

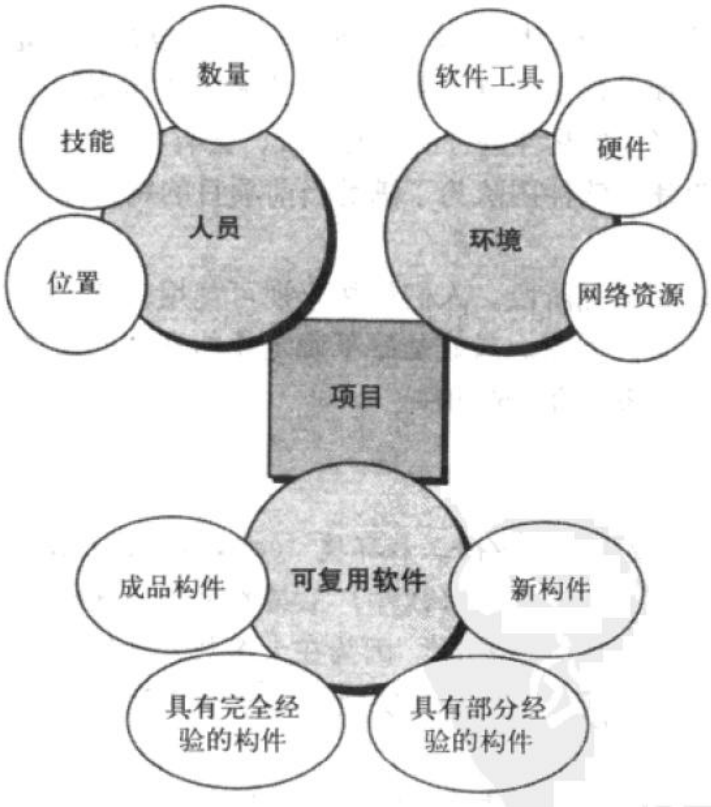
- 软件可行性有四个固定的因素[Putnam & Myers, 1997]
  - 技术：项目在技术是可行的吗？它在技术水平范围内吗？能够将缺陷减少到一定程度而满足应用的要求吗？
  - 经济：它在经济可行吗？能以开发组织、客户或市场负担得起的成本完成开发吗？
  - 时间：项目投入市场的时间可以击败竞争者吗？
  - 资源：组织拥有取得成功所需要的资源吗？



资源

资源

- 资源的类型
  - 人力资源
  - 可复用软件资源
  - 环境资源
- 对每一类资源，都要说明以下四个特征：资源的描述、可用性说明、何时需要资源、使用资源持续时间
  - 最后两个特性可以看成是时间窗口
  - 对于一个特定的时间窗口，必须在开发初期就建立的资源的可用性



人力资源

- 首先评估软件范围，并选择完成开发所需的技能，还要指定组织中的职位（如管理人员、高级软件工程师）和专业（如，电信、数据库、客户/服务器系统）
  - 对于一些比较小的项目（几个人·月），只要向专家做些咨询，也放一个人就可以完成所有的软件工程任务
  - 对于一些嵌套的项目，软件团队的成员可能分散在很多不同的地方，因此，要详细说明每个所处的位置
- 只有在估算出开发工作量（如，多少人·月）后，才能确定软件项目需要的人员数量

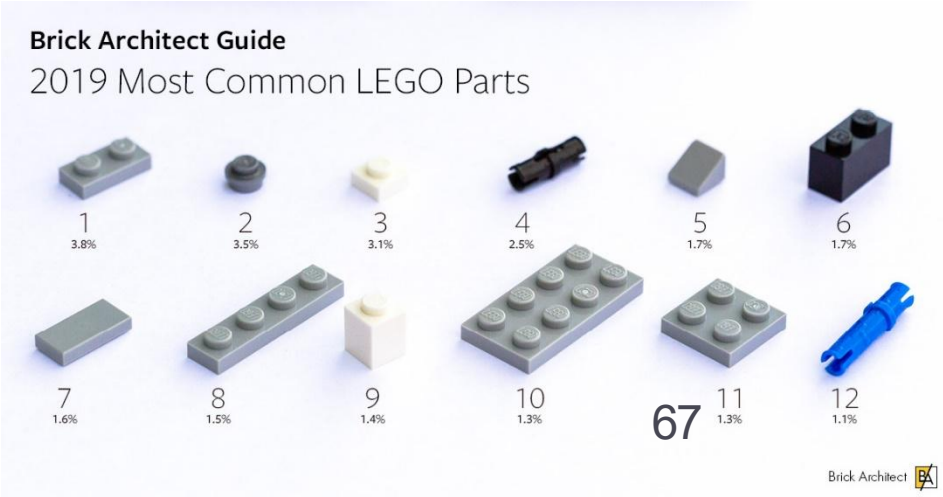




可复用软件资源

- 基于构件的软件工程强调可复用性，即创建并复用软件构造块，这种构造块通常被称为构件
  - 为了容易引用，必须对这些构件进行分类
  - 为了容易应用，必须使这些构件标准化
  - 为了容易集成，必须对这些构件进行确认

- 在制定计划时应该考虑四种软件资源：
  - 成品构件
  - 具有完全经验的构件
  - 具有部分经验的构件
  - 新构件



环境资源

- 支持软件项目的环境称为软件工程环境（software engineering environment，SEE），它集成了硬件和软件两大部分
  - 硬件提供了一个支持 （软件）工具平台，这些（软件）工具是采用良好的软件工程实践而获得工作产品所必需的。因为大多数软件组织中，有很多人都需要使用SEE，因此，项目计划人员必须详细规定硬件及软件所需的时间窗口，并验证这些资源是否可用
- 当开发一个基于计算机系统（集成特定的硬件及软件）时，软件团队可能需要用到由其他工程团队开发的硬件元件
  - 例如，在一种机械工具上使用的数控（numerical control，NC）软件可能需要某个特定的机床（如数控机床）来作为测试步骤的一部分；在开发高级排版软件项目的过程中，在某个阶段可能需要一台高质量的印刷机。软件项目计划人员必须指定硬件元素

环境

- 支
  en
  两
  -
- 当
  软
  -



次件

是采
数软件
页详细

时，
卡

C）软
的一部
要一台

软件项目估算

估算

- 几乎在所有基于计算机的系统中，软件都是最昂贵成分。对于复杂的定制的系统，如果成本估算误差很大，就会使赢利变成亏损
- 软件成本及工作量的估算从来都没有成为一门精确的科学。因为变化的因素太多—人员、技术、环境和策略，都会影响软件的最终成本和开发所用的工作量



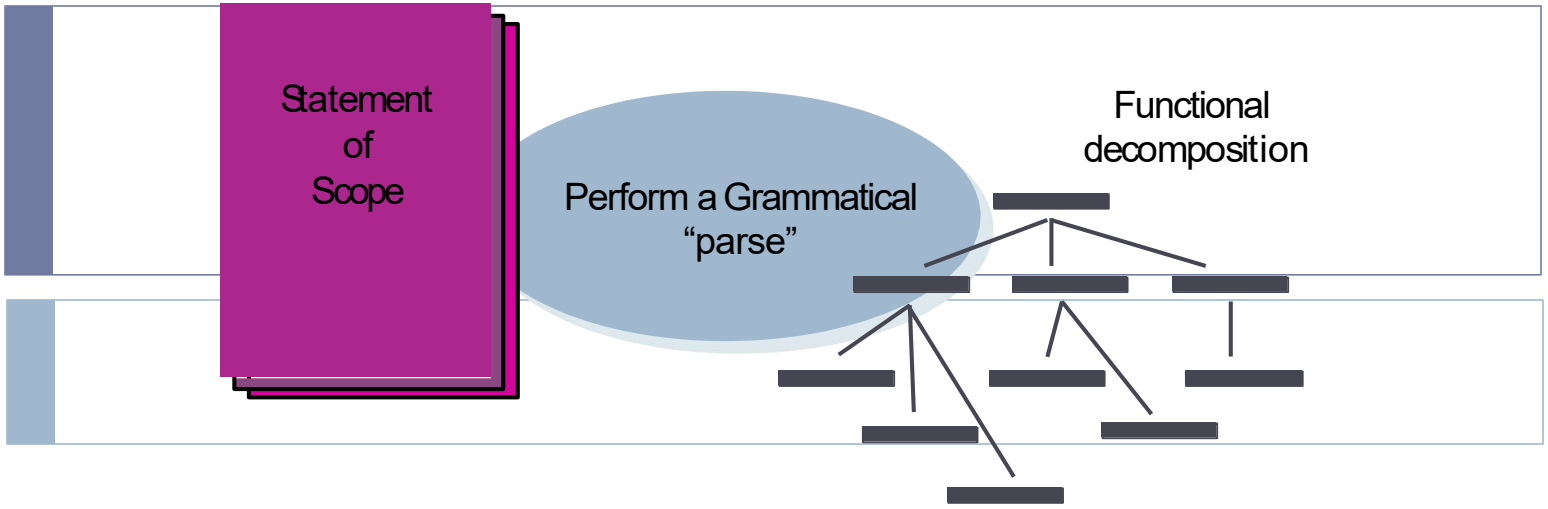


估算

- 对软件工程工作的资源、成本及进度进行估算时，需要
  - 经验
  - 了解有用的历史信息
  - 根据定性的信息时进行定量预言的勇气
- 项目复杂性对计划中固有的不确定性具有重大影响
- 项目规模是另一个影响估算准确性和效力的因素
- 结构不确定性的程度也会对估算的风险产生影响
- 历史信息的可用程度对估算的风险有较强的影响
- 估算具有与生俱来的风险，这种风险导致了不确定性
- 软件项目估算通过一系列系统化的步骤，在可接受的风险范围内提供估算结果

分解技术

- 软件项目估算是一种解决问题的形式。在大多数情况下，要解决的问题（对于软件项目来说，就是成本和工作量的估算）非常复杂，不能作为一个整体考虑。因此，我们要进行分解，把它划分成一组较小的（同时有望更容易管理的）问题，再定义它们的特性



分解技术

软件规模估算

- 软件项目估算的准确性取决于许多因素：
  - 计划人员估算待开发产品规模的正确程度；
  - 把规模估算转换成人的工作量、时间及成本的能力（受可靠软件度量的可用性的影响，这些度量数据来自以往的项目）；
  - 项目计划反映软件团队能力的程度；
  - 产品需求的稳定性及支持软件工程工作的环境。
- 在本节中，我们考虑软件规模的估算问题。由于项目估算的准确程度取决于待完成工作的规模估算，因此，规模估算是项目计划人员面临的第一个主要挑战。在项目计划中，规模是指软件项目的可量化的结果。如果采用直接的方法，规模可以用代码行（LOC）来测量。如果选择间接的方法，规模可以用功能点（FP）来表示。

分解技术

软件规模估算

- 四种估算规模问题的方法：
  - “模糊逻辑”法：使用这种方法，计划人员必须先确定应用的类型，定性地确定其量级，然后在初始范围内再细化该量级。
  - 功能点法：计划人员对信息域的特性进行估算。
  - 标准构件法：软件由许多不同的“标准构件”组成，对于某个特定的应用领域而言是通用的。例如，一个信息系统的标准构件是子系统、模块、屏幕、报表、交互程序、批处理程序、文件、LOC及对象级的指令。项目计划人员估算每个标准构件出现的次数，然后使用历史项目数据来确定每个标准构件交付时的规模。为了说明这点，让我们以一个信息系统为例。计划者估算将产生18个报表，历史数据表明每一个报表需要967行Cobol［PUT92］代码。这使得计划者估算出报表构件需要17000LOC。对于其他标准构件也可以进行类似的估算及计算，将它们合起来就得到最终的规模值。
  - 修改法：当项目要使用已有的软件作为项目的一部分，并且该软件必须做某些方面的修改时，可以使用这种方法。计划人员要估算必须完成的修改数量和类型（如，复用、增加代码、修改代码、删除代码）

分解技术

软件规模估算

- 建议：
  可以将上述每种估算规模的方法所产生的结果在统计上进行结合，产生一个三点估算或期望值估算结果。实现方法是：先确定规模的乐观值（低）、可能值和悲观值（高），然后使用下一节中描述的公式（3.1）将它们结合起来。

分解技术

基于问题的估算

- 在第2讲中，我们已描述了 LOC 和 FP 测量，从中可以计算出生产率度量。在软件项目估算中，LOC和FP数据被用于两个方面：（1）作为估算变量，用于度量软件中每个元素的规模；（2）作为基线度量，这些度量数据是从以前的项目中收集来的，将它们与估算变量联合使用，进行成本和工作量的估算。
- LOC和FP估算是两种不同的估算技术，但两者有很多共同的特性：项目计划人员从界定的软件范围陈述入手，根据该说明将软件分解成一些可分别独立进行估算的功能问题。然后，估算每个功能的LOC或FP（即估算变量）。当然，计划人员也可以选择其他元素进行规模估算，诸如类或对象、变更、受影响的业务过程。
- 然后，将基线生产率度量（如，LOC/pm或FP/pm，其中pm代表人月）应用于估算变量中，导出每个功能的成本及工作量。将所有功能估算合并起来，即可产生整个项目的总体估算。

分解技术

基于问题的估算

- 然而，应该注意到：对于一个组织而言，其生产率度量常常是变化的，使用单一的基线生产率度量是不可信的。一般情况下，平均的 LOC/pm或 FP/pm应该根据项目领域来计算。即，应该根据项目的团队规模、应用领域、复杂性、以及其他相关参数对项目进行分类，然后计算出各个本领域的生产率平均值。当估算一个新项目时，首先应将该项目对应到某个领域上，然后，再使用恰当的领域生产率平均值对其进行估算。
- LOC和FP估算技术在分解应用问题时，所要求的详细程度及划分目标有所不同。当LOC被用作估算变量时，分解是绝对必要的，而且常常需要分解到非常详细的程度。分解的程度越高，就越有可能得到非常精确的LOC估算。
- 对于FP估算，分解则是不同的。它并不关注于功能，而是要估算五个信息域（输入、输出、数据文件、查询、和外部接口）的特性，以及14种复杂度校正值。然后，利用这些估算结果导出FP值，该值可与过去的数据结合，来产生项目估算。

分解技术

基于问题的估算

- 不管使用哪种估算变量，项目计划人员都要首先为每个功能或每个信息域值确定一个估算值的范围。利用历史数据或凭直觉(当其他方法都失效时)，计划人员为每个功能或每个信息域的计数值都分别估算出一个乐观的、可能的、及悲观的规模值。当确定了值的范围后，就得到了一个不确定程度的隐含指标。
- 接着，计算三点（估算值）或期望值。可以通过乐观值(S_{opt})、可能值(S_m)和悲观值(S_{pess})估算的加权平均值来计算估算变量（规模）的期望值S：

$$S = (S_{opt} + 4S_m + S_{pess}) / 6 \qquad (4.1)$$

其中，“可能”估算值的权重最大，并遵循β概率分布。我们假定实际的规模结果落在乐观值与悲观值范围之处的概率很小。

分解技术

基于LOC 估算的实例

- 作为LOC和FP基于问题估算技术的实例，让我们来看一个例子：一个为机械零件计算机辅助设计（CAD）应用而开发的软件包。该软件将要运行在一个工程工作站上，且必须与各种外部设备有接口，包括鼠标、数字化仪、高分辨率彩色显示器以及激光打印机。可以建立一个初步的软件范围陈述：

  机械CAD软件接受工程师输入的二维或三维几何数据。工程师通过用户界面与 CAD系统进行交互并控制它，该用户界面应表现出良好的人机界面设计特征。所有几何数据和其他支持信息都保存在一个CAD数据库中。要开发一些设计分析模块，以产生所需的输出，这些输出将显示在各种不同的图形设备上。软件必须设计成能够控制外部设备，并能与外部设备（包括鼠标、数字化仪、激光打印机和绘图机）进行交互。

分解技术

基于LOC 估算的实例

- 作为LOC和FP基于问题估算技术的实例，让我们来看一个例子：一个为机械零件计算机辅助设计（CAD）应用而开发的软件包。该软件将要运行在一个工程工作站上，且必须与各种外部设备有接口，包括鼠标、数字化仪、高分辨率彩色显示器以及激光打印机。可以建立一个初步的软件范围陈述：

  机械CAD软件接受工程师输入的二维或三维几何数据。工程师通过用户界面与CAD系统进行交互并控制它，该用户界面应表现出良好的人机界面设计特征。所有几何数据和其他支持信息都保存在一个CAD数据库中。要开发一些设计分析模块，以产生所需的输出，这些输出将显示在各种不同的图形设备上。软件必须设计成能够控制外部设备，并能与外部设备（包括鼠标、数字化仪、激光打印机和绘图机）进行交互。

- 上述关于范围的陈述是初步的 - 它没有规定边界。必须对每个句子进行补充说明，提供具体的细节及定量的边界。例如，在开始估算之前，计划人中必须要确定“良好的人机界面设计特征”是什么含义，或“CAD数据库”的规模和复杂度是怎样的。

分解技术

基于LOC 估算的实例

为了进行估算，我们假定已经做了进一步的细化，并确定了该软件包应具有的主要的软件功能，如图4 - 2所示。遵照LOC的分解技术，能够得如图3 - 2所示的估算表，表中给出了每个功能的LOC估算范围。例如，三维几何分析功能的LOC估算范围是：乐观值：4600，可能值：6900，悲观值：8600。应用公式4.1，得到三维几何分析功能的期望值是6800LOC。这个值被输入到表中。其他估算也可以通过类似的方式获得。将LOC估算列的所有值相加，即得到该CAD系统的规模估算

功　能	LOC 估 算
用户接口及控制设备（UICF）	2 300
二维几何分析（2DGA）	5 300
三维几何分析（3DGA）	6 800
数据库管理（DBM）	3 350
计算机图形显示设备（CGDF）	4 950
外部设备控制功能（PCF）	2 100
设计分析模块（DAM）	8 400
总代码行估算	33 200

图4 - 2 LOC方法的估算表

分解技术

基于LOC 估算的实例

回顾历史数据可以看出，这类系统的组织平均生产率是 620 LOC/pm。如果一个劳动力价格是每月8000美元，则每行代码的成本约为13美元。根据LOC估算及历史生产率数据，该项目总成本估算值是431 000美元，工作量估算是 54 个人·月（假定有估算精确度要求，精确度取整到千美元及人·月。取到元或十分之一人·月的数学精确度是不现实的）。

功　能	LOC 估 算
用户接口及控制设备（UICF）	2 300
二维几何分析（2DGA）	5 300
三维几何分析（3DGA）	6 800
数据库管理（DBM）	3 350
计算机图形显示设备（CGDF）	4 950
外部设备控制功能（PCF）	2 100
设计分析模块（DAM）	8 400
总代码行估算	33 200

图4 - 2 LOC方法的估算表

分解技术

基于FP 估算的实例

- 基于FP估算时，问题分解关注的不是软件功能，而是信息域的值。项目计划人员分别对CAD软件的输入、外部输出、外部查询、内部逻辑文件和外部接口文件进行估算，参看图4－3。然后使用产品度量技术来计算FP的值。为了估算FP的值，我们假定复杂加权因子都取平均值。图4－3给出估算的结果。

信息域值	乐观值	可能值	悲观值	估算值	加权因子	FP值
外部输入数	20	24	30	24	4	97
外部输出数	12	15	22	16	5	78
外部查询数	16	22	28	22	5	88
内部逻辑文件数	4	4	5	4	10	42
外部接口文件数	2	2	3	2	7	15
总计						320

图4－3 估算信息域的值

分解技术

基于FP 估算的实例

- 估算出每一个复杂度加权因子，并根据产品度量方法计算出复杂度校正因子：

因　子	值	因　子	值
1. 备份和恢复	4	3. 分布式处理	0
2. 数据通信	2	4. 关键性能	4
5. 现有的操作环境	3	11. 设计可复用的代码	4
6. 联机数据输入	4	12. 设计中的转换与安装	3
7. 多屏幕输入切换	5	13. 多次安装	5
8. 主文件联机更新	3	14. 易于变更的应用设计	5
9. 信息域值复杂度	5	**复杂度校正因子**	**1.17**
10. 内部处理复杂度	5		

分解技术

基于FP 估算的实例

- 最后，得出FP的估算值：

$$FP_{estimated} = 总计 \times [\ 0.65 + 0.01 \times \Sigma(F_i)\ ]$$

$$FP_{estimated} = 375$$

- 这类系统的组织平均生产率是 6.5 FP/pm。如果一个劳动力价格是每月8 000美元，则每个FP的成本约为 1230 美元。根据FP估算及历史生产率数据，总的项目成本估算值是461 000美元，工作量估算值是 58 人·月。

分解技术

基于过程的估算

- 最通过的项目估算技术是根据将要采用的过程进行估算。即，将过程分解一组较小的任务，并估算完成任务所需的工作量。
- 同基于问题的估算技术一样，基于过程的估算首先从项目范围中抽取出软件功能。接着给出每个功能所必须执行的一系列的框架活动。这些功能和相关的框架活动可以用表格形式输出，类似于图4 - 4所示。
- 一旦将问题功能与过程活动结合起来，计划人员就可以针对每个软件功能，估算完成各个软件过程活动所需的工作量（如人·月），这些数据写在图3 - 4的中心部分。然后，将平均劳动力价格（即成本/单位工作量）应用 于每个软件过程活动的估算工作量，就可以估算出成本。但各项任务的劳动力价格很可能是不同的。高级技术人员主要投入到早期的框架活动中，而初级技术人员通常参与后期的构造和发布活动，高级技术人员的劳动力价格要高于初级技术人员。

活 动	客户沟通	策划	风险分析	工 程		构造发布		客户评估	合 计
任务→				分析	设计	编码	测试		
功能									
↓									
UICF				0.50	2.50	0.40	5.00	n/a	8.40
2DGA				0.75	4.00	0.60	2.00	n/a	7.35
3DGA				0.50	4.00	1.00	3.00	n/a	8.50
CGDF				0.50	3.00	1.00	1.50	n/a	6.00
DBM				0.50	3.00	0.75	1.50	n/a	5.75
PCF				0.25	2.00	0.50	1.50	n/a	4.25
DAM				0.50	2.00	0.50	2.00	n/a	5.00
合计	0.25	0.25	0.25	3.50	20.50	4.50	16.50		46.00
%工作量	1%	1%	1%	8%	45%	10%	36%		

图4 - 4 基于过程的估算表

分解技术

基于过程的估算

- 最后一个步骤就是计算每一个功能及框架活动的成本和工作量。如果基于过程的估算是不依赖LOC和FP估算而实现的，我们现在就已经有了两组或三组成本与工作量的估算，可以进行比较、调和。如果两组估算非常一致，则有理由相信估算是可靠的。反过来，如果这些分解技术得到的结果不一致，则必须做进一步调查和分析。



分解技术

基于过程估算的实例

- 为了说明基于过程估算的使用方法，我们还是考虑前一小节所介绍的CAD软件。系统配置及软件功能都保持不变，并已在项目范围来说明。
- 参看图4 - 4所示的基于过程的估算表，表中对CAD软件的每个功能(为了简化做了省略)，都给出了其各个软件工程活动的工作量估算(人·月)。其中，工程和构造发布活动又被细分成表中所示的主要软件工程任务。对客户沟通、策划和风险分析活动，还给出了总工作量的估算，这些数值都列在表底部的“合计”行中。水平合计和垂直合计为估算分析、设计、编码及测试所需的工作量提供了一个指标。应该注意到：“前期”的工程任务（需求分析）花费了全部工作量的53%，说明了这些工作的相对重要性。
- 如果平均一个劳动力价格是每月8000美元，则项目总成本的估算值是368000元，工作量的估算值是46人·月。如果需要的话，每一个框架活动或软件工作工程任务都可以采用不同的劳动力价格，分别进行计算。

分解技术

基于用例的估算

- 用例能使软件团队深入地了解软件的范围和需求。但由于以下的的原因，建立基于用例的估算方法还有困难：
  - 描述用例时，可以采用多种格式和风格 - 没有标准形式。
  - 用例表现的是软件的外部视图（用户视图），常常在不同的抽象级别上建立
  - 用例没有标识出它所描述的功能和特性的复杂性。
  - 用例没有描述出涉及很多功能 和特性的复杂行为（如，交互）。
- 与LOC和FP不同，一个角色的“用例”可以需要数月的工作量，而另一个角色的“用例”可能在一到两天内就能完成。

分解技术

基于用例的估算

- 尽管有很多研究已经考虑将用例作为估算的输入，但到目前为止，还没能出现被证实的估算方法。用例可以用于估算，但只有在用例描述的“结构层次”的情况下来考虑。
- 该结构层次中的任一层次都可以由不超过10个用例来描述，而每个用例包括的场景不超过30个。显然，与描述单一子系统的用例相比，描述大型系统的用例要在更高的抽象级别上建立（并代表相当大的开发工作量）。因此，在用例能够用于估算之前，首先要建立结构层次内的层次，确定每个用例的平均长度（页数），定义软件的类型（如，实时、商业、工程/科学、嵌入式），考虑系统大致的体系结构。一旦确立了这些特性，就可以利用经验数据确定每个用例的LOC或FP的估算值（对于层次结构中的每一层）。然后，再根据历史数据计算开发系统所需的工作量。

分解技术

基于用例的估算

- 为了说明如何进行计算，考虑以下关系式：
- $LOC估算 = N \times LOC_{avg} + [(S_a/S_h - 1) + (P_a/P_h - 1)] \times LOC_{adjust}$ (4.2)
- 其中，
  - N：实际的用例数
  - LOC_{avg}： 在这种类型的子系统中，每用例的历史平均LOC值。
  - LOC_{adjust} ：校正值，以LOC_{avg}的n%来表示，其中n根据当前项目的情况来确定，表示该项目与“一般”项目的差异。
  - S_a ：每个用例包含的实际场景数。
  - S_h ：在这种类型的子系统中，每个用例包含的平均场景数。
  - P_a ：每个用例的实际页数。
  - P_h ：在这种类型的子系统中，每个用例的平均页数。
- 公式（4 - 2）是根据实际的用例数对LOC值进行大致的估算，其中，实际用例数已根据用例场景数和页长度进行了校正，校正值等于每个用例的历史平均LOC值的n%。

分解技术

基于用例估算的实例

- 之前例子中的CAD系统软件包括三个子系统组：
  - 用户界面子系统（包括UICF）。
  - 工程子系统组（包括2DGA子系统，3DGA子系统和DAM子系统）。
  - 底层子系统组（包括CGDF子系统和PCF子系统）。
- 用户界面子系统由6个用例来描述，描述每个用例的场景不超过10个，用例的平均长度是6页。工程子系统组由10个用例来描述（这些是在结构层次的较高层次上来考虑的），与每个用例相关的场景不超过20个，用例的平均长度是8页。最后，底层子系统组由5个用例来描述，每个用例平均只有6个场景，平均长度是5页。

分解技术

基于用例估算的实例

- 利用公式（4.2），令 n=30%，就产生了如图3 - 5所示的表格。考虑表的第一行，历史数据显示，当描述用例的场景不超过12个，用例的长度少于5页时，用户界面软件中每个用例平均需要800 LOC。这些数据非常符合CAD系统的实际情况。因此，可以使用公式（4.2）来计算用户界面子系统的LOC估算值。使用同样的方法，可以对工程子系统线和底层子系统组做出估算。在图4 - 5中，还对估算进行了汇总，指出CAD软件总规模的估算值为42 500 LOC。

	用例	场景	页	场景	页	LOC	LOC估算
用户界面子系统	6	10	6	12	5	560	3 366
工程子系统组	10	20	8	16	8	3 100	31 233
底层子系统组	5	6	5	10	6	1 650	7 970
LOC估算合计							42 568

图4 - 5 用例估算

分解技术

基于用例估算的实例

- 以620 LOC/pm作为这类系统的平均生产率，一个劳动力价格是每月8 000美元，则每行代码的成本约为13美元。根据用例用例估算和历史生产率数据，项目总成本的估算值是552 000美元，工作量的估算值是68人·月。



调和不同的估算方法

分解技术

- 在前面讨论的估算技术导出了多种估算方法，必须对这些估算方法进行调和，以得到对工作量、项目持续时间或成本的一致估算。为了说明这个调和过程，仍以CAD软件为例：

  对CAD软件总工作量的估算，最低值是46人·月（由基于过程的估算方法得出），最高值是68人·月（由用例估算方法得出）。平均估算值（使用所有4种方法）是56人·月。与平均估算值相比，最低估算值的偏差约为18%，最高估算值的偏差约为21%。

- 如果估算方法所得到结果一致性很差，则需要对估算所使用的信息进行重新评估。不同估算之间差别很大的原因一般能够追溯到以下两个原因之一：
  - 计划人员没有充分了解或误解了项目范围。
  - 在基于问题的估算技术中所使用的生产率数据不适合本应用系统，过时了，或者被误用了。
- 计划人员必须确定产生差别的原因，再来调和估算结果。

经验估算模型

- 估算模型的结构
- COCOMO Ⅱ模型
- 软件方程式

经验估算模型

- 计算机软件估算模型使用由经验导出的公式来预测工作量，工作量是LOC或FP的函数。
- LOC或FP是利用之前“基于LOC估算的实例”以及“基于FP估算的实例”小节所描述的方法进行估算的，但不使用该节中的表，而是将LOC或FP的结果值代入到估算模型中。
- 用以支持大多数估算模型的经验数据都是从有限的项目样本中得出的。因此，还没有一种估算能够适用于所有的软件类型和开发环境。所以，从这些模型中得到的结果必须慎重使用。
- 应该对估算模型进行调整，以反映当前项目的情况。应该使用从已完成项目中收集的数据对该模型进行检验 - 方法是将数据代入到模型中，然后将实际结果与预测结果进行比较。如果两者一致性很差，则在使用该模型前，必须对其进行调整和再次检验。

经验估算模型

估算模型的结构

- 典型的估算模型是通过对以前的软件项目中收集到的数据进行回归分析而导出的。这种模型的总体结构具有下列形式[MAT94]：

$$E = A + B \times (e_v)^C \qquad (4.3)$$

  - 其中，A、B、C是经验常数，
  - E是工作量（以人·月为单位），
  - e_v是估算变量（LOC或FP）。

经验估算模型

估算模型的结构

- 除了公式(4.3)所表示的关系之外，大多数估算模型均有某种形式的项目调整成分，使得e能够根据其他的项目特性(如，问题的复杂性、开发人员的经验、开发环境等)加以调整：

$E = 5.2 \times (KLOC)^{0.91}$ Walston-Felix模型

$E = 5.5 + 0.73 \times (KLOC)^{1.16}$ Bailey-Basili模型

$E = 3.2 \times (KLOC)^{1.05}$ Boehm的简单模型

$E = 5.288 \times (KLOC)^{1.047}$ Doty模型，在KLOC > 9的情况下

同样，也提出了许多面向FP的估算模型。主要包括：

$E = -91.4 + 0.355FP$ Albrecht和Gaffney模型

$E = -37 + 0.96FP$ Kemerer模型

$E = -12.88 + 0.405FP$ 小型项目回归模型

从这些模型可以看出，对于相同的LOC或FP值，不同的模型会产生不同的结果。其含义很清楚，必须根据当前项目的需要对估算模型进行调整。

COCOMO（Constructive Cost model）

由Barry Boehm开发的
详见：www.usc.edu(南加州大学网站)

- **基本COCOMO**:静态单变量模型，它用一个以已估算出来的源代码行数(LOC)为自变量的函数来计算软件开发工作量。
- **中等COCOMO**：在用LOC为自变量的函数计算软件开发工作量的基础上，再用涉及产品、硬件、人员、项目等方面属性的影响因素来调整工作量的估算。
- **高级COCOMO**：包括中级COCOMO模型的所有特性，但用上述各种影响因素调整工作量估算时，还要考虑对项目过程中分析、设计等各步骤的影响。

COCOMO模型

模型的核心是方程

$$ED=rS^{c} \text{ 和 } TD=a(ED)b$$

其中:

ED为总的开发工作量（到交付为止），单位为人月；S为源指令数（不包括注释，但包括数据说明、公式或类似的语句），常数r和c为校正因子。S的单位为10^3，ED的单位为人月。

TD为开发时间，经验常数r、c、a和b取决于项目的总体类型（结构型、半独立型或嵌入型）.

项目总体类型

特性	结构型	半独立型	嵌入型
对开发产品目标的了解	充分	很多	一般
对软件系统有关的工作经验	广泛	很多	中等
为软件一致性需要预先建立的需求	基本	很多	完全
为软件一致性需要外部接口规格说明	基本	很多	完全
关联的新硬件和操作过程的并行开发	少量	中等	广泛
对改进数据处理体系结构算法的要求	极少	少量	很多
早期实施费用	极少	中等	较高
产品规模（交付的源指令数）	<5万行	<30万行	任意
实例	批数据处理 科学模块 事务模块 熟悉的操作系统， 编译程序 简单的编目生产控制	大型事务处理系统 新的操作系统数据库管理系统 大型编目生产控制 简单的指挥系统	大而复杂的事务处理系统 大型的操作系统 宇航控制系统 大型指挥系统

工作量和进度的基本COCOMO方程

开发类型	工作量	进度
结构型	$ED=2.4S^{1.05}$	$TD=2.5(ED)^{0.38}$
半独立型	$ED=3.0S^{1.12}$	$TD=2.5(ED)^{0.35}$
嵌入型	$ED=3.6S^{1.20}$	$TD=2.5(ED)^{0.32}$

影响经验常数r值的15个成本因素

类型	成本因素
产品属性	1.要求的软件可靠性 2.数据库规模 3.产品复杂性
计算机属性	4.执行时间约束 5.主存限制 6.虚拟机变动性 7.计算机周转时间
人员属性	8.分析人员能力 9.应用经验 10.程序设计人员能力 11.虚拟机经验 12.程序设计语言经验
工程属性	13.最新程序设计实践 14.软件开发工具的作用 15.开发进度限制

COCOMO举例

一个33.3 KLOC的软件开发项目，属于中等规模、半独立型的项目，采用基本COCOMO：

- a=3.0，b=1.12。
- E = 3.0 * L ^1.12 = 3.0 * 33.3 ^1.12 = 152 PM

经验估算模型

COCOMO Ⅱ模型

- COCOMO Ⅱ模型是一种层次结构的软件估算模型。主要应用于以下领域：
  - 应用组装模型。在软件工程的早期阶段使用，这时，用户界面的原型开发、对软件和系统交互的考虑、性能和评估以及技术成熟度的评价是最重要的
  - 早期设计阶段模型。在需求已经稳定且基本的软件体系结构已经建立时使用
  - 体系结构后阶段模型。在软件的构造过程中使用
- COCOMO Ⅱ模型也需要使用规模估算信息，在模型层次结构中有三种不同的规模估算选项：
  - 对象点
  - 功能点
  - 源代码行

经验估算模型

COCOMO Ⅱ模型

- COCOMO Ⅱ应用组装模型使用的是对象点。与功能点一样，对象点是一种间接的软件测量。
- 计算对象点时，使用如下的数值：
  ① （用户界面的）屏幕数
  ② 报表数
  ③ 构造应用可能需要的构件数
- 根据Boehm[BOE96]给出的标准，将每个对象实例（如一个屏幕或一个报表）归类到三个复杂度级别之一（即简单的、中等的或困难的）。本质上，复杂度是以下变量的函数：客户和服务器数据表（产生屏幕显示和报表需要它们）的数量和来源，以有视图或版面（是屏幕或报表的一部分）的数量。

经验估算模型

COCOMO Ⅱ模型

- 一旦确定了复杂度，就可以根据图3－6对屏幕、报表和构件的数量进行加权。首先将初始的对象实例数与表中的加权因子相乘就确定了对象点数，求和后就得到了总的对象点数。当采用基于构件的开发或一般的软件利用时，还要估算复用的百分比，并调整对象点数：

$$NOP = 对象点 \times [\ (\ 100\ -\ 复用的百分比)\ /\ 100\ ]$$

其中，NOP是新的对象点。

对象类型	复杂度加权		
	简单的	中等的	困难的
屏幕	1	2	3
报表	2	5	8
3GL构件			10

图3－6 不同对象类型的复杂度加权[BOE96]

经验估算模型

COCOMO Ⅱ模型

- 根据计算得到的NOP值进行工作量估算时，必须先确定“生产率”的值。图3－7中分别给出了在不同水平的开发者经验和开发环境成熟度下的生产率。

$$PROD = NOP \times 人\cdot月$$

- 一旦确定了生产率，就可以得到项目工作量的估算值：

$$估算工作量 = NOP / PROD$$

开发者的经验/能力	非常低	低	正常	高	非常高
环境成熟度/能力	非常低	低	正常	高	非常高
PROD	4	7	13	25	50

图3－7 应用于对象点的生产率[BOE96]

经验估算模型

软件方程

- 软件方程[PUT92]是一个动态的多变量模型，它假定在软件开发项目的整个生命周期中的有特定的工作量分布。该模型是从4000多个当代的软件项目中收集的生产率数据中导出的。基于这些数据，估算模型具有以下形式：

$$E = [(LOC \times B^{0.333}) / P^{3}] \times (1 / t^{4})$$

- 其中，
  - E为工作量，以人·月或人·年为单位。
  - t为项目持续时间，以月或年为单位。
  - B为“特殊技能因子”，它随着“对集成、测试、质量保证、文档、及管理技术的需求的增长”而缓慢增加。对于较小的程序(KLOC=5到15)，B=0.16。对于超过70KLOC的较大程序，B=0.39
  - P为“生产率参数”，它反映了：总体的过程成熟度及管理水平；采用良好的软件工程实践的程度；使用的程序设计语言的水平；软件环境的状态；软件团队的技能及经验；应用系统的复杂性。

经验估算模型

软件方程

- 软件方程[PUT92]是一个动态的多变量模型，它假定在软件开发项目的整个生命周期中的有特定的工作量分布。该模型是从4000多个当代的软件项目中收集的生产率数据中导出的。基于这些数据，估算模型具有以下形式：

$$E = [(LOC \times B^{0.333}) / P^{3}] \times (1 / t^{4}) \text{ 其中，}$$

  - E为工作量，以人·月或人·年为单位。
  - t为项目持续时间，以月或年为单位。
  - B为“特殊技能因子”，它随着“对集成、测试、质量保证、文档、及管理技术的需求的增长”而缓慢增加。对于较小的程序(KLOC=5到15)，B=0.16。对于超过70KLOC的较大程序，B=0.39
  - P为“生产率参数”，它反映了：总体的过程成熟度及管理水平；采用良好的软件工程实践的程度；使用的程序设计语言的水平；软件环境的状态；软件团队的技能及经验；应用系统的复杂性。

经验估算模型

软件方程

- 对于实时嵌入式软件的开发，典型值是P = 2000；
- 对于电信及系统软件，P = 10000；
- 而对于商业系统应用，P = 28000。
- 当前情况下的生产率参数可以根据以往开发工作中收集到的历史数据来导出。
- 重要的是要注意到，软件方程有两个独立的参数：
  - 规模的估算值（以LOC为单位）
  - 项目持续时间，以月或年为单位。



经验估算模型

软件方程

- 为了简化估算过程，并将该模型表示成更通用的形式，Putnam和Myers[PUT92]又提出了一组方程式，这些方程式都是从软件方式中导出来的。最短开发时间定义为：

$t_{min} = 8.14(LOC/P)^{0.43}$，以月为单位，用于$t_{min}$>6个月的情况（4.5a）

$E = 180Bt^3$，以人月为单位，用于E≥ 20人月的情况 （4.5b）

注意方程（4.5b）中的 t 是以年为单位的。

经验估算模型

软件方程

对前面所讨论的CAD软件，使用方程（4.5），令 P = 12 000（对科学计算软件的推荐值）：

$$tmin = 8.14\ (33200/12000)^{0.43}$$

$$tmin = 12.6\ (月)$$

$$E = 180 \times 0.28 \times (1.05)^3$$

$$E = 58\ (人\cdot月)$$

由软件方程得到的结果与分解技术小节产生的估算值非常一致。同上一节提到的COCOMO模型一样，软件方程已经演化了十多年。

经验估算模型

面向对象项目的估算

最好用明确为OO软件设计的估算方法对传统的软件成本估算方法加以补充：

1. 使用工作量分解、FP分析和任何其他适用于传统应用的方法进行估算。
2. 使用面向对象的分析模型建立用例并确定用例数。要认识到随着项目的进展，用例数可能会改变。
3. 由分析模型确定关键类（分析类）的数量



经验估算模型

面向对象项目的估算

最好用明确为OO软件设计的估算方法对传统的软件成本估算方法加以补充：

4．对应用 的界面类型进行归类，确定支持类的乘数：

界面类型	乘　数
没有图形用户界面（GUI）	2.0
基于文本的用户界面（GUI）	2.25
图形用户界面（GUI）	2.5
复杂的图形用户界面（GUI）	3.0

关键类的数量乘上乘数就得到了支持类数量的估算值。

经验估算模型

面向对象项目的估算

5. 将类的总数（关键类+支持类）乘以每个类的的平均工作单元数。建议每个类的平均工作单元数是15~20人·日。
6. 将用例数乘以每个用例的平均工作单元数，对基于类的估算做交叉检查。



小结

- 在项目开始之前，软件项目计划人员必须先估算三件事：需要多长时间、需要多少工作量、以及需要多少人员。此外，计划人员必须预测所需要的资源(硬件及软件)和蕴含的风险。
- 范围陈述能够帮助计划人员使用一种或多种技术进行估算，这些技术主要分为两大类：分解和经验建模。分解技术需要划分出主要的软件功能，接着估算：（1）LOC的数量；（2）信息域内的选择值；（3）用例的数量；（4）实现每个功能所需的人·月数；或者，（5）每个软件工程活动所需的人·月数。经验技术使用根据经验导出的关于工作量和时间的公式来预测这些项目数字，可以使用自动 工具来实现特定的经验模型。
- 对项目做精确估算时，一般至少会用到上述3种技术中的两种。通过对不同技术产生的估算值进行比较和调和，计划人员更有可能得到精确的估算。软件项目估算永远不会是一门精确的科学，但可靠的历史数据与系统化的技术结合起来能够提高估算的精确度。

第6章：风险分析和管理

基本概念


IDENTIFY
ASSESS
ANALYZE
RISK
MANAGEMENT
REDUCE
CONTROL
TRANSFER

基本概念

- 很多问题都会困扰软件项目，风险分析和风险管理就辅助软件团队理解和管理不确定事物的活动。
- 风险是潜在的 - 它可能发生也可能不发生。但是不管发生还是不发生，都应该去识别它，评估它发生的概率，估算它的影响，并制定它实际发生时的应急计划。
- 风险管理的步骤：（1）风险识别，即辨识出什么情况下可能会出问题。（2）风险分析：分析每个风险，确定其可能发生的概率以及发生时将带来的危害。了解这些信息之后，就可以按照可能发生的概率和危害程序对风险进行排序。（3）风险管理：制定一个计划来管理那些发生概率高和危害程度大的风险。
- 工作产品：风险缓解、监测和管理（risk mitigation, monitoring and management, RMMM）计划或一组风险信息表单

基本概念

- Robert Charette［CHA89］在他的关于风险分析及管理的书中给出了风险概念的定义是：

  首先，风险涉及的是未来将要发生的事情。今天和昨天已不再被关心，如同我们已经在收获由我们过去的行为所播下的种子。问题是：我们是否能够通过改变我们今天的行为，而为一个不同的、充满希望的、更美好的明天创造机会。其次，风险涉及改变，如思想、观念、行为、或地点的改变……第三，风险涉及选择，而选择本身就具有不确定性。因此，就象死亡和税收一样，风险是生活中最不确定的元素之一。

基本概

- Robert C 险概念的定义是：

首先，风 如同我们已经在 通过改变我们今天的 会。其次，风险涉及 及选择，而选择本 活中最不确定的元素



基本概念

对于软件工程领域中的，Charette的三个概念定义是显而易见的。

- 未来是我们所关心的——什么样的风险会导致软件项目彻底失败呢?
- 改变也是我们所关心的——用户需求、开发技术、目标计算机、以及所有其他与项目相关的因素的改变将会对按时交付和总体成功产生什么影响呢?
- 最后，我们必须抓住选择机会——我们应该采用什么方法及工具？需要多少人员参与工作？对质量的要求要达到什么程度才是“足够的”？按照以往项目的历史数据进行详细的估算。确定项目的估算工作量和工期。

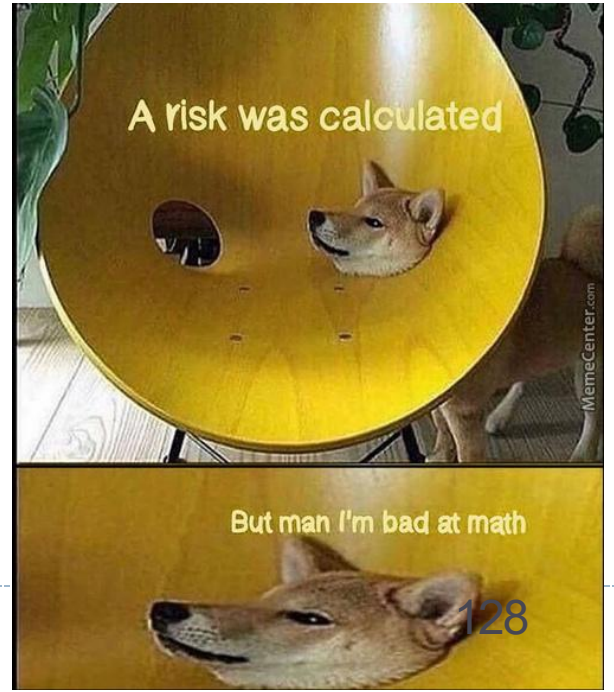


基本概念

- Peter Drucker［DRU75］曾经说过：

  “当没有办法消除风险，甚至连试图降低该风险也存在疑问时，这些风险就是真正的风险了”。

- 在我们能够标识出软件项目中的“真正风险”之前，识别出所有对管理者及开发者而言均为明显的风险是很重要的。

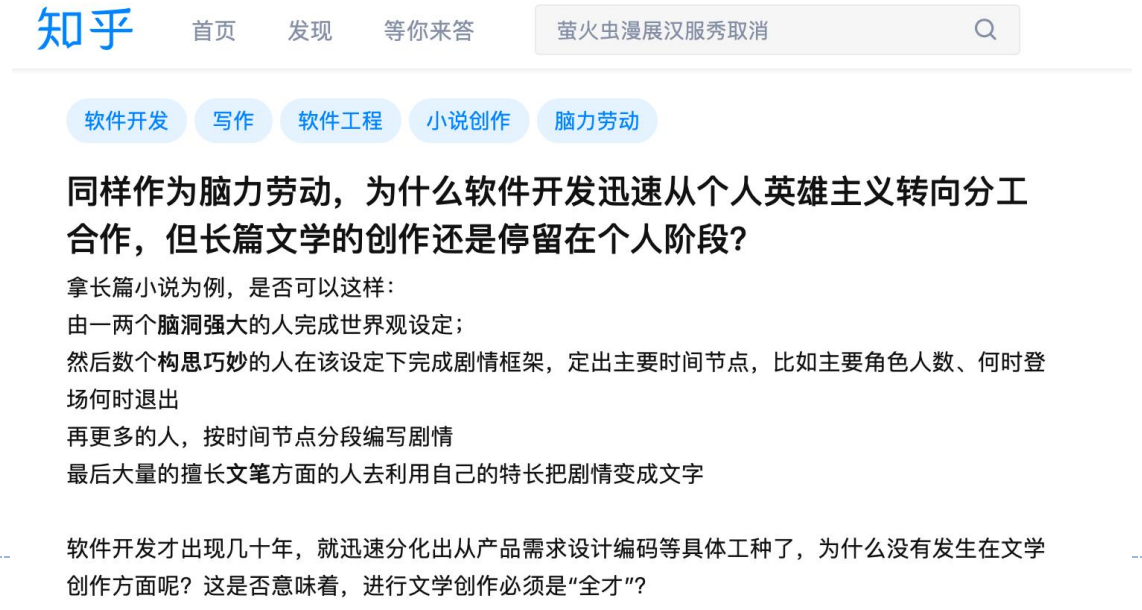


被动风险策略和主动风险策略

风险策略

被动风险策略

- 被动风险策略（reactive risk strategies）被戏称为“印地安那·琼斯学派的风险管理”［Tho92］。印地安那·琼斯在以其名字为影片名的电影中，每当面临无法克服的困难时，总是一成不变地说：“不要担心，我会想出办法来的！”。印地安那·琼斯从不担心任何问题，直到风险发生，再做出英雄式的反应。



https://www.zhihu.com/question/52668645/answer/131532940

风险策略

被动风险策略

- 被动风险策略（reactive risk strategies）被戏称为“印地安那·琼斯学派的风险管理”［Tho92］。印地安那·琼斯在以其名字为影片名的电影中，每当面临无法克服的困难时，总是一成不变地说：“不要担心，我会想出办法来的！”。印地安那·琼斯从不担心任何问题，直到风险发生，再做出英雄式的反应。

- 大多数软件项目团队还是仅仅依赖于被动的风险策略。被动策略最多不过是针对可能发生的风险来监测项目，直到风险发生时，才会拨出资源来处理它们。

- 大多数情况下，软件项目团队对于风险不闻不问，直到出现了问题。这时，项目团队才赶紧采取行动，试图迅速地纠正错误。这常常被称为“救火模式”（fire-fighting mode）。当这样的努力失败后，“危机管理”［Cha92］接管一切，这时项目已经处于真正的危机中了。

风险策略

主动风险策略

- 主动（proactive）风险策略早在技术工作开始之前就已经启动了。标识出潜在的风险，评估它们出现的概率及产生的影响，并按其重要性进行排序。
- 然后，软件项目团队就可以制定一个计划来管理风险。计划的主要目标是回避风险，但因为不是所有的风险都能够回避，所以项目团队必须制定一个应急计划，使其在必要时能够以可控和有效的方式作出反应。



软件风险

软件风险

虽然对于软件风险的严格定义还存在很多争议，但一般认为软件风险包含两个特性[Hig95]：

- 不确定性（uncertainty）。风险可能发生也可能不发生；即没有100%会发生的风险（100%发生的风险是加在项目上的约束）。
- 损失（loss）。如果风险发生，就会产生恶性后果或损失。

进行风险分析时，重要的是量化每个风险的不确定性程度和损失程度。为了实现这点，必须考虑不同类型的风险。

软件风险

- 不同类型的风险

- 项目风险（project risk）威胁到项目计划。也就是说，如果项目风险发生，就有可能会拖延项目的进度和增加项目的成本。项目风险是指预算、进度、人员(聘用职员及组织)、资源、利益相关者、需求等方面的潜在问题以及它们对软件项目的影响。在第4讲估算中，项目复杂度、规模、及结构不确定性也属于项目(和估算)风险因素。

- 技术风险（technical risk）威胁到要开发软件的质量及交付时间。如果技术风险发生，开发工作可能变得很困难或根本不可能。技术风险指设计、实现、接口、验证和维护等方面的问题。此外，规格说明的歧义性、技术的不确定性、陈旧的技术以及“前沿”技术也是技术风险因素。技术风险的发生是因为问题比我们所设想的更加难以解决。

软件风险

- 不同类型的风险

■ 商业风险（business risk）威胁到要开发软件的生存能力，且常常会危害到项目或产品。五个主要的商业风险是：

(1)开发了一个没有人真正需要的优秀产品或系统(市场风险)；

(2)开发的产品不再符合公司的整体商业策略(策略风险)；

(3)开发了一个销售部门不知道如何去销售的产品（销售风险）；

(4)由于重点的转移或人员的变动而失去了高级管理层的支持(管理风险)；

(5)没有得到预算或人员的保证(预算风险)。

应该注意到的很重要的一点是：简单的分类并不总是行得通。某些风险根本无法事先预测。

软件风险

- 不同类型的

■ 商业风险 害到项目或产品。

(1)开发了

(2)开发的

(3)开发了

(4)由于重);

(5)没有得

应该注意到的 本无法事先预测。



风险识别

引子

蜀国风险演绎及借鉴：刘备伐吴的三个风险

- 刘备为什么伐吴？关羽被东吴杀了。刘备伐吴，是蜀国国力衰退的开始，造成的不良影响是巨大的，集中体现了蜀国经营管理的三个风险。





引子

- 蜀国风险演绎及借鉴：刘备伐吴的三个风险

- 其一：声誉风险。什么是声誉风险？简单的说，就是大家不信任你了，认为你的产品有严重问题，你的社会责任感不行，你的经营不佳或者认为和你业务往来不安全，这样的情况下就会造成企业相当的损害。刘备是什么人？大汉皇叔。维护大汉正统是刘备的宗旨，也是刘备的立国根本。可是刘备现在为了兄弟私义，抛下大义去攻打吴国。刘备当时他还有更重要的大义要办，即兴汉伐魏。但现在刘备却念念不忘的是君臣之义，不顾大义，这就使得大家对蜀国的社会责任感有了看法，信任度下降，可以说这件事使蜀国的声誉受到很大的损害，蜀国的归心力也因此减退。正如赵云所说的 “汉贼之仇，公也；兄弟之仇，私也。愿以天下为重。” 换成白话文的意思就是，“你不要公私不分，要打你就应该打魏国” 。

引子

- 蜀国风险演绎及借鉴：刘备伐吴的三个风险

- 其二、长期战略风险。什么是战略风险？是指在追求短期目的和长期发展目标的系统化管理过程中，不适当的未来发展规划和战略决策可能威胁未来发展的潜在风险。
- 蜀国的长期战略是什么？这在孔明出山的时候就确定了---三分天下，联孙抗操。这个战略无比正确，使刘备从颠沛流离的困境中一步步发展起来，并最终建立蜀国。当时刘、孙弱而曹操强，只有两弱联合才能抗一强，进而在三国鼎立的情况下，蜀必须联合吴伐魏才能完成战略的延伸——统一天下，蜀想自保不被灭亡也必须得联合吴，才不至于受到两面夹击。刘备改变了这个经多年实践考验的长期战略，作出了全力伐吴的最高决策，最终吴蜀夷陵之战再次以蜀的彻底惨败告终！吴虽然获胜，但也消耗不少，已经无多少军事兵力争衡中原了，再者有保守政策的束缚，两国只能眼睁睁的看着魏国坐大。蜀国伐吴决策动摇了三国制衡基础，导致蜀国发展长期不利。
- 战败后，刘备觉悟自己错误的决策，使他重新认识到联盟的重要性，所以重新联盟，又回到了原来的决策上，但巨大的损失已无法弥补。战略风险在蜀国身上体现的如此淋漓尽致。

引子

- 蜀国风险演绎及借鉴：刘备伐吴的三个风险

- 其三、法人治理结构不完善。刘备伐吴，大部分人都认为是错误的，但谁劝刘备也不听。赵云力劝不听；孔明多次苦谏，无可奈何；秦宓陈天时必无其利，被坐下狱幽闭。刘备坚持自己的主见，不从大局执意伐吴！项目负责人一家独大，刚愎自用，自行其事，这对企业经营是非常不利的，一个不好就关门倒闭了。
- 不能否认刘备是一个非常出色的领导人，具有相当强的领导能力，但在军事上的才能却有不足。也许刘备经过多年征战，积累了不少经验，同时当时公司又发展的不错（都建国了），一时自信心急剧膨胀，想更大的实现自我价值，才亲自领兵出征。理想是好的，结果比较伤感。联营七百里是低级军事错误，连孔明先生都说，谁这样做的，该杀。

风险识别

- 风险识别试图系统化地指出对项目计划（估算、进度、资源分配等）的威胁。识别出已知的和可预测的风险后，项目管理者首先要做的是：在可能时避免这些风险，在必要时控制这些风险。
- 上一小节提出的每一类风险又可以分为两个不同的类型：一般性风险和产品特定的风险。
  - 一般性风险（generic risk）对每一个软件项目而言都是一个潜在的威胁。
  - 产品特定的风险（product-specific risk）只有那些对当前项目特定的技术、人员及环境非常了解的人才能识别出来。为了识别产品特定的风险，必须检查项目计划及软件范围说明，然后回答这个问题：“本产品中有什么特殊的特性可能会威胁到我们的项目计划？”

风险识别

识别风险的一种方法是建立风险条目检查表。该检查表可以用于识别风险，并且主要用来识别下列几种类型中的一些已知的及可预测的风险：

- 产品规模——与要开发或要修改的软件的总体规模相关的风险。
- 商业影响——与管理者或市场所施加的约束相关的风险。
- 客户特性——与客户的素质以及开发者和客户定期沟通的能力相关的风险。
- 过程定义——与软件过程定义的程度以及该过程被开发组织遵守的程度相关的风险
- 开发环境——与用来开发产品的工具的可用性及质量相关的风险。
- 开发技术——与待开发软件的复杂性及系统所包含技术的“新奇性”相关的风险。
- 人员数目及经验——与软件工程师的总体技术水平及项目经验相关的风险。

风险识别

风险条目检查表可以采用不同的方式来组织。与上述每个主题相关的问题针对每一个软件项目来回答。有了这些问题的答案，项目管理者就可以估计风险产生的影响。也可以采用另一个不同的风险条目检查表，即仅仅列出与每一个常见子类型有关的特性。最终，给出一组“风险因素和驱动因子”［AFC88］以及它们发生的概率。

风险r_i	损失概率l_i	损失大小x_i（周）	期望风险yi（周）
计划过于乐观	50%	5	2.5
自动从主机更新数据的额外需求	5%	20	1.0
由市场部门增加额外的功能（特指未知的）	35%	8	2.8
图形格式子系统接口不稳定	25%	4	1.0
设计欠佳——需要重新设计	15%	15	2.25
项目审批花费时间比预期的要长	25%	4	1.0
设施未能及时到位	10%	2	0.2
为管理层撰写进程报告占用开发人员的时间比预期的多	10%	1	0.1
签约商的图形格式子系统推迟交付	10-20%	4	0.4-0.8
新的编程工具没有节省预期的时间	30%	5	1.5

风险识别

评估整体项目风险

下面的提问来源于对世界各地有经验的软件项目管理人员的调查而得到的风险资料，根据各个问题对项目成功的相对重要性将问题进行了排序：

1. 高层的软件管理者和客户管理者已经正式承诺支持该项目了吗?
2. 最终用户对项目和待开发的系统或产品热心支持吗?
3. 软件工程团队及其客户充分理解需求了吗?
4. 客户已经完全地参与到需求定义中了吗?
5. 最终用户的期望现实吗?
6. 项目范围稳定吗?
7. 软件工程团队的技能搭配合理吗?
8. 项目需求稳定吗
9. 项目团队对将实现的技术有经验吗?
10. 项目团队的人员数量满足项目需要吗?
11. 所有的客户或用户对项目的重要性和待开发的系统或产品的需求有共识吗?

如果对这些问题的任何一个回答是否定的，则应务必启动缓解、监测和管理风险的步骤。项目的风险程度与对这些问题否定回答的数量成正比。

风险识别

风险因素和驱动因子

风险因素包括：性能、成本、支持和进度。在这里，风险因素是以如下的方式定义的：

- 性能风险——产品能够满足需求且符合其使用目的不确定程度。
- 成本风险——能够维持项目预算的不确定程度。
- 支持风险——开发出软件易于纠错、修改及升级的不确定程度。
- 进度风险——与能够维持项目进度且按时交付产品的不确定程度。

每一个风险驱动因子对风险因素的影响均可分为四个影响类别：
可忽略的、轻微的、严重的或灾难的。

图6－1指出了由于未识别出的软件失误而产生的潜在影响（标号为1的行），或没有达到预期的结果所产生的潜在影响（标号为2的行）。影响类别的选择是以最符合表中描述的特征为基础的。

影响类别 \ 风险因素		性能	支持	成本	进度
灾难的	1	无法满足需求而导致任务失败		失误将导致进度延迟和成本增加，预计超支$500K	
	2	严重退化使得根本无法达到要求的技术性能	无法做出响应或无法支持的软件	资金严重短缺，很可能超出预算	无法在交付日期内完成
严重的	1	无法满足需求而导致系统性能下降，使得任务能否成功受到质疑		失误将导致系统运行的延迟并使成本增加，预计超支$100K到$500K	
	2	技术性能有些降低	在软件修改中有少量的延迟	资金不足，可能会超支	交付日期可能延迟
轻微的	1	无法满足需求而导致次要任务的降级		成本、影响和（或）可以补救的进度延误上的小问题，预计超支$1K到$100K	
	2	技术性能有些降低	较敏感的软件支持	有充足的资金来源	现实的、可完成的进度计划
可忽略的	1	无法满足需求而导致使用不方便或不易操作		失误对进度和（或）成本的影响很小，预计超支少于$1K	
	2	技术性能没有降低	易于进行软件支持	可能低于预算	交付日期将会提前

注：(1) 未识别出的软件失误或缺陷所产生的潜在影响
(2) 如果没有到达预期的结果所产生的潜在影响

图6－1 影响评估

风险预测

风险预测

- 风险预测（risk projection），又称风险估计（risk estimation），试图从两个方面评估每一个风险：
  （1）风险发生的可能性或概率，
  （2）如果风险发生，风险相关问题产生的后果。



风险预测

- 项目计划人员，其他管理人员和技术人员都要进行以下4步风险预测活动：
  （1）建立一个尺度，以反映风险发生的可能性；
  （2）描述风险产生的后果；
  （3）估算风险对项目及产品的影响；
  （4）标明风险预测的整体精确度，以免产生误解。
- 按此步骤进行风险预测，目的是使我们可以按照优先级来考虑风险。任何软件团队都不可能以两样的严格程序来为每个可能的风险分配资源，通过将风险按优先级排序，软件团队可以把资源分配给那些具有最大影响的风险。

风险预测

建立风险表

- 风险表给项目管理者提供了一种简单的风险预测方法（风险表可以采用电子表格模式来实现，使表中的条目易于操作和排序）。

风　险	风险类型	概　率	影响值	RMMM
规模估算可能很不正确	PS	60%	2	
用户数量大大超出计划	PS	30%	3	
复用程度低于计划	PS	70%	2	
最终用户抵制该系统	BU	40%	3	
交付日期将提前	BU	50%	2	
资金将会流失	CU	40%	1	
用户将改变需求	PS	80%	2	
技术达不到预期的效果	TE	30%	1	
缺少对工具的培训	DE	80%	3	
人员缺乏经验	ST	30%	2	
人员变动比较频繁	ST	60%	2	
•				
•				
•				

影响值：
1 — 灾难的
2 — 严重的
3 — 轻微的
4 — 可忽略的

风险类型：
PS — 产品规模
BU — 商业影响
CU — 客户特性
TE — 开发技术
DE — 开发环境
ST — 人员

图6 - 2 排序前的风险表样本

风险预测

建立风险表

- 项目管理者首先要在表中的第一列列出所有风险(不管多么细微)。这可以利用风险条目检查表来完成。
- 在第二列中给出每一个风险的类型（如，PS指产品规模风险，BU指商业风险）。
- 在第三列中填入各个风险发生的概率。每个风险的概率值可以由团队成员各自估算，然后按循环投票的方式进行，直到大家对风险概率的评估值趋于接近为止。
- 下一步是评估每个风险所产生的影响。使用图6-1所示的特性评估每个风险因素，并确定其影响的类别。将4个风险因素——性能、支持、成本、及进度——的影响类别求平均可得到一个整体的影响值（如果某个风险因素对项目来说比较重要，可以使用加权平均法）。

风险预测

建立风险表

- 完成了风险表的前4列内容之后，就可以按照概率及影响值来进行排序。高概率、高影响的风险放在表的上方，而低概率风险则移到表的下方。这样就完成了第一次风险排序。
- 项目管理者研究排序后的表，并定义一条中截线。该中截线（表中某处之上的一条水平线）表示：
  只有那些在中截线之上的风险才会得到进一步的关注，而在线之下的风险则需要重新评估以进行第二次排序。

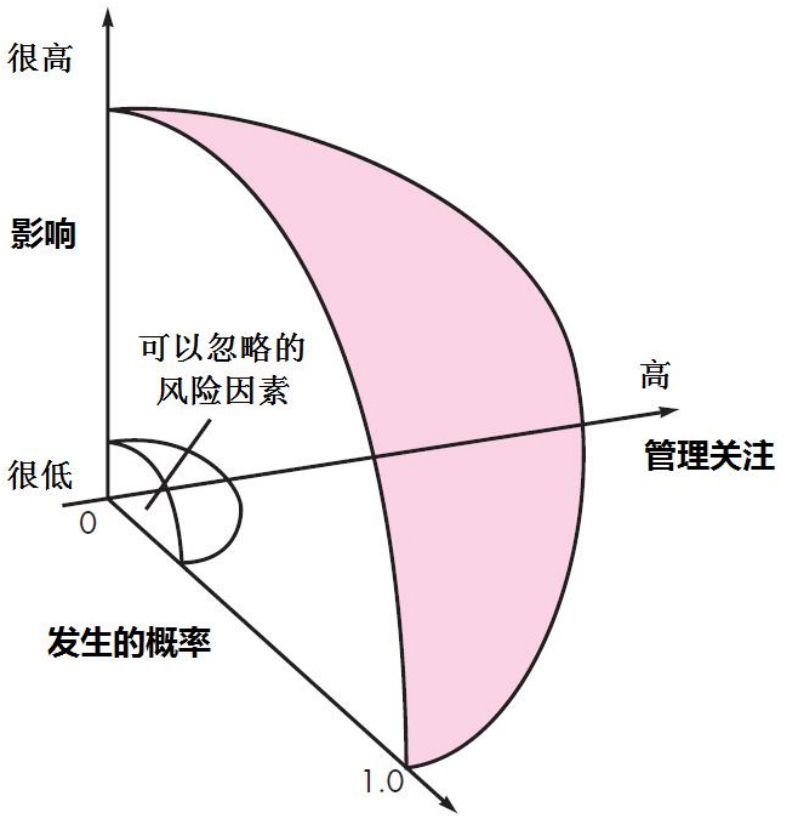


图6 - 3 风险与管理

风险预测

建立风险表

- 参照图6－3，从管理关注的角度来看，风险的影响和发生的概率是截然不同的。一个具有高影响但发生概率很低的风险因素不应该耗费太多的管理时间。而高影响且发生概率为中到高的风险、以及低影响且高概率的风险，应该首先列入随后的风险分析步骤中。

- 所有在中截线之上的风险都必须进行管理。
  标有RMMM的列中包含了一个指示器，指向为所有中截线之上的风险所建立的风险缓解、监测及管理计划（Risk Mitigation，Monitoring and Management Plan，RMMM计划）或一组风险信息表单。

风险预测

建立风险表

- 风险概率可以通过先做个别估计而后求出一个有代表性的值来确定。
- 虽然该方法是可行的，不过还有很多其他更复杂的确定风险概率的技术［AFC88］
- 风险驱动因子的评估是基于定性的概率等级，可表示为：不可能、不一定、可能和极可能，
- 然后再将每一个定性值概率等与数学上的概率值相关联（如，概率为0.7到0.99表示极可能发生的风险）。



风险预测

评估风险影响

- 如果风险真的发生了，有三个因素可能会影响风险所产生的后果，即风险的性质、范围和时间。
- 风险的性质是指当风险发生时可能产生的问题。例如，一个定义得很差的与客户硬件的外部接口(技术风险)会妨碍早期的设计及测试，也有可能导致项目后期阶段的系统集成问题。
- 风险的范围包括风险的严重性（即风险有多严重）及风险的整体分布情况（项目中有多少部分受到影响或有多少客户受到损害？）。
- 风险的时间是指何时能够感受到风险及风险的影响会持续多长时间。在大多数情况下，项目管理者希望“坏消息”越早出现越好，但在某些情况下，则是越迟越好。

风险预测

评估风险影响

- 风险分析方法。以下的步骤被建议用来确定风险的整体影响：
  1. 确定每个风险因素发生的平均概率。
  2. 使用图6 - 1中列出的标准来确定每个因素的影响。
  3. 按照前面几节给出的方法填写风险表，并分析其结果。

- 整体的风险显露度（risk exposure,RE）可由下面的关系确定：

$$\boldsymbol{RE = P \times C}$$

其中P是风险发生的概率，C是风险发生时带来的项目成本

风险预测

评估风险影响

- 例如，假设软件团队按如下方式定义项目风险：
  - 风险识别。事实上，计划可复用的软件构件中只有70%将集成到应用系统中，其他功能必须定制开发。
  - 风险概率。80%（大约）
  - 风险影响。计划了60个可复用的软体构件，如果只能利用70%，则18%构件必须从头开发（除了已经计划开发的定制软件外）。平均每个构件的程序行数是100LOC，本地数据表明每个LOC的软件工程成本是$14.00，开发构件的总成本（影响）将是$\mathbf{18 \times 100 \times 14 = \$25\ 200}$
  - 风险显露度。$\boldsymbol{RE = 0.80 \times \$25\ 200 \approx \$20\ 200}$

风险预测

评估风险影响

- 风险的成本估算完成之后，就可以为风险表中的每个风险计算其风险显露度。
- 所有风险（风险表中截线之上）的总体风险显露度不仅为调整项目最终的成本估算提供了依据，还可预测在项目进展过程中不同阶段所需人员的资源增长情况。
- 在“建立风险表”和“评估风险影响”小节中所述的风险预测和分析方法可以在软件项目进展过程中反复运用。项目团队应该定期复查风险表，重新评估每一个风险，以确定新环境是否引起其概率和影响发生改变。这样可能需要在风险表中添加一些新的风险，删除一些不再有影响的风险，或改变一些风险的相对位置。



风险求精

风险求精

- 在项目计划的早期，风险很可能只是一个大概的描述。随着时间的推移，对项目和风险的了解加深，可以将风险精华为一组更详细的风险，在某种程度上，这些风险更易于缓解、监测和管理。

  - 实现方法之一是按条件 - 转化 - 结果（condition-transition-consequence,CTC）格式来表示风险，即采用如下方式来描述风险：

    给定<条件>，则（可能）将导致：<结果>

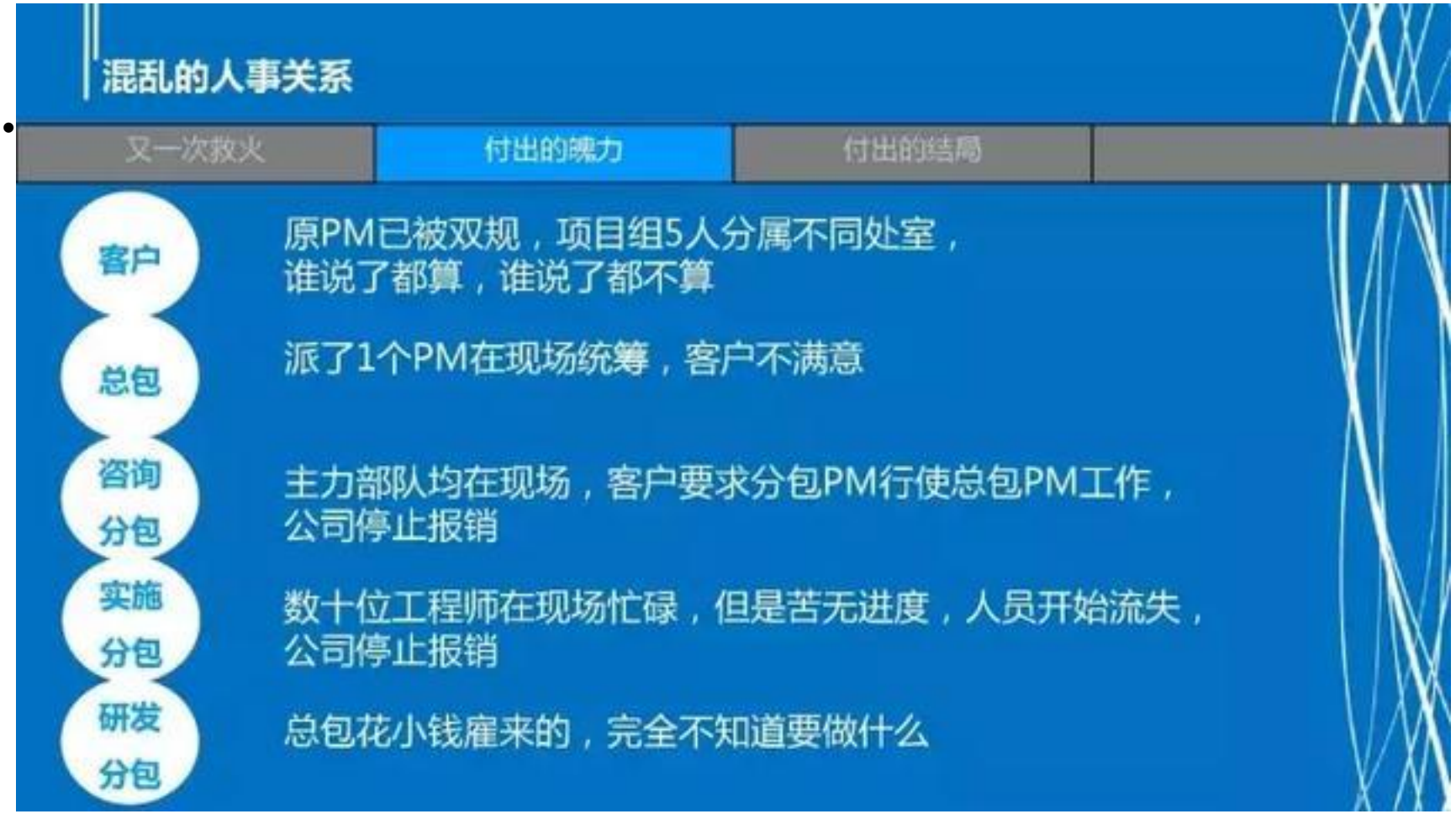
  - 使用CTC格式，在“评估风险影响”节中提到的可复用软件构件的风险可描述为：

    给定条件：所有可复用软件构件必须符合特定设计标准，但是某些并不符合；则有结论：（可能）仅70%的计划可复用构件将集成到最终的系统中，需定制开发剩余30%的构件。

风险求精


资源耗尽，偏偏自己也不行
一次救火
恶劣的背景
筹码永远都不够
1个人 VS 客户+领导
没有人
没有钱
客户要求很高
领导在搞啥
老板求赐个人
最大的几个问题
客户在领导面前把我们吹上天，但东西根本做的不能看
项目人天已经超出，客户认为做了3成都不到
内忧外患，项目没有人，还被说成本
缺乏满足客户条件的人，包括自己

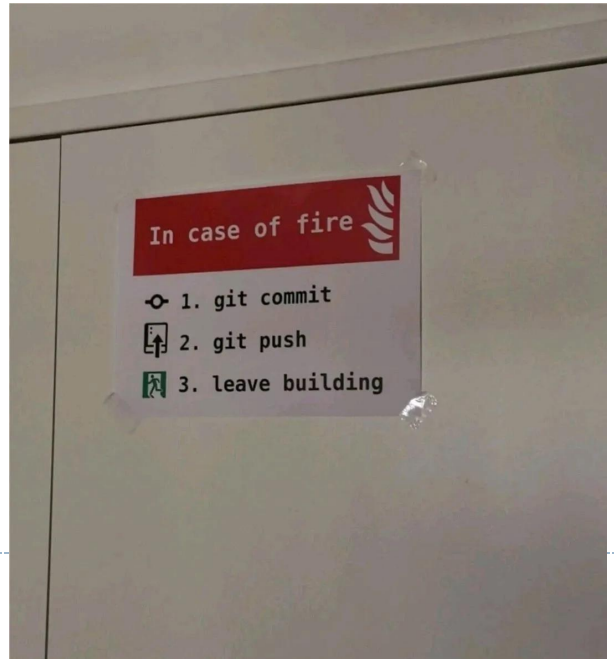
风险求精


混乱的人事关系
又一次救火
付出的魄力
付出的结局
客户
总包
咨询分包
实施分包
研发分包
原PM已被双规，项目组5人分属不同处室，谁说了都算，谁说了都不算
派了1个PM在现场统筹，客户不满意
主力部队均在现场，客户要求分包PM行使总包PM工作，公司停止报销
数十位工程师在现场忙碌，但是苦无进度，人员开始流失，公司停止报销
总包花小钱雇来的，完全不知道要做什么

风险求精

- 可按如下方式对这个一般条件进行求精：
- 子条件1
  某些可复用构件是由第三方开发的，没有其内部设计标准相关资料。
- 子条件2
  构件接口的设计标准尚未确定，有可能和某些现有的软件可复用构件不一致
- 子条件3
  某些可复用构件是采用不支持目标环境的语言开发的。
- 这些子条件的结论是相同的（即必须定制开发30%的软件构件），但求精可以帮助我们排除重大风险，使我们更易于分析风险和采取措施。

风险缓解、监测和管理



风险缓解、监测和管理

- 所有风险分析活动都只有一个目的：辅助项目团队制定处理风险的策略。
- 一个有效的策略必须考虑三个问题：
  - ■风险回避
  - ■风险监测
  - ■风险管理及应急计划

魏文王问扁鹊曰：「子昆弟三人其孰最善为医？」扁鹊曰：「长兄最善，中兄次之，扁鹊最为下。」魏文侯曰：「可得闻邪？」扁鹊曰：「长兄於病视神，未有形而除之，故名不出於家。中兄治病，其在毫毛，故名不出於闾。若扁鹊者，鑱血脉，投毒药，副肌肤，闲而名出闻於诸侯。」

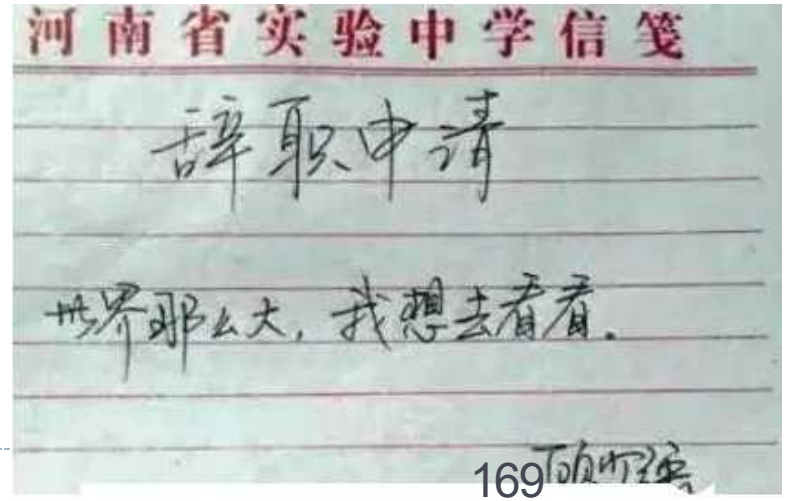
风险缓解、监测和管理

- 如果软件项目团队采取主动的方法，最好的策略就是风险回避。这可以通过建立一个风险缓解（risk mitigation）计划来达到。
- 例如，假设频繁的人员流动被标注为一个项目风险 r_1。基于以往的历史及管理经验，可以估计人员频繁变动的概率l_1为0.7（70%，相当高），并预测影响x_1为严重的。也就是说，频繁的人员变动将对项目成本及进度有严重的影响。
- 为了缓解这个风险，项目管理者必须制定一个策略来减少人员变动。可能采取的步骤包括：（下一页继续）

风险缓解、监测和管理

- 与现有人员一起探讨人员变动的起因（如恶劣的工作条件、报酬低...）
- 在项目开始之前采取行动，设法缓解那些我们能够控制的起因
- 项目启动后，假设会发生人员变动，找到能够保证工作连续性的方法
- 组织项目团队，使用得每一个开发活动的信息能被广泛传播和交流
- 制定工作产品标准，并建立相应机制以确保能够及时创建所有的模型和文档
- 给每个关键的技术人员都指定一个后备人员

```
/*
 * TODO 在下一个版本优化这里
 * TODO 产品经理已经跑路了，所以，我已经忘了要优化什么，好运。
 * Fuck
 */
```



风险缓解、监测和管理

- 随着项目的进展，风险监测（risk-monitoring）活动开始了。项目管理者应该监测那些可以表明风险是否正在变高或变低的因素。
- 在频繁的人员变动的例子中，应该监测：团队成员对项目压力的普遍态度、团队的凝聚力、团队成员彼此之间的关系、与报酬和利益相关的潜在问题、在公司内及公司外工作的可能性。
- 还应该监测风险缓解步骤的效力。例如，前述的风险缓解步骤中要求制定工作产品标准，并建立相应机制以确保能够适时开发出工作产品。万一有关键成员离开此项目，有一个保证工作连续性的机制。项目管理者应该仔细监测这些工作产品，以保证每一个工作产品的正确性，在项目进行中有新员工加入时，能为他们提供必要的信息。

风险缓解、监测和管理

- 风险管理应急计划是以缓解工作已经失败、而且风险已经发生为先决条件的。
- 继续前面的例子，假定项目正在进行之中，有一些人宣布将要离开。如果已经按照缓解策略行事，则有后备人员可用，信息已经文档化，有关知识已经在团队中广泛进行了交流。
- 此外，对那些人员充足的岗位，项目管理者还可以暂时重新调整资源（并重新调整项目进度），从而使得新加入团队的人员能够“赶上进度”。
- 同时，应该要求那些将要离开的人员停止所有的工作，并在最后几星期进入“知识交接模式”。比如：得到视频知识，建立“注释文档或Wiki”和（或）与仍留在团队中的成员进行交流。

风险缓解、监测和管理

什么是救火？

救火的定义 | 救火的产生原因 | 救火的基本原则 | 救火之道

当有一天，正在忙碌的你接到老板邮件，把你派去一个听上去就不靠谱的项目；

你惊讶的听同事说，这个项目谁去谁死；

进了项目你更惊讶，进度不到一半，但钱用的差不多了，而人，基本处于无序状态；

客户直接投诉给大老板，销售说搞不定就影响其他项目，老板用亲切（？）的目光注视着你；

当你看到一堆连垃圾都称不上的交付成果，心中默念着EdwardYourdon定义的死亡之旅；

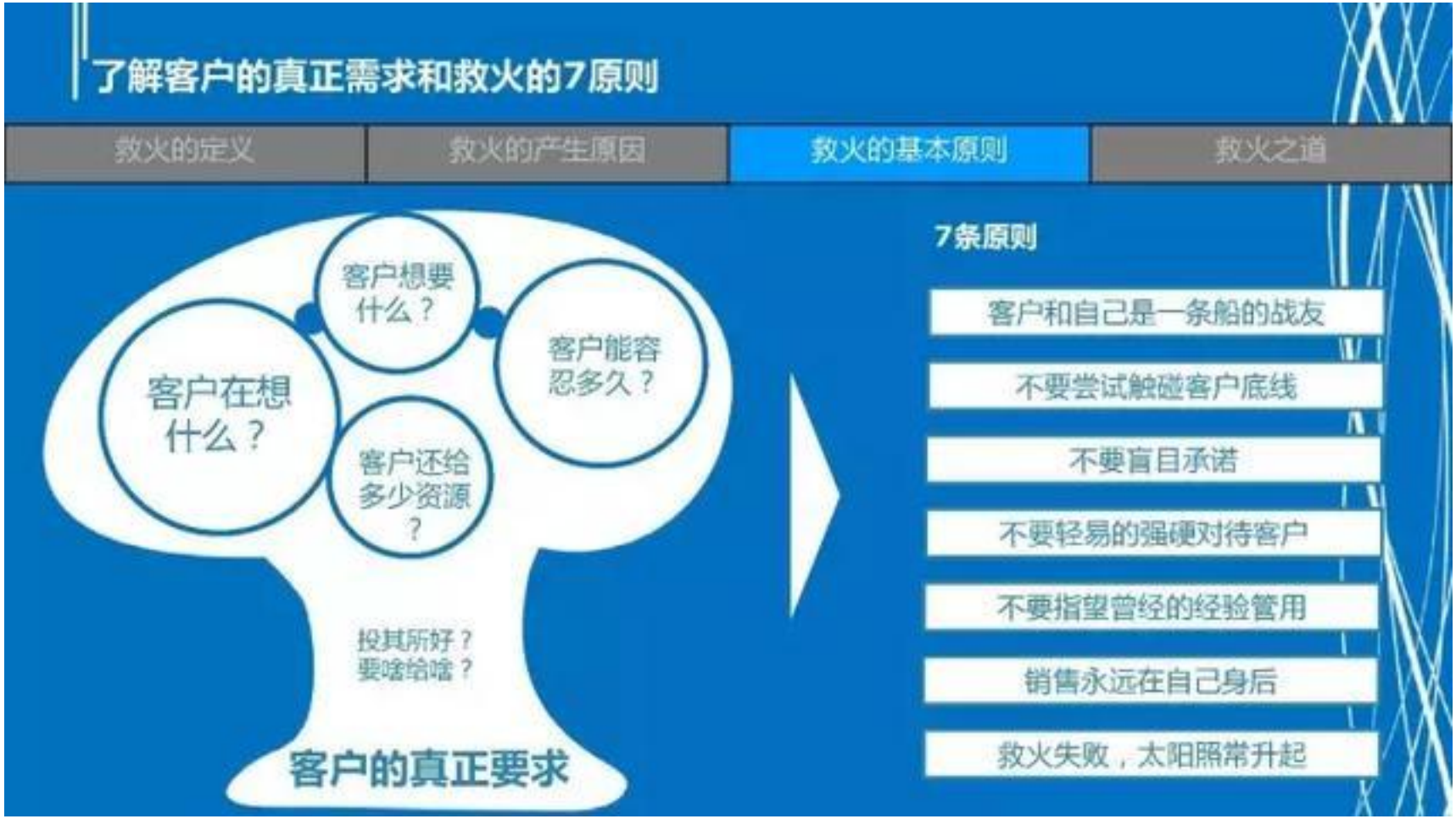
恭喜你，这就是救火了，而你是否愿意，都是一名**救火队员**。



风险缓解、监测和管理



风险缓解、监测和管理


了解客户的真正需求和救火的7原则
救火的定义
救火的产生原因
救火的基本原则
救火之道
客户想要什么？
客户在想什么？
客户能容忍多久？
客户还给多少资源？
投其所好？要啥给啥？
客户的真正要求
7条原则
客户和自己是一条船的战友
不要尝试触碰客户底线
不要盲目承诺
不要轻易的强硬对待客户
不要指望曾经的经验管用
销售永远在自己身后
救火失败，太阳照常升起

风险缓解、监测和管理

- 值得注意的是，RMMM步骤会导致额外的项目成本
  - 例如，花时间给每个关键技术人员配备“后备人员”得承担费用
- 因此，风险管理的另一个任务就是评估什么情况下由RMMM步骤所产生的效益高于实现这些步骤所花需的成本。通常，项目管理者要进行典型的成本/效益分析
  - 如果频繁的人员变动风险的缓解步骤经评估将会增加15%的项目成本和工期，而主要的成本因素是“配备后备人员”，则管理者可能决定不执行这一步骤
  - 另一方面，如果风险缓解步骤经预测仅增加5%的项目成本及3%的工期，则管理者极有可能将这一步骤付诸实现

风险缓解、监测和管理

- 值得注意的是，RMMM步骤会导致额外的项目成本
  - 例如，花时间给每个关键技术人员配备“后备人员”得承担费用
- 因此，风险管理的另一个任务就是评估什么情况下由RMMM步骤所产生的效益高于实现这些步骤所花需的成本。通常，项目管理者要进行典型的成本/效益分析
  - 如果频繁的人员变动风险的缓解步骤经评估将会增加15%的项目成本和工期，而主要的成本因素是“配备后备人员”，则管理者可能决定不执行这一步骤
  - 另一方面，如果风险缓解步骤经预测仅增加5%的项目成本及3%的工期，则管理者极有可能将这一步骤付诸实现

风险缓解、监测和管理

- 对于大型项目，可以识别出30或40种风险。如果为每一个风险制定3~7个风险缓解步骤，则风险管理本身就可能变成一个“项目”！因此，项目管理者可以将Pareto 的 80—20 规则用于软件风险上。
- 经验表明，整个项目的80%风险（即可能导致项目失败的80%的潜在因素）可能是由只占20%的已经识别出的风险所引发。
- 早期风险分析步骤中所做的工作能够帮助项目管理者确定哪些风险在这20%中（如果导致高风险显露度的风险）。
- 因此，某些已经识别、评估和预测过的风险可能并不纳入RMMM计划之中——这些风险不属于那关键的20%（具有最高项目优先级的风险）。

风险缓解、监测和管理

- 风险并不仅限于软件项目本身。在软件已经被成功开发并交付给客户之后，仍有可能发生风险。这些风险一般与该领域中的软件缺陷相关。
- 软件安全和危险分析是一种软件质量保证活动，主要用来识别和评估可能对软件产生负面影响并使整个系统失败的潜在灾难。
- 如果能够在软件工程的早期阶段识别灾难，就可以安排软件设计特征来消除或控制这些潜在的危险。



RMMM 计划

RMMM计划

- 风险管理策略可以包含在软件项目计划中，也可以将风险管理步骤组织成一个独立的风险缓解、监测和管理计划（RMMM计划）。
- RMMM计划将所有风险分析工作文档化，项目管理者还可将其作为整个项目计划的一部分。
- 有些软件团队并不建立正式的RMMM文档，而是将每个风险分别使用风险信息表单（risk information sheet, RIS）进行文档化。在大多数情况下，RIS采用数据 库系统进行维护，这样容易完成创建、信息输入、优先级排序、查找以及其他分析。

RMMM计划

- RIS的格式如图6 - 4所示

风险信息表单			
风险标识号：P02-4-32	日期：5/9/09	概率：80%	影响：高
描述 事实上，计划可复用的软件构件中只有70%将集成到应用系统中，其他功能必须定制开发。			
精化/环境 子条件1：某些可复用构件是由第三方开发的，没有其内部设计标准相关资料。 子条件2：构件接口的设计标准尚未确定，有可能和某些现有的软件可复用构件不一致。 子条件3：某些可复用构件是采用不支持目标环境的语言开发的。			
缓解/监测 1. 与第三方交流以确定其与设计标准的符合程度。 2. 强调接口标准的完整性，在确定接口协议时应考虑构件的结构。 3. 检查并确定属于子条件3的构件数量，检查并确定是否能够获得语言支持。			
管理/应急计划/触发 RE的计算结果为$20 200。在项目应急计划中分配这些费用。修订进度表，假定必须定制开发18个附加构件，据此分配人员。 触发：缓解步骤自7/1/09起没有效果。			
当前状态 5/12/09：缓解步骤启动。			
创建者：D.Gagne	受托者：B.Laster		

图6 - 4 风险信息表单

Risk information sheet			
Risk ID: P02-4-32	Date: 5/9/09	Prob: 80%	Impact: high
Description: Only 70 percent of the software components scheduled for reuse will, in fact, be integrated into the application. The remaining functionality will have to be custom developed.			
Refinement/context: Subcondition 1: Certain reusable components were developed by a third party with no knowledge of internal design standards. Subcondition 2: The design standard for component interfaces has not been solidified and may not conform to certain existing reusable components. Subcondition 3: Certain reusable components have been implemented in a language that is not supported on the target environment.			
Mitigation/monitoring: 1. Contact third party to determine conformance with design standards. 2. Press for interface standards completion; consider component structure when deciding on interface protocol. 3. Check to determine number of components in subcondition 3 category; check to determine if language support can be acquired.			
Management/contingency plan/trigger: *RE* computed to be $20,200. Allocate this amount within project contingency cost. Develop revised schedule assuming that 18 additional components will have to be custom built; allocate staff accordingly. Trigger: Mitigation steps unproductive as of 7/1/09.			
Current status: 5/12/09: Mitigation steps initiated.			
Originator: D. Gagne		Assigned: B. Laster	

RMMM计划

- 一旦建立了RMMM计划，且项目已经启动之后，风险缓解及监测步骤也就开始了
- 正如前面讨论过的，风险缓解是一种问题规避活动，而风险监测则是一种项目跟踪活动，这种监测活动有三个主要目的：
  （1）评估所预测的风险是否真正发生了；
  （2）保证正确地实施了各风险缓解步骤；
  （3）收集能够用于今后风险分析的信息。
- 在很多情况下，项目中发生的问题可以追溯到不止一个风险，所以风险监测的另一个任务就是试图找到“起源”（在整个项目中是哪些风险引起了哪些问题）。

小结

- 对软件项目期望很高时，一般都会进行风险分析。不过，即使进行这项工作，大多数软件项目管理者都是非正式地和表面上地完成它。花在识别、分析和管理风险上的时间可以从多个方面得到回报：更加平稳的项目进展过程；较高的跟踪和控制项目的能力；在问题发生之前已经做了周密计划而产生的信心。
- 风险分析需要占用大量项目计划的工作量。识别、预测、评估、管理和监测都要花费时间。但这是值得的。引用中国2500多年前的军事家孙子的一句话：“知己知彼，百战不殆”。对于软件项目管理者而言，这个“彼”指的就是风险。

小结

- 对软件项目期望很高时，一般都会进行风险分析。不过，即使进行这项工作，大多数软件项目管理者都是非正式地和表面上地完成它。花在识别、分析和管理风险上的时间可以从多个方面得到回报：更加平稳的项目进展过程；较高的跟踪和控制项目的能力；在问题发生之前已经做了周密计划而产生的信心。
- 风险分析需要占用大量项目计划的工作量。识别、预测、评估、管理和监测都要花费时间。但这是值得的。引用中国2500多年前的军事家孙子的一句话：“知己知彼，百战不殆”。对于软件项目管理者而言，这个“彼”指的就是风险。

第7 章：项目进度安排及跟踪

基本概念



基本概念

虽然软件延期交付的原因很多，但是大多数都可以追溯到下面几个根本原因上：

- 不切实际的项目期限
- 客户需求发生变更，没有在项目变更进度表上预先安排
- 对完成该工作所需的工作量、资源数量估计不足
- 项目开始时没有考虑可预测的、不可预测的风险
- 事先无法预计的技术困难
- 事先无法预计的人力困难
- 由于项目团队成员之间的交流不畅而导致的延期
- 管理者未能发现进度拖后，也未能采取措施解决这一问题



基本概念

在软件行业中，人们对过于乐观的(即"不切实际的")项目结束期限已经司空见惯。从设定项目结束期限的人的角度来看，有时候这样的项目结束期限是合理的，但是常识告诉我们，合理与否还必须由完成工作的人员来判断。

- 假定一个软件开发小组的任务是构造一个医疗诊断仪器的实时控制器，该控制器需要在9个月之内推向市场。在进行了仔细的估算和风险分析之后，软件项目管理人员得到的结论是在现有人员条件下，需要14个月的时间才能完成这一软件。这位项目管理者下一步该怎么办?
- 闯进客户的办公室(这里的客户非常可能是市场或销售人员)并要求修改交付日期似乎不太现实。外部市场压力决定了交付日期，届时必须发布产品。而(从事业前途的角度出发)拒绝这一项目同样是鲁莽的。
- 那么应该怎么办呢?

基本概念

在这种情况下，推荐以下的处理步骤：

- 按照以往项目的历史数据进行详细估算。确定项目的估算工作量和工期。
- 采用增量过程模型，制定一个软件开发策略，以保证能够在规定的交付日期提供主要功能，而将其他功能的实现推到以后。然后将这一计划做成文档。
- 与客户会谈并(用详细估算结果)来解释为什么规定的交付日期是不现实的，一定要指出所有这些估算都是基于以往的项目实践，而且一定要指出为了在目前规定的交付期限完成项目，与以往相比在工作效率上必须提高的百分比
- 将增量开发策略作为可选计划提交给客户



项目进度安排

项目进度安排

- 软件项目进度安排 是一种活动，它通过将工作量分配给特定的软件工程任务，从而将所估算的工作量分配到计划的项目工期内。
- 但要注意的是，进度是随着时间的改变而不断演化的。在项目计划的早期，首先建立一个宏观进度表，该进度表标识出所有主要的软件工程活动和这些活动影响到的产品功能。
- 随着项目的进展，宏观进度表中的每个条目都被精化成一个详细进度表。于是(完成一个活动所必须实现的)特定软件任务被标识出来，同时进行了进度安排。
- 可以下从两个不同的角度来讨论软件工程项目的进度安排：
  - 第一个视角，计算机系统的最终发布日期已经确定(而且不能更改)，软件开发组织必须将工作量分布在预先确定的时间框架内。
  - 第二个视角，假定已知大致的时间界限，但是最终发布日期是由软件开发组自行确定，工作量以一种能够最好地利用资源的方式进行分配，且在对软件进行仔细分析之后才决定最终发布日期。但不幸的是，第一种情况发生的频率远远高于第二种情况。

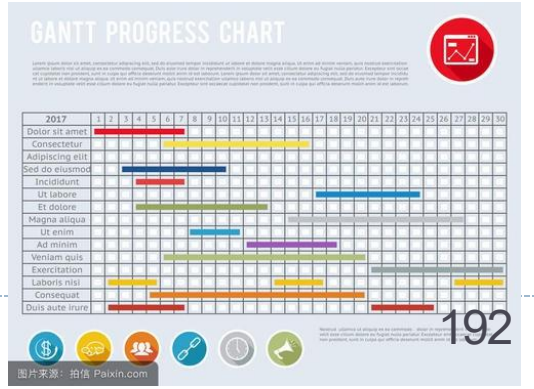
项目进度安排

基本原则

- 项目划分：将项目划分成多个可以管理的活动、动作和任务。为了实现项目的划分，产品和过程都需要进行分解。
- 依赖性：划分后的各个活动、动作或任务之间的相互依赖关系必须是明确的。有些任务必须按顺序出现，而有些任务则可以并发进行。有些活动或动作只有在其他活动产生的工作产品完成后才能够开始，而有些则可以独立进行。
- 时间分配：每个安排了进度计划的任务必须分配一定数量的工作单位（例如，若干人·日的工作量）。此外，还必须为每个任务指定开始日期和完成日期，任务的开始日期和完成日期取决于任务之间的相互依赖性及工作方式是全职还是兼职。

项目进度安排表

执行人	任务	4.11	4.12	4.13	4.14	4.15	4.16	4.17	4.18	4.19
A	项目申报	★								
B	购买物资		★	★	★					
B	借用物资		★	★	★					
B	调动物资			★	★					
V	场地审批		★	★						
V	联系有关方				★					
X	活动预告					★	★			
Z	外场活动							★		
B	整理物资							★		
B	经费报销							★	★	
A	后期反馈							★	★	★



项目进度安排

项目进度安排：基本原则

- 工作量确认：每个项目都有预定数量的人员参与。在进行时间分配时，项目管理人员必须确保在任意时段中分配的人员数量不会超过项目团队中的总人员数量。例如，某项目分配了3名软件工作师（例如，每天可分配的工作量为3人·日）。在某一天中，需要完成7项并发的任务，每个任务需要0.50人·日的工作量，在这种情况下，所分配的工作量就大于可供分配的工作量。
- 确定责任：安排了进度计划的每个任务都应该指定特定的团队成员来负责。
- 明确结果：安排了进度计划的每个任务都应该有一个明确的输出结果。对于软件项目而言，输出结果通常是一个工作产品（例如一个模块的设计）或某个工作产品的一部分。通常可将多个工作产品组合成“可交付产品”。
- 确定里程碑：每个任务或任务组都应该与一个项目里程相关联。当一个或多个工作产品经过质量评审并且得到认可时，标志着一个里程碑的完成。

项目进度安排

项目进度安排：人员与工作量之间的关系

- 对于小型软件开发项目，只需一个人就可以完成需求分析、设计、编码和测试。随着项目规模的增长，必须会有更多的人员参与
- 在项目后期增加人手通常会对项目产生破坏性的影响，其结果是使进度进一步拖延。后期增加的人员必须学习这一系统，而培训他们的人员正是一直在工作着的那些人，当他们进行教学时，就不能完成任何工作，从而使项目进一步拖延；除去学习系统所需的时间之外，新加入人员将会增加人员之间交流的路径数量和整个项目中交流的复杂度，从而需要更多的时间。
- 多年以来的经验数据和理论分析都表明项目进度是具有弹性的。即在一定程度上可以缩短项目交付日期（通过增加额外资源），也可以拖延项目交付日期（减少资源数量）。



项目进度安排

项目进度安排：人员与工作量之间的关系

- PNR（Putnam-Norden-Rayleigh）曲线表明了一个软件项目中所投入的工作量与交付时间的关系。项目工作量和交付时间的函数关系曲线如图5 - 1所示。图中的t_o表示项目最低交付成本所需的最少时间（即花费工作量最少的项目交付时间），而t_o左边（即当我们想提前交付时）的曲线是非线性上升的。

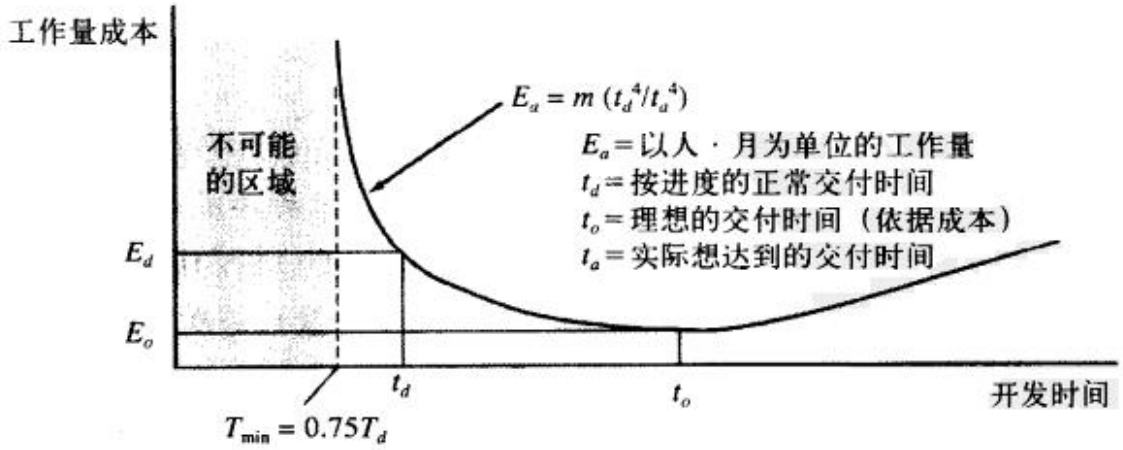




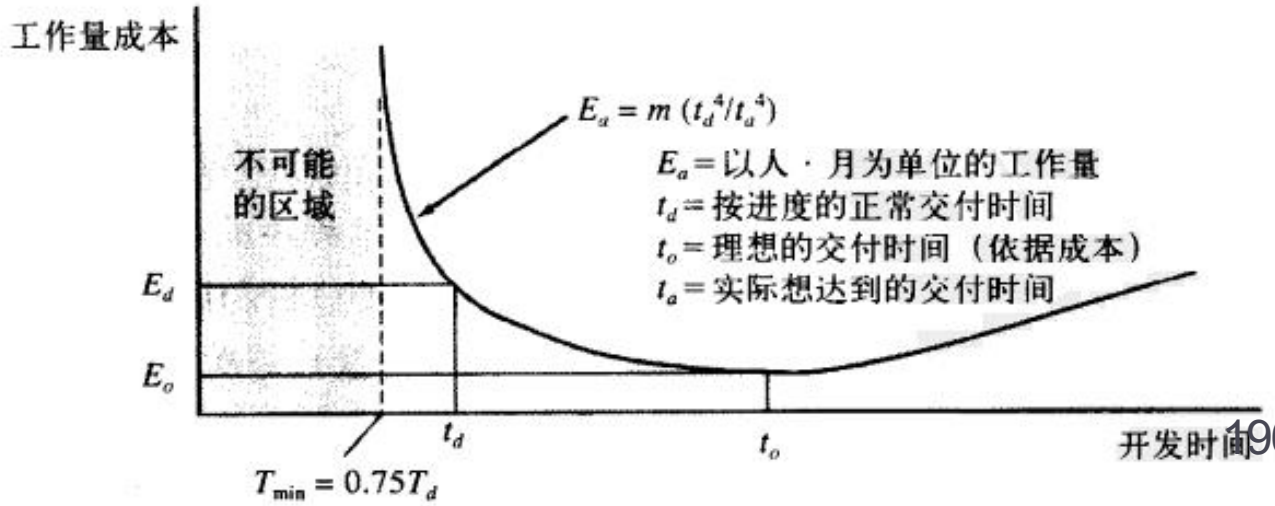
图5 - 1 工作量和交付时间的关系

PNR曲线表明完成一个项目的时间与投入该项目的人员工作量之间是高度非线性的

项目进度安排

项目进度安排：人员与工作量之间的关系

- 假设一个软件项目团队根据进度安排和现有的人员配置，估算所需要的工作量应为E_d，正常的交付时间应为t_d。虽然可以提前交付，但曲线在t_d左侧急剧上升。事实上，PNR曲线不仅说明了项目的交付时间不能少于$0.75t_d$，如果想更少，项目会进入“不可能的区域”，并面临着很高的失败风险；还说明了最低成本的交付时间t_o应该满足$t_o=2t_d$，即拖延项目交付可以明显降低成本，当然，这里的成本必须将与延期相关的营销成本排除在外。



项目进度安排

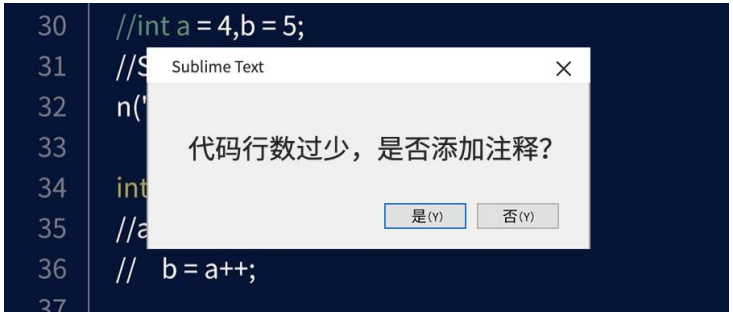
项目进度安排：人员与工作量之间的关系

- 交付的代码行数L与工作量和开发时间的关系可以用公式表示：

$$L = P \times E^{1/3} t^{4/3}$$

- 其中，E是以人月为单位的开发工作量；P是生产率参数，它反映了影响高质量软件工程工作的各种因素的综合效果；t是以月为单位的项目工期。





腾讯公布的高校代码行数排名，华科华工中大前三

项目进度安排

项目进度安排：人员与工作量之间的关系

- 重新调整前一页的软件方程式，可以得到开发工作量E的计算公式：

$$E = L^3/(P^3t^4) \qquad (5.1)$$

- 其中，E是在软件开发和维护的整个生命周期内所需的工作量（按人年计算）；t是以年为单位的开发时间；引入平均劳动力价格因素（￥/人年）之后，开发工作量的计算公式还能够与开发成本相关联。

工作量估算单位	1人年=12人月,1人月=22人天,1人天=8人小时		
开发工作量估算公式	项目开发工作量=新开发的软件规模/人均生产率		
开发阶段	人均生产率	软件规模	工作量
需求开发			
系统设计			
实现			
测试			
……			
开发工作量汇总			
管理工作量估算公式	项目管理工作量=项目开发工作量×比例系数		
项目管理的主要事务	项目规划、项目监控、[illegible]理、质量管理……		
比例系数	例如20%	项目管理工作量	

项目进度安排：人员与工作量之间的关系

- 调整后的方程式引出一些有趣的结果。
  - 假设有一个复杂的实时软件项目，估计需要33000LOC、12人年的工作量。如果项目团有8个人，那么项目大约需要1.3年的时间完成。但是如果将交付日期延长到1.75年，则由公式5.1所描述的模型所具有的高度非线性特性将得出以下结论：

$$E = L^3/(P^3t^4) \sim 3.8\text{人年}$$

- 这意味着通过将交付日期推迟到6个月，我们可以将项目团队的人数从8人减少到4人。这一结果有效性待考证，但是其含义却十分清楚：通过在略为延长的时间内使用较少的人员，可以实现同样目标。

项目进度安排

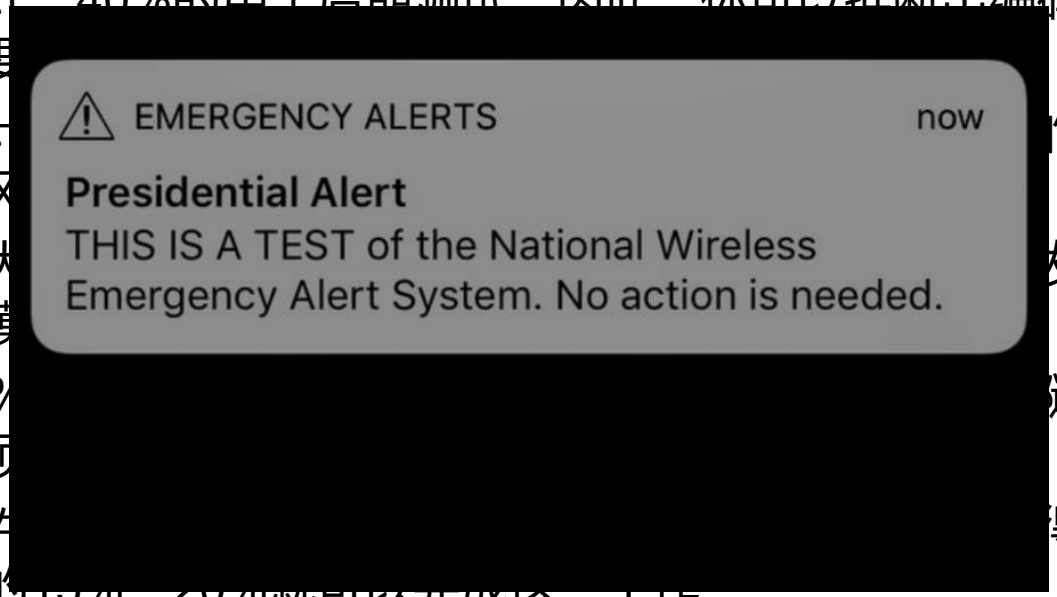
项目进度安排：工作量分配

- 软件过程中的工作量分配通常采用40 - 20 - 40法则。总体工作量的40%分配给前期的分析和设计，40%的用于后期测试。因此，你可以推断出编码工作（20%的工作量）是次要的。这种工作量分配法只能作为指导原则。
  - 用于项目计划的工作量很少超过2% - 3%，除非提交给组织的项目费用极高且具有高风险。
  - 需求分析大约占用10% - 25%的工作量，用于分析或原型开发的工作量应该与项目规模和复杂度成正比增长。
  - 通常有20% - 25%的工作量用于软件设计，用于设计评审和随之而来的迭代开发也必须考虑。
  - 因为在软件设计时投入相当的工作量，随后的编码工作就变得相对简单。总体工作量的15% - 20%就可以完成这一工作。
  - 测试和随之而来的调试工作将占用30% - 40%的软件开发工作量。软件的重要性决定了所需测试工作的分量，如果软件系统是人命相关的，就应该考虑分配更高的测试工作量比例。

项目进度安排

项目进度安排：工作量分配

- 软件过程中的工作量分配通常采用40 - 20 - 40法则。总体工作量的40%分配给前期的分析和设计，40%的用于后期测试，因此，你可以推断出编码工作（20%的工作量）是次要
  - 用于项目计 的项目费用极高且具有高风
  - 需求分析大 发的工作量应该与项目规模
  - 通常有20% 随之而来的迭代开发也必须
  - 因为在软件 得相对简单。总体工作量的15% - 20%就可以完成这一工作。
  - 测试和随之而来的调试工作将占用30% - 40%的软件开发工作量。软件的重要性决定了所需测试工作的分量，如果软件系统是人命相关的，就应该考虑分配更高的测试工作量比例。

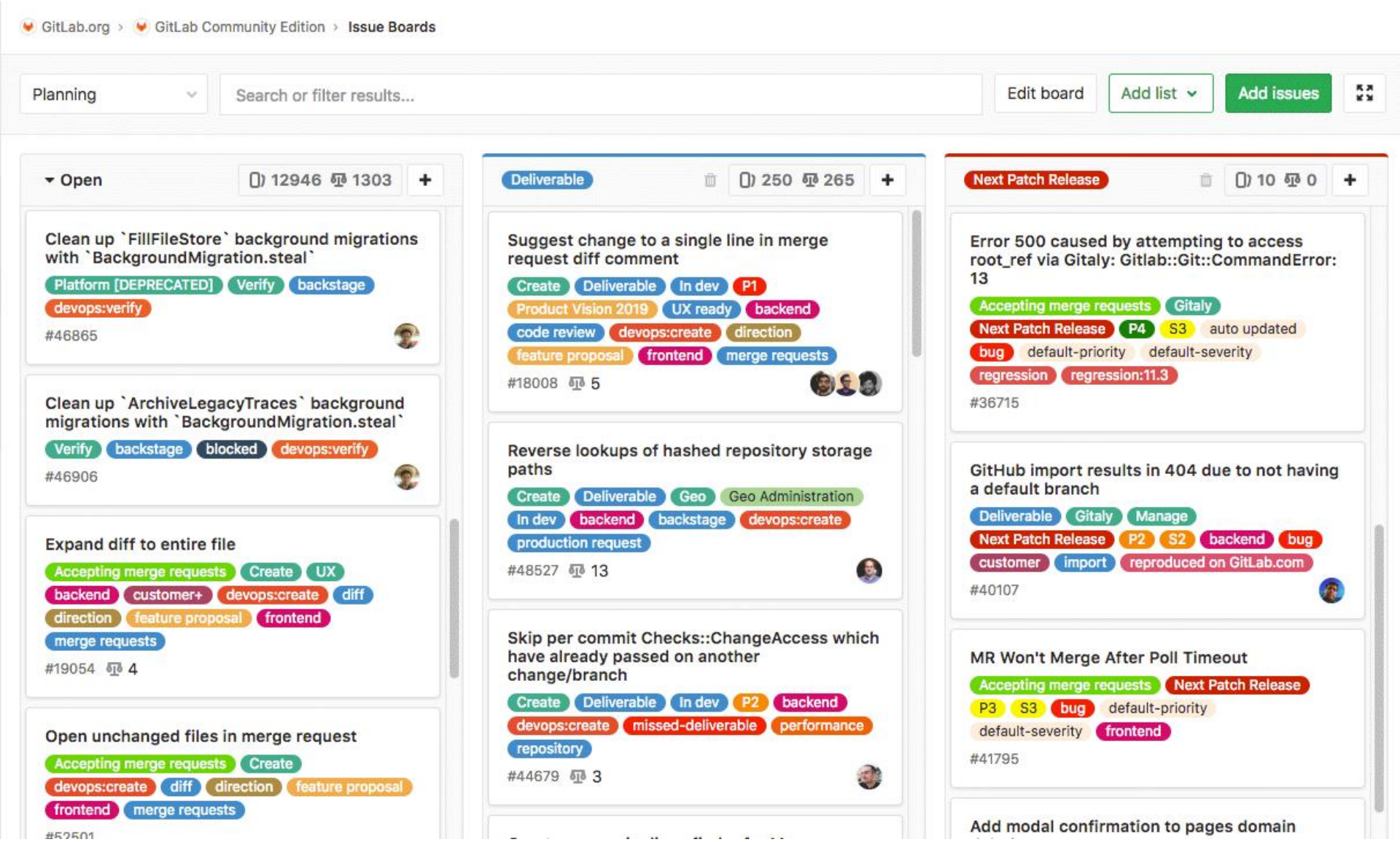


为软件项目定义任务集

定义任务集

为软件项目定义任务集

- 我们已经学习了多种不同的过程模型。无论软件团队选择的是线性顺序模型、增量模型、演化模型，或它们的某种变型，过程模型都是由一个任务集组成的，这些任务集使得软件团队能够定义、开发和最终维护计算机软件。
- 没有能普遍适用于所有软件项目的任务集合。适用于大型复杂系统的任务集可能对于小型且相对简单的项目而言就过于复杂。因此一个有效的软件过程应该定义一组任务集来满足不同类型项目的要求。
- 一个任务集包括软件工程工作任务、里程碑，以及为完成某个特定项目就必须完成的工作产品。为获得高质量的软件产品，所选择的任务集必须提供充分的规程要求，但同时又不能让项目团队负担不必要的工作。


GitLab.org › GitLab Community Edition › Issue Boards
Planning
Search or filter results...
Edit board
Add list
Add issues
Open
12946 1303
Clean up `FillFileStore` background migrations with `BackgroundMigration.steal`
Platform [DEPRECATED] Verify backstage devops:verify
#46865
Clean up `ArchiveLegacyTraces` background migrations with `BackgroundMigration.steal`
Verify backstage blocked devops:verify
#46906
Expand diff to entire file
Accepting merge requests Create UX backend customer+ devops:create diff direction feature proposal frontend merge requests
#19054 4
Open unchanged files in merge request
Accepting merge requests Create devops:create diff direction feature proposal frontend merge requests
Deliverable
250 265
Suggest change to a single line in merge request diff comment
Create Deliverable In dev P1 Product Vision 2019 UX ready backend code review devops:create direction feature proposal frontend merge requests
#18008 5
Reverse lookups of hashed repository storage paths
Create Deliverable Geo Geo Administration In dev backend backstage devops:create production request
#48527 13
Skip per commit Checks::ChangeAccess which have already passed on another change/branch
Create Deliverable In dev P2 backend devops:create missed-deliverable performance repository
#44679 3
Next Patch Release
10 0
Error 500 caused by attempting to access root_ref via Gitaly: Gitlab::Git::CommandError: 13
Accepting merge requests Gitaly Next Patch Release P4 S3 auto updated bug default-priority default-severity regression regression:11.3
#36715
GitHub import results in 404 due to not having a default branch
Deliverable Gitaly Manage Next Patch Release P2 S2 backend bug customer import reproduced on GitLab.com
#40107
MR Won't Merge After Poll Timeout
Accepting merge requests Next Patch Release P3 S3 bug default-priority default-severity frontend
#41795
Add modal confirmation to pages domain

定义任务集

为软件项目定义任务集

- 在进行项目进度安排时，必须将任务集分布在项目时序图上。任务集应该根据软件团队所决定的项目类型和严格程序而有所不同。一般项目的类型如下：
  - 概念开发项目：目的是为了探索某些新的业务概念或者某种新技术的应用。
  - 新应用开发项目：根据特定的客户需求而承担的项目。
  - 应用增强项目：对现有软件进行最终用户可见的功能、性能或界面的修改。
  - 应用维护项目：以用户不会立即察觉的方式对现有软件进行纠错、适应或扩展。
  - 再工程项目：为了全部或部分重建一个现有(遗留)系统而承担的项目（重构）。

进度安排

进度安排

- 软件项目的进度安排与其他多任务工程工作的进度安排几乎没有差别。因此，通用的项目进度安排工具和技术不必做太多修改就可以应用于软件项目。
- 程序评估和复审技术(program evaluation and review technique，PERT)和关键路径方法(critical path method，CPM)就是两种可以用于软件开发的项目进度安排方法。这两种技术都是由早期项目计划活动中已经产生的信息来驱动的，这些信息包括：
  - 工作量的估算。
  - 产品功能的分解。
  - 适当过程模型和任务集的选择。
  - 任务的分解。
- 任务之间的依赖关系可以使用任务网络来定义。任务有时也称作项目的工作分解结构(work breaddown stucture，WBS)，其定义可以是针对整个产品，也可以是针对单个功能进行的。

定义任务网络

- 任务网络也称为活动网络，是一个项目的任务流程的图形表示。有时将任务网络作为在自动项目进度安排工具中输入任务序列和依赖关系的机制。最简单的任务网络形式(当创建宏观进度表时使用)只描述主要的软件工程任务，图5 - 2显示了一个概念开发项目的任务网络示意图。

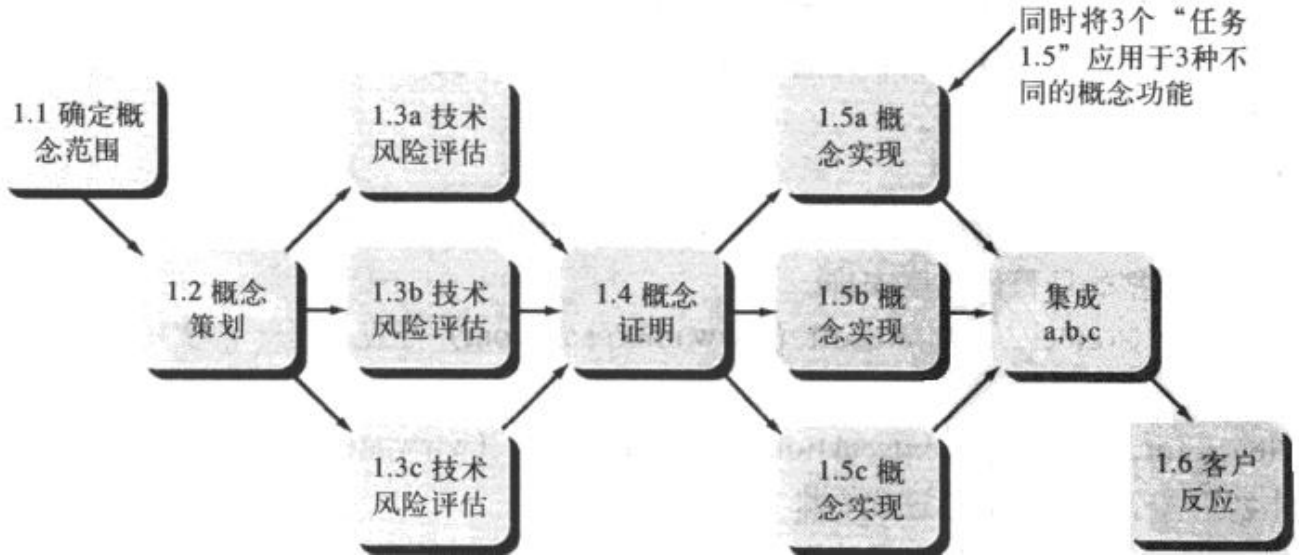
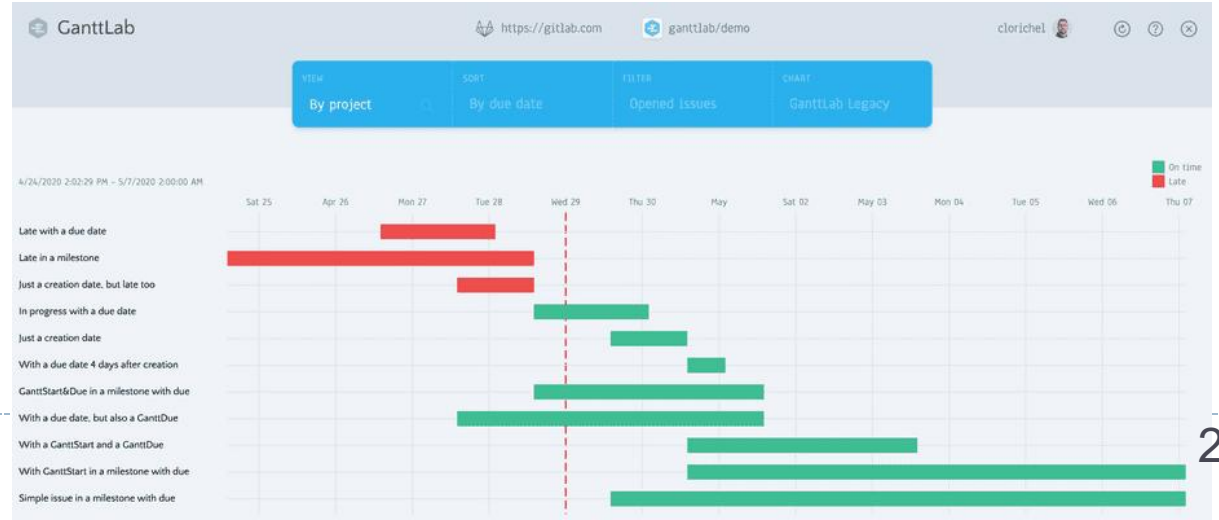


图5 - 2 概念开发项目的任务网络

进度安排

进度安排：时序图

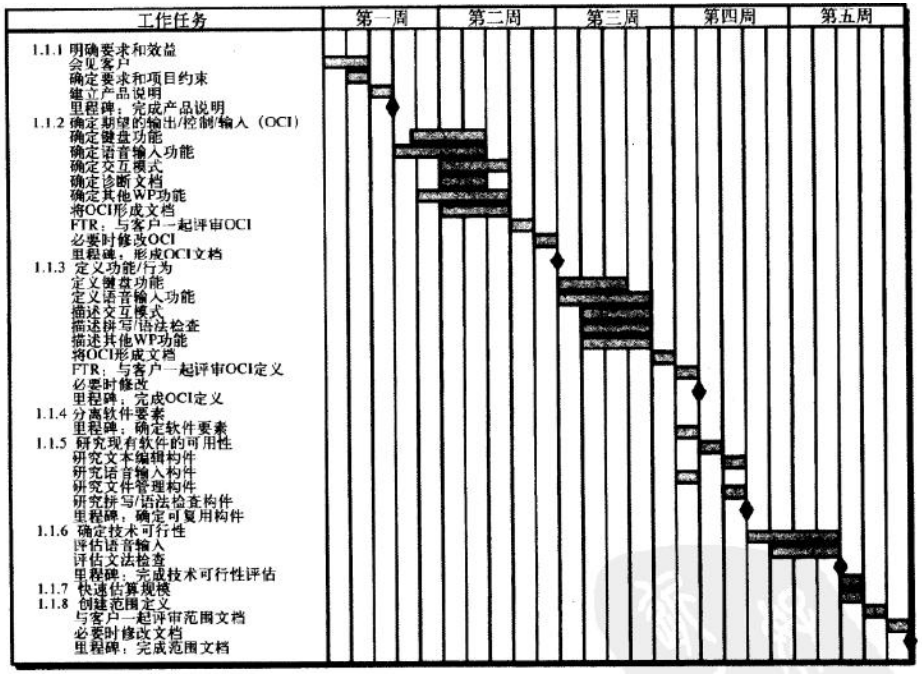
- 在创建软件项目进度表时，计划人员将从一组任务入手。如果使用自动工具，就可以用任务网络或任务大纲的形式输入工作分解结构，然后为每一项任务输入工作量、工期和开始日期。此外，还可以将某些任务分配给特定的人员。
- 输入信息之后，就可以产生时序图(Timeline Chart)，也叫做甘特图(Gantt Chart)。可以为整个项目建立一个时序图，也可以为各个项目功能或各个项目参与者分别建立各自的时序图。

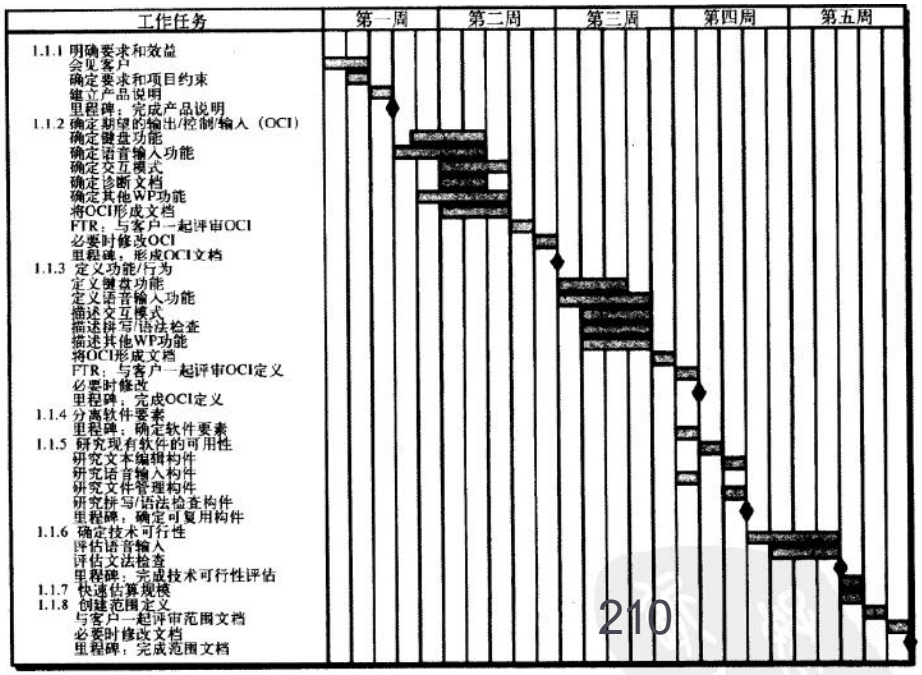


进度安排

进度安排：时序图

- 表 5 - 3 给出了时序图的格式，该图描述了一个字处理软件产品中“确定概念范围”任务的软件项目进度安排。所有的项目任务(对于“确定概念范围”)都在左边的栏中列出。水平条表示每个任务的工期，同一时段中存在多个水平条时，就代表任务之间存在并发。图中的菱形表示里程碑。





进度安排

进度安排：时序图

- 输入了生成时序图所需的信息之后，软件项目进度安排工具都会生成项目表，一种列出所有项目任务的表格。项目表中列出了各个任务的开始与结束日期及各种相关信息(图5 - 4)。通过项目表与时序图，项目管理者就可以跟踪项目进展情况

工作任务	计划开始	实际开始	计划结束	实际结束	人员分配	工作量分配	备注
1.1.1 明确要求和效益							
会见客户	wk1, d1	wk1, d1	wk1, d2	wk1, d2	BLS	2 pd	范围定义需要更多的工作量/时间
确定要求和项目约束	wk1, d2	wk1, d2	wk1, d2	wk1, d2	JPP	1 pd	
建立产品说明	wk1, d3	wk1, d3	wk1, d3	wk1, d3	BLS/JPP	1 pd	
里程碑：完成产品说明	wk1, d3	wk1, d3	wk1, d3	wk1, d3			
1.1.2 确定期望的输出/控制/输入（OCI）	wk1, d4	wk1, d4	wk2, d2		BLS	1.5 pd	
确定键盘功能	wk1, d3	wk1, d3	wk2, d2		JPP	2 pd	
确定语音输入功能	wk2, d1		wk2, d3		MLL	1 pd	
确定交互模式	wk2, d1		wk2, d2		BLS	1.5 pd	
确定诊断文档	wk1, d4	wk1, d4	wk2, d3		JPP	2 pd	
确定其他WP功能	wk2, d1		wk2, d3		MLL	3 pd	
将OCI形成文档	wk2, d3		wk2, d3		all	3 pd	
FTR：与客户一起评审OCI	wk2, d4		wk2, d4		all	3 pd	
必要时修改OCI	wk2, d5		wk2, d5				
里程碑：形成OCI文档							
1.1.3 定义功能/行为							

图5 - 4　一个项目表（也称资源表）的例子

进度安排：跟踪进度

- 在面对交付期限的巨大压力时，有经验的项目管理者有时会使用一种称为时间盒的项目进度安排和控制技术。时间盒方法认为完整的产品可能难以在预定时间内交付，应该选择增量软件开发范型，并为每个增量的交付制定各自的进度表。
- 接着，对与每个增量相关的任务实行时间盒技术。也就是按该增量的交付日期向后进行推算来调整每项任务的进度。将各个任务放入相应的"盒子"中，当一个任务触及其时间盒边界时(正负10%的范围内)，则该项任务停止，下一任务开始。
- 对时间盒方法的最初反应通常是消极的：如果工作尚未完成，我们该如何继续？问题的答案在于完成工作的方式。当遇到时间盒的边界时，很可能已经完成了任务的90%，余下的10%工作尽管重要，但是可以(1)被推迟到下一个增量中；或(2)在以后需要时再完成。项目朝着交付日期推进，而不是卡在某项任务上。

挣值分析

挣值分析

在“进度安排”小节中讨论了一系列项目跟踪的定性方法，为项目管理者提供了项目进展情况的指标。但是，对所提供信息的评估在某种程序上是主观的。那么当软件团队按项目进度表实施工作任务时，是否存在某种定量的技术来评估项目进展情况呢?





挣值分析

- 事实上，存在一种用于项目进展的定量分析技术，称为挣值分析（earned value analysis，EVA），也叫获得值分析。
- 不管要完成何种类型的工作，挣值系统为每个（软件项目）任务提供了通用的值尺度，可以估算完成整个项目所需要的总小时数，并且可以根据各个任务所估算的小时数占总小时数的百分比来确定该任务的挣值。 - - Humphrey [HUM 95]
- 简单地说，挣值是对项目进展的测量。它使得计划者能够不依赖于感觉，采用定量的分析方法来评估一个项目的“完成百分比”。

挣值分析

确定挣值的步骤

1. 为进度表中的每个工作任务确定其预计工作的预算成本（budgeted cost of work scheduled，BCWS）。在估算过程中，要计划每个软件工程任务的工作量（以人时或人日为单位），因此，$BCWS_i$是指工作任务i的计划工作量。
2. 将所有工作任务的BCWS值加起来，可计算出完成工作的预算（budget at completion，BAC），因此，对所有任务，$BAC = \Sigma(BCWS_i)$
3. 接着，可计算已完成工作的预算成本（budgeted cost of work performed，BCWP）。BCWP的值是在项目进度表中该时间点已经实际完成的所有工作任务的BCWS值之和。

挣值分析

- BCWS和BCWP的不同点是，前者表示计划将完成的工作的预算，而后表示已经实际完成的工作的预算。给定BCWS、BAC和BCWP的值，就可以得出相关的项目进展指标：
  - 进度表执行指标　SPI = BCWP / BCWS
  - 进度表偏差　SV = BCWS - BCWS
- 其中，SPI是效率指标，指出项目使用预定资源的效率，SPI值越接近1.0说明项目的执行效率越高。SV只表示与计划进度的偏差。
  - 预定完成百分比 = BCWS / BAC（表示在时间点t应该完成工作的百分比值）
  - 完成百分比 = BCWP / BAC（表示在特定时间点t实际完成工作的百分比值）

挣值分析

- 也可以计算出已完成工作的实际成本（actual cost of work performed，ACWP）。ACWP的值是在项目进度表中某时间点已经完成的工作任务的实际工作量之和。然后，再计算：
  - 成本执行指标 CPI = BCWP / ACWP
  - 成本偏差 CV = BCWP － ACWP
- CPI值越接近1.0说明项目与预算越接近。CV表示在项目特定阶段的成本节省（相对于计划成本）或短缺。

挣值分析在可能出现问题之前就指出了进度安排的难点，这使得软件项目管理者能够在项目危机出现前采取有效措施。

小结

- 计划活动是软件项目管理的重要组成部分，而进度安排是计划活动的首要活动。进度安排与估算方法及风险相结合，可以为项目管理者画出一张路线图。
- 进度安排始于过程分解。根据项目特性，为将要完成的工作选择适当的任务集。任务网络描述了各项工作任务、每一项任务与其他任务之间的依赖关系以及计划工期。任务网络可以用来确定项目的关键路径、时序图以及各种项目信息。以进度表为指导，项目管理者可以跟踪和控制软件工程过程中的每一个步骤。



思考题

1. 当管理者要求的项目结束期限我们无法实现时，我们应该怎么办?
2. 在软件过程的工作流程中应如何分配工作量?
3. 使用Putnam的软件方程式编制一个表，以反映软件项目中人员数量与项目工期之间的关系。此项目需要50000LOC和15人年的工作量（生产率参数为5000）。假定该软件必须在24±12个月的时间期限内交付。

思考题

1. 假设你是一个软件项目管理者，受命为一个小型软件项目进行挣值统计。这个项目共计划了56个工作任务，估计需要582人日才能完成，但是，按照项目进度，现在应该完成15个任务，下面给出相关进度安排数据（单位：人日），请你做出挣值分析，计算该项目的进度表执行指标SPI、进度偏差SV、预定完成百分比、完成百分比、成本执行指标CPI和成本偏差CV。

任务	计划工作量	实际工作量
1	12.0	12.5
2	15.0	11.0
3	13.0	17.0
4	8.0	9.5
5	9.5	9.0
6	18.0	19.0
7	10.0	10.0
8	4.0	4.5
9	12.0	10.0
10	6.0	6.5
11	5.0	4.0
12	14.0	14.5
13	16.0	-
14	6.0	-
15	8.0	-

第8章：软件质量保证

质量概念

质量概念

什么是质量

- American Heritage Dictionary（《美国传统字典》）中对质量的定义是：“某一事物的特征或属性”。
  - 作为一个事物的属性，质量指的是可以度量的特征——那些可以与已知标准进行比较的东西，如长度、颜色、电的性质、可延展性等等。
  - 但是软件，很大程度上是一种知识实体，其特征的定义远比物理对象要困难得多。
- 然而，确实存在程序特征的测量。这些特征属性包括循环复杂度、内聚力、功能点、代码行数和其他许多在产品度量一讲中讨论的属性。
- 当根据对象的可测量特征考察一个对象时，可以有两种不同的质量：设计质量和 一致性质量。

质量概念

什么是质量

- 设计质量：设计者为一个产品规定的特征。软件设计质量包括系统的需求、规约和设计。
- 一致性质量：在制造产品的过程中遵守设计规格的程度。如果软件实现过程符合设计要求，并且开发的系统满足需求和性能目标，则一致性质量较高。





质量概念

什么是质量

- 设计质量和一致性质量比较“直观的”关系符合下面的公式：

  用户满意度 = 合格的产品 + 好的质量 + 按预算和进度安排交付

- 实际上，质量是很重要的。但是，如果用户不满意，其他任何事情也就都不重要了。而另一种质量观点认为：如果一个软件产品能给最终用户带来实质性的益处，他们可能会心甘情愿地忍受偶尔的可靠性或性能问题。

质量概念

质量控制

- 质量控制是为了保证每一件工作产品都能满足对它的需求而应用于整个开发周期中的一系列审查、复审和测试。
- 质量控制在创建工作产品的过程中包括一个反馈循环。测量和反馈相结合，使得我们能够在得到的工作产品不能满足其规格说明时调整开发过程。
- 质量控制中的关键概念之一就是所有工作产品都具有明确的和可测量的规格说明，我们可以将每个过程的产品 与这一规格说明进行比较。反馈循环对于减少产生的缺陷是至关重要的。

质量概念

质量保证

- 质量保证由评估质量控制活动有效性和完整性的一系列审核和报告功能构成。
- 质量保证的目的是：为管理层提供了解产品质量所必需的数据，从而获得产品质量是否符合预定目标的认识和信心。
- 当然如果质量保证所提供的数据发现了问题，则管理层负责解决这些问题，并为解决这些质量问题分配所需的资源。



质量概念

质量成本

- 质量成本包括所有由质量工作或者进行与质量有关的活动所引发的成本。
- 进行质量成本研究不仅能够为当前质量成本设定基线，还能确定降低质量成本的时机会，以及提供规范化的比较标准。
- 质量成本可以细分为与预防成本、鉴定成本 及失效成本。
  - 预防成本包括质量计划、正式技术复审、测试设备及培训。
  - 鉴定成本包括为深入了解“首次通过”各个过程时的产品状态而开展的那些活动。如：过程内和过程间的审查，设备校准和维护，测试。
  - 失效成本是指如果将产品交付给客户之前没有缺陷的话就不会存在的成本。

软件质量保证（SQA）

软件质量保证

- 文献中对软件质量的定义有很多种。在这里对软件质量做如下定义：

软件要符合明确的功能和性能需求，符合已清晰文档化的开发标准，并且具有专业人员开发的软件所应有的隐含特征。

- 定义强调了以下三个重要方面：
  - 软件需求是进行质量测量的基础。与需求不符就是质量不高。
  - 规定的标准定义了一组指导软件开发的准则。如果不能遵照这些准则，就极有可能导致质量不高。
  - 通常有一组“隐含需求”是不会被明确提出的（如对易用性和易维护性的需求）。如果软件明确提出的需求，却没有满足隐含需求，软件质量仍然值得怀疑。

软件质量保证

SQA 活动

- 软件质量保证由很多任务组成，这些任务分别与两类不同的参与者相关：一类是做技术工作的软件工程师，另一类是负责质量保证计划、监督、记录保存、分析和报告工作的SQA小组。
- 软件工程师通过运用可靠的技术方法和措施、进行正式的技术评审和执行计划周密的软件测试来解决质量问题（并完成质量保证和质量控制活动)。
- SQA小组的职责是辅助软件团队实现高质量的软件产品。SEI建议了一组有关质量保证计划、监督、记录保存、分析和报告工作的SQA活动。

软件质量保证

SQA 活动

- SQA活动都是由一个独立的SQA小组完成的，包括以下的工作：
- 编制项目质量保证计划
- 参与项目的软件过程描述的编写
- 评审软件工程活动，以验证是否符合规定的软件过程
- 审核指定的软件工作产品以验证是否遵守作为软件过程一部分的那些规定
- 确保根据文档化的规程记录和处理软件工作及工作产品中的偏差
- 记录各种不符合的部分，并报告给高层管理人员



软件质量保证

目标、属性和度量

- 执行软件质量保证活动，以实现一套务实的目标：
- 需求质量。需求模型的正确性、完整性和一致性将对所有后续工作产品的质量有很大的影响。软件质量保证必需确保软件团队严格评审需求模型，以达到高水平的质量。
- 设计质量。软件团队应该评估设计模型的每个元素，以确保设计模型显示出高质量，并且设计本身符合需求。SQA寻找能反映设计质量的属性。
- 代码质量。源代码和相关的工作产品（如其他说明资料）必须符合本地的编码标准，并显示出易于维护的特点。SQA应该找出那些能合理分析质量的属性。
- 质量控制有效性。软件团队应使用有限的资源，在某种程度上最有可能得到高品质的结果。SQA分析在评审和测试上的资源分配，评估是否以最有效的方式进行分配的。

软件质量保证

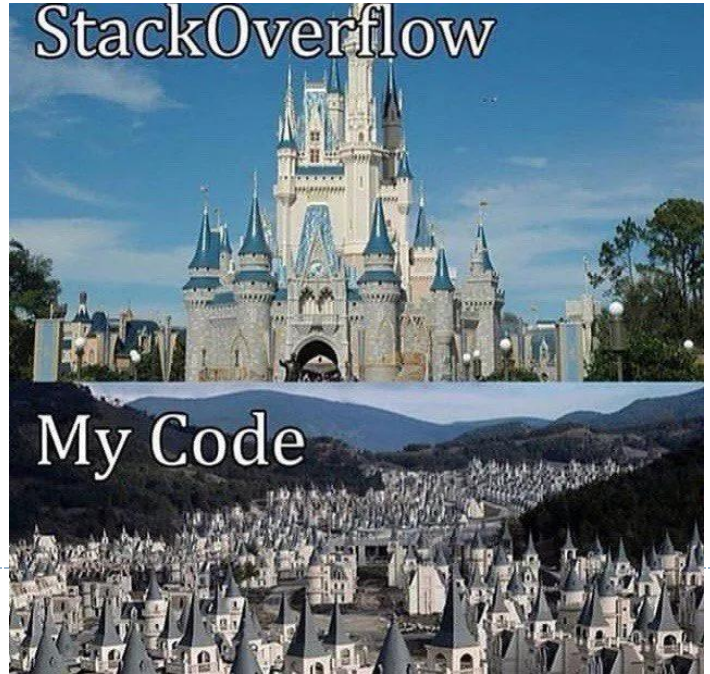
目标、属性和度量

- 对于所讨论的每个目标，图7 - 1标出现有的质量属性。可以使用度量数据来标明所示属性的相对强度。

目标	属性	度量
需求质量	歧义	引起歧义地方的修改数量（例如：许多、大量、与人友好）
	完备性	TBA, TBD的数量
	可理解性	节/小节的数量
	易变性	每项需求变更的数量
		变更所需要的时间（通过活动）
	可追溯性	不能追溯到设计/代码的需求数
	模型清晰性	UML模型数
		每个模型中描述文字的页数·
		UML错误数
设计质量	体系结构完整性	是否存在现成的体系结构模型
	构件完备性	追溯到结构模型的构件数
		过程设计的复杂性
	接口复杂性	挑选一个典型功能或内容的平均数
		布局合理性
	模式	使用的模式数量
代码质量	复杂性	环路复杂性
	可维护性	设计要素（第8章）
	可理解性	内部注释的百分比
		变量命名约定
	可重用性	可重用构件的百分比
	文档	可读性指数
质量控制效率	资源分配	每个活动花费的人员时间百分比
	完成率	实际完成时间与预算完成时间之比
	评审效率	参见评审度量（第14章）
	测试效率	发现的错误及关键性问题数
		改正一个错误所需的工作量
		错误的根源

图7 - 1 软件质量目标、属性和度量

软件评审



软件评审

- 软件评审是软件过程中的“过滤器”。也就是说，在软件工程 过程中的不同阶段进行软件评审，可以起到发现错误和缺陷，进而消除它们的作用。软件评审还能够“净化”需要模型、设计模型、源代码和测试数据等软件工程工作产品。
- 在软件工程过程中可以进行的评审有很多种，它们各有各的作用。
  - 在休息室里讨论技术问题的非正式会谈是一种评审方式；
  - 将软件架构正式介绍给客户、管理层和技术人员也是一种评审方式。
  - 正式技术评审（FTR），即同行评审，有时也称为走查（Walkthrough）或审查（inspection）。从质量保证的角度出发，正式的技术复审是最有效的过滤器。由软件工程师（或其他人）对软件工程师进行的正式技术复审是一种发现错误和提高软件质量的有效手段。



INFO

隐错、错误和缺陷

软件质量控制的目标是消除软件中存在的质量问题，从广义上讲，这也是一般质量管理的目标。有很多术语可用来描述这些质量问题，如“隐错”（bug）、“故障”（fault）、“错误”（error）或“缺陷”（defect）。它们的含义是相同的吗？还是存在微小的差别呢？

在本书中，我们明确指出了“错误”（指在软件交付给最终用户之前发现的质量问题）和“缺陷”（指在软件交付给最终用户之后发现的质量问题[⊖]）的差别。这样区别的原因是“错误”和“缺陷”对经济、商业、心理和人员的影响有很大区别。作为软件工程师，我们期望在客户和（或）最终用户遇到问题之前尽可能多地发现并改正“错误”。我们同样期望避免“缺陷”——因为“缺陷”（有理由）使软件工作者显得水平低下。

然而要指出的是，本书中描述的“错误”和“缺陷”的差别并不是主流观点。在软件工程领域中，大多数人都认为“缺陷”、“错误”、“故障”和“隐错”是没有差别的。也就是说，遇到问题的时间与采用哪个术语描述毫无关系。这个观点中争论的焦点是：有些时候很难明确地区分是交付之前还是交付之后（如敏捷开发中采用的增量过程）。

不管怎么说都应该认识到：发现问题的时间点是非常关键的。软件工程师应该努力再努力，力图在他们的客户和最终用户遇到问题之前将其发现。有关“隐错”术语的讨论，感兴趣的读者可从www.softwaredevelopment.ca/bugs.shtml找到。

软件评审

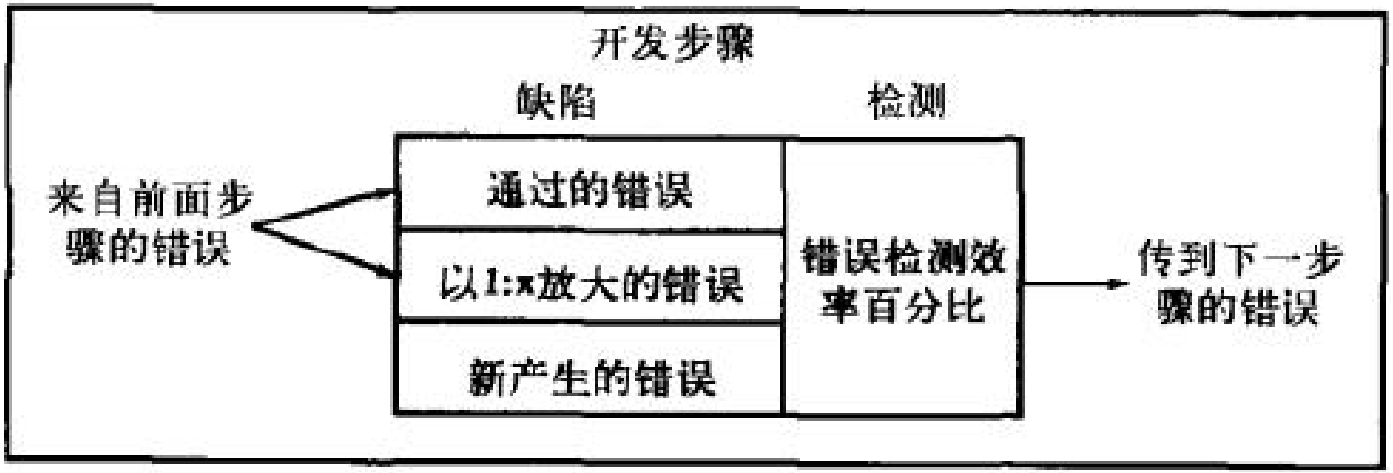
软件缺陷对成本的影响

- 正式技术评审的主要目标是在此过程中发现错误，以使它们不会在软件交付之后变成缺陷。正式技术评审的最明显的优点是可以早发现错误，以防止错误被传播到软件过程的后续阶段。
- 产业界的大量研究表明：设计活动引入的错误占软件过程中出现的所有错误(和最终的缺陷)数量的50%~65%。然而，已经证明，评审技术在发现设计缺陷方面高达75%的有效性。通过检测和排除大量设计错误，评审过程将极大降低后续活动的成本。

软件评审

缺陷放大和消除

图7-2 缺陷放大模型



- 可以用“缺陷放大模型”［IBM81］来说明在软件工程过程中的概要设计、详细设计和编码阶段中错误的产生和检测过程。如图7-2所示，其中最外层方框代表一个软件开发步骤。在该开发步骤中，错误可能由于疏忽而产生，在评审过程可能没有发现新产生的错误以及来自前面步骤的错误，从而导致一定数量的错误通过当前步骤。在有些情况下，从前面步骤传过来的错误在当前步骤中会被放大（放大倍数为X）。将开发步骤方框进一步细分可以说明这些特点及错误检测的有效性百分比，后者是评审完善性的函数。

软件评审

缺陷放大和消除

- 图7-3所示的是一个假设在软件过程中不进行任何评审的缺陷放大的例子。
- 如图所示，假设每一个测试步骤都能够发现和改正50%的输入错误，而又不引入新的错误（乐观的估计）。
- 10个概要设计阶段的错误在测试开始之前已经放大成为94个错误。12个隐藏的缺陷则随软件发布进入客户现场。

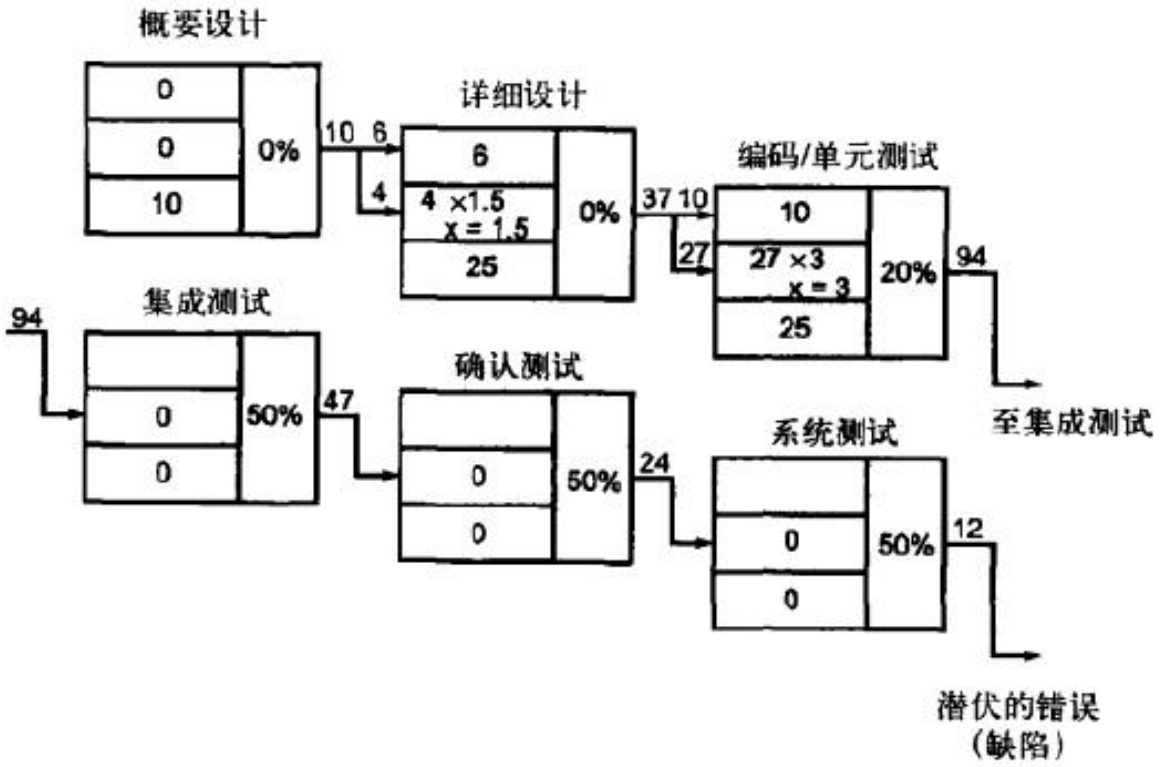


图7-3 缺陷放大 - 无评审

软件评审

缺陷放大和消除

- 图7-4只是在设计和编码开发步骤中引入了评审。这种情况下，最初的10个概要设计错误在测试开始之前放大成为24个。只有3个隐藏的缺陷。

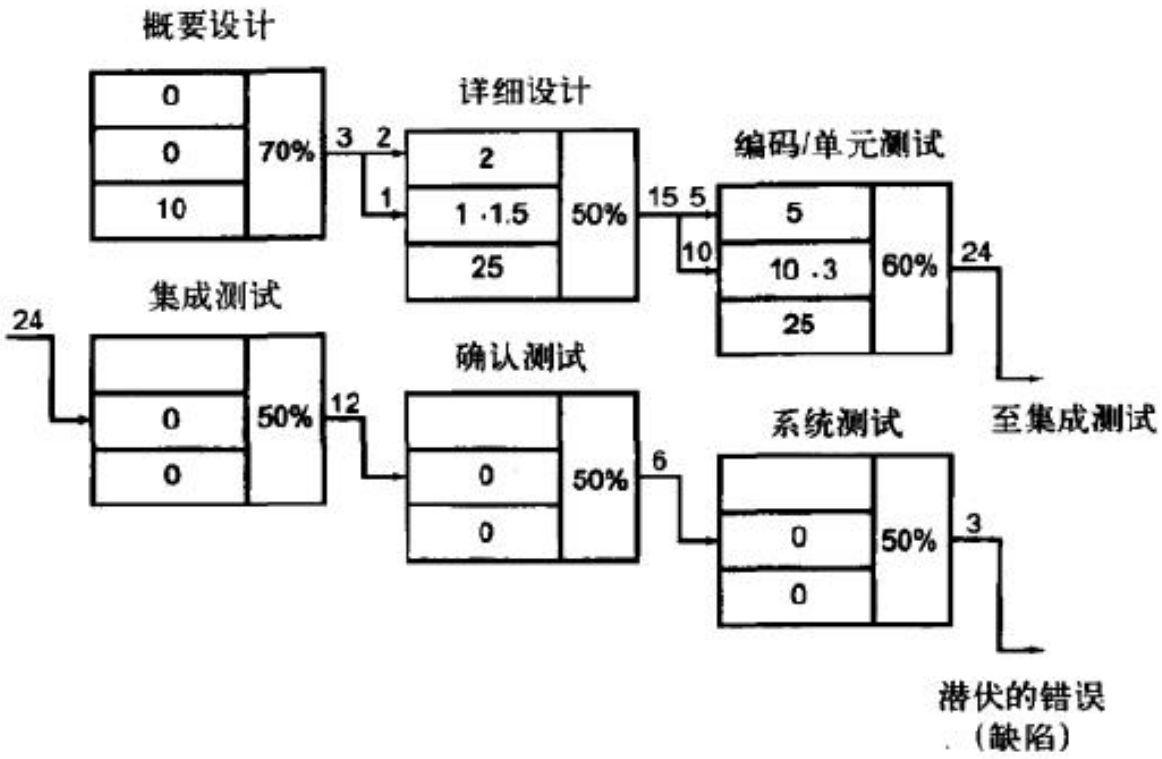


图7-4 缺陷放大 - 有评审

软件评审

缺陷放大和消除

- 回忆一下与发现和改正错误相关的相对成本，将图7-3和图7-4每一个步骤中发现的错误数量乘以消除一个错误所需要的成本（设计是1.5个成本单位，测试前是6.5个成本单位，测试中是15个成本单位，发布后是67个成本单位），由此可以确定总开发成本（在我们假设的例子中无论有没有复审的情况下）。通过这些数据，在进行了评审的情况下，开发和维护的总成本是783个成本单位。而在不进行评审的情况下，总成本是2177个成本单位 - 几乎是前者的3倍。

开发成本比较

错误发现时机	数量	成本单位	总计
进行复审			
设计期间	22	1.5	33
测试之前	36	6.5	234
测试期间	15	15	315
发布之后	3	67	201
			783
不进行复审			
测试之前	22	6.5	143
测试期间	82	15	1230
发布之后	12	67	804

软件评审

缺陷放大和消除

- 为了进行评审，软件工程师必须花费时间和工作量，开发组织也必须提供相应费用。然而，上述例子的结果证明了我们面临的是一种选择是：要么现在付出、否则以后会付出更多。
- 正式技术评审（在设计或其他技术活动中的）带来了显而易见的成本效益。因此，应该进行正式技术评审。



正式技术评审

正式技术评审

- 正式技术评审（FTR）是一种由软件工程师（以及其他人）进行的软件质量控制活动。
- FTR的目标是：
  (1)发现软件的任何一种表示形式中的功能、逻辑或实现上的错误；
  (2)验证评审中的软件是否满足其需求；
  (3)保证软件的表示符合预先定义的标准；
  (4)得到以统一的方式开发的软件；
  (5)使项目更易于管理。
- 由于FTR的进行使大量人员对软件系统中原本并不熟悉的部分更为了解，因此，FTR还起到了提高项目连续性和培训后备人员的作用。

正式技术评审

- FTR实际上是一组评审方式，包括走查（Walkthrough）、审查（Inspection）、轮查（Round-robin Review）以及其他软件小组的技术评估。
- 每次FTR都以会议形式进行，只有经过适当的计划、控制和参与，FTR才能获得成功。
- 在后面的段落中，我们给出了类似于走查的典型正式技术评审的指导原则。



正式技术评审

评审会议

不论选择何种FTR形式，每个评审会议都应该遵守下面的约束：

- 评审会议（通常）应该在3到5个人之间进行。
- 应该进行提前准备，但是每人占用工作时间应该少于2小时。
- 评审会议时间应该不超过2小时。

在上述约束之下，显然FTR应该关注的是整个软件中的某个特定(且较小)部分。例如，不要试图评审整个设计，而是对每个模块或者一小组模块进行走查。当FTR的关注范围较小时，发现错误的可能性更大。

正式技术评审

评审会议

- FTR关注的是某个工作产品（例如，一部分需求规格说明、一份详细的构件设计、一个构件的源代码清单)。
- 开发这一产品的个人(即“生产者”)通知项目管理者工作产品已经完成，需要进行评审。
- 项目管理者与“评审主席”联系，主席负责评估工作产品是否准备就绪，创建副本，并将其分发给两到三个“复审者”以便事先准备。
- 每个评审者应该花1到2个小时评审工作产品、做笔记或者用其他方法熟悉这一工作。
- 与此同时，评审主席也对工作产品进行评审、并制定评审会议的日程表，通常安排在第二天开会。

正式技术评审

评审会议

- 评审会议由评审会主席、所有评审者和生产者参加。其中一个评审者还充当“记录员”的角色，负责记录（书面的）在评审过程中发现的所有重要问题。
- FTR一般从介绍会议日程并由生产者做简单的介绍开始。然后由生产者“走查”该工作产品，并对材料做出解释，而评审者则根据预先的准备提出问题。当发现了有根据的问题或错误时，记录员逐一加以记录。

正式技术评审

评审会议

■ 在评审结束时，所有 FTR 与会者必须做出以下决定中的一个：

- 可以不经修改而接受该工作产品；
- 由于严重错误而否决该工作产品（错误改正后必须再次进行评审）
- 暂时接受该工作产品（发现一些必须改正的小错误，但是不再需要进行评审)。

作出决定之后，所有FTR与会者都需要签名，以表示他们参加了此次FTR，并且同意评审小组所做的决定。

正式技术评审

评审报告和记录保存

- 在FTR期间，由一名评审者（记录员）主动记录所有提出的问题。在评审会议结束时要对这些问题进行汇总，并生成一份“评审问题清单”。此外，还要完成一份“正式技术评审总结报告”。评审总结报告中要回答以下 3 问题：

1. 评审什么?
2. 由谁评审?
3. 发现了什么，结论是什么?

正式技术评审

评审报告和记录保存

- 评审总结报告通常是一页纸左右（可能还有附件）。它是项目历史记录的一部分，有可能被分发给项目负责人和其他感兴趣的参与方。
- 评审问题清单有两个作用：（1）标识产品中存在问题的区域；（2）作为行动条目检查表以指导生产者进行改正。通常将评审问题清单附在总结报告的后面。
- 为了保证评审问题清单中的每一条目都得到适当的改正，建立跟踪规程非常重要。只有做到这一点，才能保证提出的问题真正得到控制。方法之一就是将跟踪的责任指派给评审会主席。

正式技术评审

评审指导原则

1. 评审工作产品，而不是评审生产者。FTR涉及他人和自已。如果进行得适当，FTR可以使所有参与者体会到温暖的成就感。如果进行得不恰当，则可能陷入一种审问的气氛之中。应该温和的指出错误，会议的气氛应该是轻松的和建设性的；不要试图贬低或羞辱他人。评审会主席应该引导评审会议，确保维护适当的气氛和态度，应立即终止已变得失控的评审。
2. 制定并遵守日程。各种类型会议的主要缺点之一就是：放任自流。必须保证FTR不要离题且按照计划进行。评审主席负有维持会议程序的责任，在有人转移话题时应该提醒他。

正式技术评审

评审指导原则

3. 限制争论和辩驳。在评审员提出问题时，未必所有人都认同该问题的严重性。不要花时间去争论这类问题，这类问题应该被记录在案，留到会后进行讨论。

4. 要阐明问题，但是不要试图解决所有记录的问题。评审不是一个解决问题的会议。问题的解决应该开发人员自己或是在个别人的帮助下放到评审会议之后进行。



正式技术评审

评审指导原则

5. 作笔记。有时候让记录员在黑板上做笔记是一种很好的方式，这样，在记录员记录信息时，其他的评审者可以推敲措辞，并确定问题的优先次序，或者，笔记可以直接输入到笔记本电脑。

6. 限制参与者人数。并坚持事先做准备。虽然两个人的比一个好，但是14个人并不一定就比4个人好。应该根据需要将参与评审的人员数量限制到最少。而且所有参与评审的小组成员都必须事先作好准备。评审会主席应该向评审员要求书面意见（以表明评审员的确对材料进行了评审）。

正式技术评审

评审指导原则

7. 为每个将要评审的工作产品建立检查清单。检查清单能够帮助评审会主席组织FTR会议，并帮助每位评审员将注意力集中到重要问题上。应该为分析、设计、编码、甚至测试等工作产品建立都建立检查清单。

8. 为FTR分配资源和时间。为了进行有效的评审，应该将评审作为软件过程中的任务列入进度计划，而且还要为评审结果所引发的不可避免的修改活动分配时间。

正式技术评审

评审指导原则

9. 对所有评审员进行有意义的培训。为了提高效率，所有评审参与者都应该接受某种正式培训。培训要强调的不仅有与过程相关的问题，而且还应该涉及评审的心理学因素。Freedman和Weinberg［FRE90］估计每20个人进行为期一个月的学习，将能够有效的参与评审。

10. 评审以前所做的评审。听取汇报对发现评审过程本身的问题十分有益。最早被评审的工作产品本身就是评审的指导原则。

    由于成功的评审涉及到许多变数（如参与者数量、工作产品类型、时间和长度、特定的评审方法等），软件组织应该在实验中确定何种方法最为对自己的特定环境最为适用。

基于统计的软件质量保证

基于软件的软件质量保证

- 基于统计的质量保证反映了一种在产业界不断增长的趋势：质量的量化。对于软件而言，基于统计的质量保证包含以下步骤：

1. 收集和分类软件缺陷信息。
2. 追溯每个缺陷的形成原因（例如，不符合规格说明、设计错误、违背标准、缺乏与客户的交流)。
3. 使用Pareto规则（80%的缺陷可以追溯到所有可能原因中的20%)，将这20%（重要的少数）原因分离出来。
4. 一旦找出这些重要的少数原因，就可以开始纠正引起缺陷的问题。

- “基于统计的质量保证”这一比较简单的概念代表的是创建自适应软件过程的一个重要步骤，要对软件过程进行修改，以改进那些引入错误的过程要素。

基于软件的软件质量保证

一个普通的例子

- 举一个例子来说明统计方法在软件工程工作中的应用。

- 假定软件开发组织收集了为期一年的错误和缺陷信息，其中有些错误是在软件开发过程中发现的，别外一些（缺陷）则是在软件交付给最终用户之后发现的。尽管发现了数以百计的不同问题，所以问题都可以追溯到下述原因中的一个（或几个）：（转下页）

基于软件的软件质量保证

一个普通的例子

- (接上页)
  - 不完整或错误的规格说明（IES）
  - 与客户交流中所产生的误解（MCC）
  - 故意违背规格说明（IDS）
  - 违反程序设计标准（VPS）
  - 数据表示有错（EDR）
  - 构件接口不一致（ICI）
  - 设计逻辑的错误（EDL）
  - 不完整或错误的测试（IET）
  - 不准确或不完整的文档（IID）
  - 将设计转换国程序设计语言时的错误（PLT）
  - 不清晰或不一致的人机界面（HCI）
  - 其他（MIS）



基于软件的软件质量保证

一个普通的例子

- 为了使用统计质量保证方法，需要建一张如下图所示的表格。

	总 计		严 重		中 等		微 小	
错误	数量	百分比	数量	百分比	数量	百分比	数量	百分比
IES	205	22%	34	27%	68	18%	103	24%
MCC	156	17%	12	9%	68	18%	76	17%
IDS	48	5%	1	1%	24	6%	23	5%
VPS	25	3%	0	0%	15	4%	10	2%
EDR	130	14%	26	20%	68	18%	36	8%
ICI	58	6%	9	7%	18	5%	31	7%
EDL	45	5%	14	11%	12	3%	19	4%
IET	95	10%	12	9%	35	9%	48	11%
IID	36	4%	2	2%	20	5%	14	3%
PLT	60	6%	15	12%	19	5%	26	6%
HCI	28	3%	3	2%	17	4%	8	2%
MIS	56	6%	0	0%	15	4%	41	9%
总计	942	100%	128	100%	379	100%	435	100%

统计SQA数据收集

基于软件的软件质量保证

一个普通的例子

- 表中显示IES、MCC和EDR是造成所有错误的53%的“重要的少数”原因。
- 但要注意，在只考虑严重错误时，应该将IES、EDR、PLT和EDL作为“重要的少数”原因。
- 一旦确定了这些重要的少数原因，软件开发组织就可以开始采取改正行动了。例如，为了改正MCC错误，软件开发者可能要采用需要收集技术，以提高与客户交流和规格说明的质量。为了改正EDR错误，开发者可能要使用工具进行数据建模，并进行理为严格的数据设计评审。
- 值得注意的是，改正行动主要是针对“重要的少数”。随着这些“重要的少数”原因不断改正，新的“重要的少数”原因将提到改正日程上来。

基于软件的软件质量保证

一个普通的例子

- 统计SQA及Pareto原则的应用可以用一句话概括：

  将时间用于真正重要的地方，但是首先你必须知道什么是真正重要的！

基于软件的软件质量保证

软件工程中的六西格玛

- 六西格玛是目前产业界应用最广泛的基于统计的质量保证策略。“是严格且规范的方法学，它运用数据和统计分析，通过识别和消除制造以及服务相关过程中的‘缺陷’来测量和改进企业的运转状况”。
- 六西格玛一词来源于6个标准偏差 - 每百万个操作发生3.4个偏差（缺陷），它意味着非常高的质量标准。

基于软件的软件质量保证

软件工程中的六西格玛

- 六西格玛是目前产业界应用最广泛的基于统计的质量保证策略。“是严格且规范的方法学，它运用数据和统计分析，通过识别和消除制造以及服务相关过程中的‘缺陷’来测量和改进企业的运转状况”。
- 六西格玛一词来源于6个标准偏差 - 每百万个操作发生3.4个偏差（缺陷），它意味着非常高的质量标准。

基于软件的软件质量保证

软件工程中的六西格玛

六西格玛方法学有3个主要的核心步骤：

- 定义：通过与客户交流的方法来定义客户需求、可交付的产品及项目目标。
- 测量：测量现有的过程及其产品，以确定当前的质量状况（收集缺陷度量信息）
- 分析：分析缺陷度量信息，并挑选出重要的少数原因。

如果某个现有软件过程是适当的，只是需要改进，六西格玛还需要另外两个核心步骤：

- 改进：通过消除缺陷根本原因的方式来改进过程。
- 控制：控制过程以保证以后的工作不会再引入缺陷原因。

以上3个核心步骤和另两个附加步骤有时叫做 DMAIC方法。

基于软件的软件质量保证

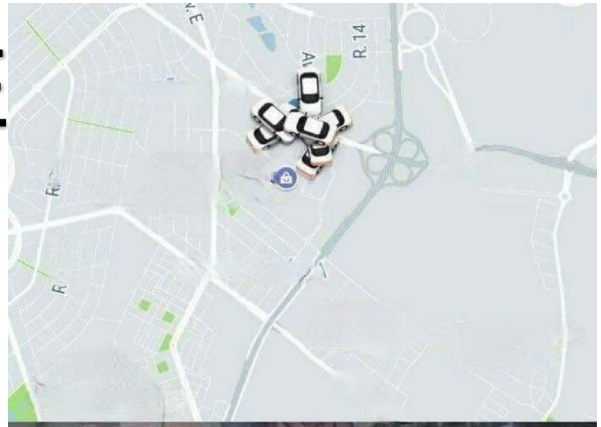
软件工程中的六西格玛

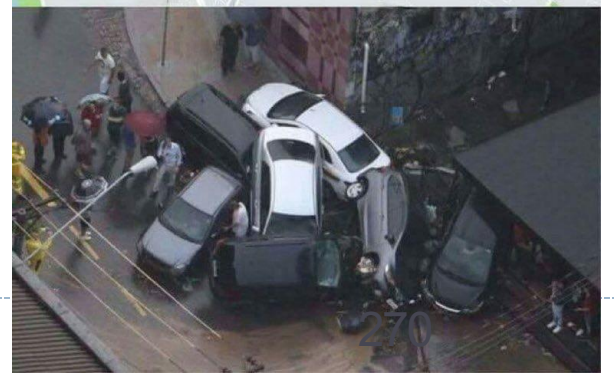
如果某组织正在开发软件过程（而不是改进现有的过程），则需要增加下面两个核心步骤：

- 设计：设计过程，以达到（1）避免缺陷产生的根本原因；（2）满足客户需求。
- 验证：验证过程模型是否确实能够避免缺陷，并且满足客户需求。

上述步骤有时称为DMADV（定义、测量、分析、设计和验证）方法。

软件可靠性





软件可靠性

- 与其他质量因素不同，软件可靠性可以通过历史数据和开发数据直接测量和估算出来。
- 按统计术语所定义的软件可靠性是： “在特定环境和特定时间内，计算机程序正常运行的概率”
- 举个例子来说，如果程序X在8小时处理时间内的可靠性估计为0.999，也就意味着，如果程序X执行1000次，每次运行8小时处理时间（执行时间），则1000次中正确运行（无失效）的次数可能是999次。



软件可靠性

- 无论何时谈到软件可靠性，都会涉及一个关键问题：术语失效（failure）一词是什么含义？在有关软件质量和软件可靠性的任何讨论中，失效意味着与软件需求的不符，但是这一定义是有等级之分的。
- 失效可能仅仅是令人厌烦的，也可能是灾难性的。有的失效可以在几秒钟之内得到纠正，有的则需要几个星期甚至几个月的时间才能纠正。
- 让问题更复杂的是，纠正一个失效事实上可能会引入其他的错误，而这些错误最终又会导致其他的失效。



软件可靠性

可靠性和可用性的测量

- 软件可靠性问题几乎总能追溯到设计或实现中的缺陷。
- 早期的软件可靠性测量工作试图将硬件可靠性理论中的数学公式外推来进行软件可靠性的预测。大多数与硬件相关的可靠性模型依据的是由于“磨损”而导致的失效，而不是由于设计缺陷而导致的失效。
- 在硬件中，由于物理磨损（如温度、腐蚀、震动的影响）导致的失效远比与设计缺陷有关的失效多。
- 不幸的是，软件恰好相反。实际上，所有软件失效都可以追溯到设计或实现问题上，磨损在这里根本没有影响。

软件可靠性

可靠性和可用性的测量

- 当我们考虑基于计算机的系统时，可靠性的简单测量是“平均失效间隔时间”（Mean-Time-Between-Failure, MTBF）：

$$MTBF = MTTF + MTTR$$

- 其中，MTTF （Mean-Time-To-Failure）和MTTR（Mean-Time-To-Repair）分别是“平均失效时间”和“平均维修时间”
- 重要的是要注意，平均无故障时间和相关测量是基于CPU时间，而不是挂钟时间。
- 可用性的某些方面与失效没有关系。例如，计划停机也使软件不可用。



软件可靠性

可靠性和可用性的测量

- 许多研究人员认为MTBF是一个远比其他与质量有关的软件度量更有用的测量指标
- 简而言之，最终用户关心的是失效，而不是总缺陷数。
- 由于一个程序中包含的每个缺陷所具有的失效率不同，总缺陷数难以表示系统的可靠性。
- 例如，一个程序已运行3000处理器小时没有发生故障。该程序中的许多缺陷在被发发现之前可能有数万小时都不会被发现。隐藏如此深的错误的MTBF可能是30000甚至60000处理器小时。其他尚未发现的缺陷，可能有4000或5000小时的失效率。即使第一类错误（那些具有长时间的MTBF）都去除了，对软件可靠性的影响也是微不足道的。

软件可靠性

可靠性和可用性的测量

- 然而，MTBF可能会产生问题的原因有两个：（1）它突出失效之间的时间跨度，但不会为我们提供一个凸显的失效率；（2）MTBF可能被误解为平均寿命，即使这不是它的含义。
- 可靠性另一个可选的衡量是失效率（Failures-In-Time，FIT）：一个部件每10亿机时发生多少次失效的统计测量。因此 1FIT 相当于每10亿机时发生一次失效。



软件可靠性

可靠性和可用性的测量

- 除可靠性测量之外，还应该进行可用性测量。
- 软件可用性是指在某个给定时间点上程序能够按照需求执行的概率。其定义为：

$$\text{可用性} = \frac{\mathbf{MTTF}}{\mathbf{MTTF} + \mathbf{MTTR}} \times \mathbf{100\%}$$

- MTBF 可靠性测量对 MTTF 和 MTTR 同样敏感，而可用性测量在某种程序上对 MTTR 较为敏感，MTTR是对软件可维护性的间接测量。

软件可靠性

软件安全

- 软件安全是一种软件质量保证活动，它主要用来识别和评估可能对软件产生负面影响并促使整个系统失效的潜在灾难。如果能够在软件过程的早期阶段识别出这些灾难，就可以指定软件设计特性来消除或控制这些潜在的灾难。
- 建模和分析过程可以视为软件安全的一部分。开始时，根据危险程序和风险高低对灾难进行识别和分类。
- 一旦识别出系统级的灾难，就可运用分析技术来确定这些灾难发生的严重性和概率。为了达到高效，应该将软件置于整个系统中进行分析。
- 例如，一个微小的用户输入错误（人也是系统组成部分）有可能会被软件错误放大，产生将机械设备置于不正确位置的控制数据，此时当且仅当外部环境条件满足时，机械设备的不正确位置将引发灾难性的失效。

软件可靠性

软件安全

- 一旦完成了灾难识别和分析，就可以进行软件中与安全相关的需求规格说明了。在规格说明中包括一张不希望发生的事件清单，以及针对这些事件所希望产生的系统响应。这样就指明了软件在管理不希望发生的事件方面应起的作用。
- 软件可靠性使用统计分析的方法来确定软件失效发生的可能性，而失效的发生未必导致灾难或灾祸。软件安全则考察失效导致灾难发生的条件。也就是说，不能在真空中考虑失效，而是要在整个计算机系统及其所处环境的范围内加以考虑。

小结

- 软件质量管理是在软件过程中的每一步都进行的“普适性活动” - 包括质量控制和质量保证两部分。SQA包括：对方法和工具有效应用的规程，正式技术评审，测试策略和技术，变更控制规程，保证符合标准的规程，以及测量和报告机制。
- 软件质量，是计算机程序的一种属性，其定义是“与明确和隐含定义的需求的符合程度”。软件质量本质上的复杂性也使SQA复杂化。但是一般说来，软件质量包括了许多不同的产品和过程因素及其相关的度量信息。
- 软件评审是最为重要的质量控制活动之一。评审作为所有软件工程活动的过滤器，当发现及改正错误的成本比较小时，就消除错误。正式技术评审是一种典型的评审会议，在实践中已经证明这种形式对于发现错误极其有效。
- 为了正确软件质量保证，必须收集、评估和发布有关软件工程过程的数据。基于统计的SQA有助于提高产品和过程本身的质量。软件可靠性模型将测量加以扩展，能够由所收集的缺陷数据推导出相应的失效率和进行可靠性预测。

思考题

1. 什么是软件质量控制?
2. 质量成本的构成有哪些?
3. 如何定义软件质量?
4. SQA小组的作用是什么?
5. 我们什么时候进行FTR，我们的目标是什么?



思考题

6. 假设需求模型引入了10个错误，每个错误按2：1的比例在设计阶段放大，设计阶段引入了另外的20个错误，并且这20个错误按1.5：1的比例在编码阶段放大，在编码阶段又引入了另外30个错误。进一步假设，所有单元测试会发现所有错误的30%，集成测试将找到剩余错误的30%，验证测试会发现剩余错误的50%。没有进行评审，有多少错误将被发布到现场？
7. 重新考虑第6题所述情形，但现在进行了需求评审、设计评审和代码评审，并且这些步骤能有效地发现所有错误的60%。有多少错误将被发布到现场？
8. 重新考虑第6题和第7题所述情形，对于每个发布到现场的错误，发现和改正的成本是4800美元，在评审时发现并改正每个错误花费240美元，通过进行评审节约多少钱？

思考题

9. 完成基于统计的SQA需要哪些步骤?
10. 六西格玛方法学的核心步骤 是什么?

第9章：软件配置管理

要点浏览

概念

- 在开发计算机软件时，会发生变更。而且由于变更的发生，迫使你必须有效地去管理变更。
- 变更管理，也称为软件配置管理（SCM），是一组活动，它通过以下方式管理变更：识别可能发生变更的工作产品，建立这些工作产品之间的关系，制定管理这些工作产品的不同版本的机制，控制所施加的变更，审核和报告所发生的变更。

要点浏览

步骤

- 由于在开发软件时会产生很多工作产品，所以必须唯一标识每一个工作产品。
- 标识了工作产品之后，就可以制定版本和变更控制机制了。
- 为了保证变更发生时维护好质量，就要审核变更过程；
- 为了保证将变更通知到那些需要知道的人员，就要报告变更。



要点浏览

工作产品

- 软件配置管理计划定义了变更管理的项目策略。
- 此外，当启动正式SCM时，变更控制过程将产生软件变更请求、变更报告和工作变更工单。

引言

引言

- 变更管理，通常叫做软件配置管理（SCM），是贯穿于整个软件过程的普适性活动。因为变更可能随时发生，所以SCM活动的目标就是为了：
  （1）标识变更
  （2）控制变更
  （3）保证正确地实现变更
  （4）向那些利害相关的人员报告变更

- 软件配置管理的主要目标是：当变更发生时，能够提高适应变更的容易程度，并且能够减少所需花费的工作量。

引言

- 变更管理，通常叫做软件配置管理（SCM），是贯穿于整个软件过程的普适性活动。因为变更可能随时发生，所以SCM活动的目标就是为了：
  （1）标识变更
  （2）控制变更
  （3）保证正确地实现变更
  （4）向那些利害相关的人员报告变更

- 软件配置管理的主要目标是：当变更发生时，能够提高适应变更的容易程度，并且能够减少所需花费的工作量。

引言

- 软件维护和软件配置管理的区别：
- 软件维护是一组软件工程活动，它发生在软件已经交付给客户并投入运行之后；
- 而软件配置管理是一组跟踪和控制活动，它开始于软件工程项目开始之时，并且终止于该软件被淘汰之时。



软件配置管理

软件配置管理

- 软件过程的输出信息主要分为三类：
  （1）计算机程序（源代码和可执行程序）
  （2）描述计算机程序的文档（针对技术开发者和用户）
  （3）数据（包含在程序内部的数据，或程序外部的数据）

- 在软件过程中产生的所有信息项总称为软件配置。

软件配置管理

- 变更的起因是什么：
  （1）新的业务或市场条件导致产品需求或业务规则的变更
  （2）新的客户需求，要求修改信息系统产生的数据、产品提供的功能或系统提供的服务。
  （3）企业改组或扩大/缩小规模，导致项目优先级或软件工程团队结构的变更
  （4）预算或进度安排的限制，导致系统或产品的重新定义。

- 软件配置管理是一组在计算机软件的整个生命周期内管理变更的活动。可以把SCM看作是应用于整个软件过程的软件质量保证活动。

软件配置管理

- 变更的起因是什么：
  （1）新的业务或市场条件导致产品需求或业务规则的变更
  （2）新的客户需求，要求修改信息系统产生的数据、产品提供的功能或系统提供的服务。
  （3）企业改组或扩大/缩小规模，导致项目优先级或软件工程团队结构的变更
  （4）预算或进度安排的限制，导致系统或产品的重新定义。

- 软件配置管理是一组在计算机软件的整个生命周期内管理变更的活动。可以把SCM看作是应用于整个软件过程的软件质量保证活动。

软件配置管理：SCM 场景

- 典型的CM工作场景包括：
  总体负责软件小组的项目经理、总体负责CM规程和方针的配置管理员、负责开发和维护软件产品的软件工程师以及使用软件产品的客户。
  在该场景中，我们假定由6个人组成的团队正在开发一个约15 000行代码的小型软件。

- 在操作级别上，SCM场景包括多种角色和任务。

软件配置管理

软件配置管理：SCM 场景

- 项目经理的职责是保证在确定的时间框架内开发产品。因此项目经理必须对软件的开发进展情况进行监控，找出问题，并对问题做出反应。这可以通过建立和分析软件系统状态报告，并执行对系统的评审来完成。
- 配置管理员的职责不仅是要保证代码的创建、变更和测试要遵循相应的规程和方针，还要使项目的相关信息容易得到。
- 软件工程师的目标是高效地工作。也就是说，软件工程师在代码的创建和测试以及编写支持文档时不做不必要的相互交流；但同时，软件工程师又尽可能地进行有效的沟通和协调。
- 客户只是使用产品。

软件配置管理

软件配置管理：SCM 场景

- 项目经理的职责是保证在确定的时间框架内开发产品。因此项目经理必须对软件的开发进展情况进行监控，找出问题，并对问题做出反应。这可以通过建立和分析软件系统状态报告，并执行对系统的评审来完成。
- 配置管理员的职责不仅是要保证代码的创建、变更和测试要遵循相应的规程和方针，还要使项目的相关信息容易得到。
- 软件工程师的目标是高效地工作。也就是说，软件工程师在代码的创建和测试以及编写支持文档时不做不必要的相互交流；但同时，软件工程师又尽可能地进行有效的沟通和协调。
- 客户只是使用产品。

软件配置管理

软件配置管理：配置管理系统元素

- 开发配置管理系统时应具备四个重要元素：

构件元素 - 是一组具有文件管理系统（如，数据库）功能的工具，使我们能够访问和管理每一个软件配置项。

过程元素 - 是一个规程和任务的集合，它为所有参与管理，开发和使用计算机软件的人员定义了变更管理的有效方法。

构造元素 - 是一组自动软件构造工具，用以确保装配了正确的有效构件（即正确的版本）集。

人员元素 - 由实施有效SCM的软件团队使用的一组工具和过程特性（包括其他CM元素）

软件配置管理

软件配置管理：基线

- 基线是一个软件配置管理概念，它能够帮助我们在不严重阻碍合理变更的条件下控制变更。IEEE是这样定义基线的：
  已经通过正式评审和批准的规格说明或产品，它可以作为进一步开发的基础，并且只有通过正式的变更控制规程才能修改它。

- 在软件配置项成为基线之前，可以迅速而随意地进行变更。然而，一旦成为基线，这时虽然可以进行变更，但是必须应用特定的、正式的规程来评估和验证每一个变更。

- 在软件工程范畴中，基线是软件开发中的里程碑，其标志是在正式技术评审中已经获得批准的一个或多个软件配置项（SCI）的发布。

软件配置管理

软件配置管理：基线

- 例如，某设计模型的元素已经形成文档并通过评审，错误已经被发现并得到纠正，一旦该模型的所有部分都经过了评审、纠正和批准，该设计模型就成为一个基线

- 任何对程序体系结构（在设计模型中已形成文档）的变更只能在每一个变更被评估和批准之后方可进行。虽然可以任意细节层次上定义基线，但最常见的软件基线如下图所示。

该图说明了基线的形成过程。软件工程任务可能会产生一个或多个SCI，在这些SCI经过评审并被批准之后，就可以将它们放置到项目数据库（也称为项目库或软件中心存储库）中。

图中的箭头说明了某个已成为基线的SCI的修改路径。

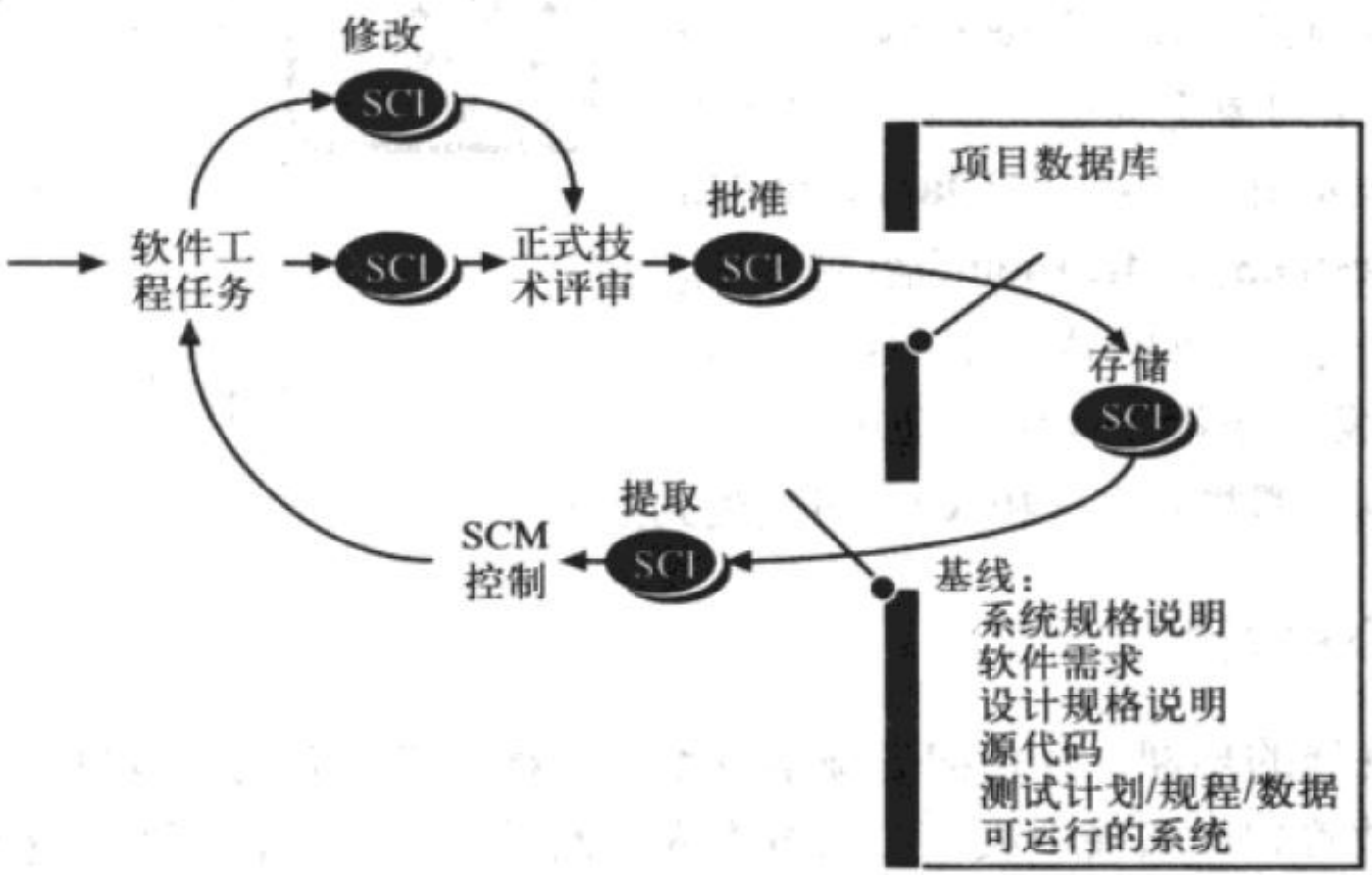


图8 - 1 基线化的 SCI 和项目数据库

软件配置管理

软件配置管理：软件配置项

- 软件配置项是在软件工程过程中创建的信息。
- 在极端情况下，大型规格说明中的一节、大型测试用例集中的一个测试用例都可以看作是一个SCI。
- 再实际点，一个SCI可以是一个文档，一个全套的测试用例，也可以是一个已命名的程序构件（例如，一个C++函数或一个JAVA程序）。



软件配置管理

软件配置管理：软件配置项

- 除了来自软件工程工作产品的SCI，很多软件工程组织也将软件工具列入配置管理的范畴，即，特定版本的编辑器、编译器、浏览器以及其他自动化工具都被“固定”作为软件配置的一部分。
- 因为这些工具被用于生成文档、源代码和数据，所以当对软件配置进行改变时，必然要用到它们。
- 虽然问题并不多见，但一个工具的新版本(如，编译器)有可能产生和原版本不同的结果。因此，就象它们协助开发的软件一样，工具也可以被基线化为完整配置管理过程的一部分。



软件配置管理

软件配置管理：软件配置项

- 在现实中，是将 SCI 组织成配置对象，这些配置对象具有自己的名字，并且按类别存储在项目数据库中。
- 一个配置对象具有一个名称和多个属性，并通过关系来表示与其他对象的“关联”。

在右图中，分别定义了配置对象的

- DesignSpecification（设计规格说明）、
- DataModel（数据模型）、
- C ommponentN（模块N）、
- SourceCode（源代码）
- 和TestSpecification（测试规格说明）。

各个对象的关系如图中箭头所示，单向箭头表示组成关系，即D ataModel和ComponentN是DesignSpecification的组成部分。双向箭头说明对象之间的内在联系，如果SourceCode对象发生变化，软件工程师通过查看内在联系能够确定哪些对象（和SCI）可能受到影响（这些关系可以在数据库内部定义）

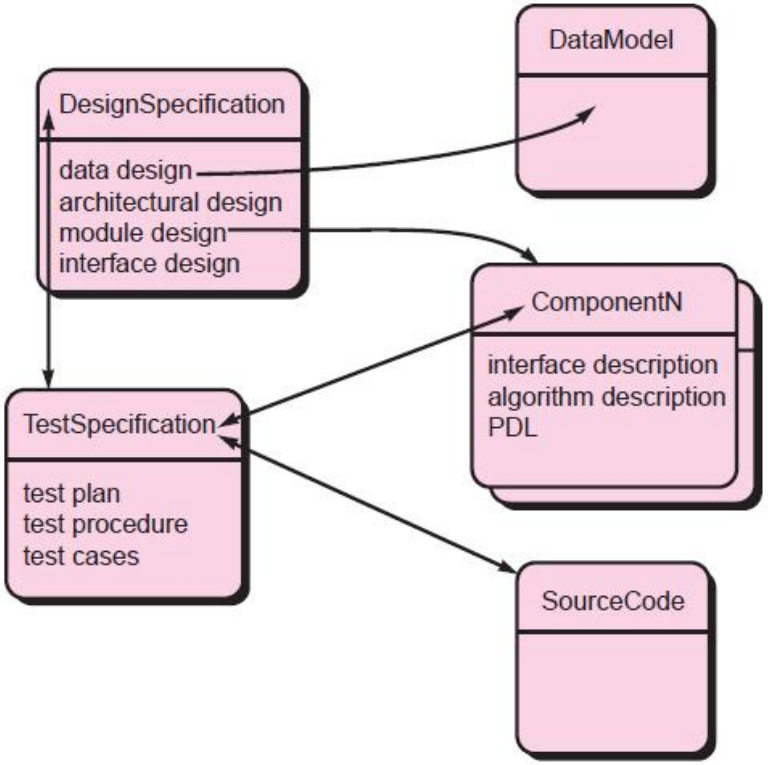


图8 - 2 配置对象

SCM 中心存储库

SCM中心存储库

在软件工程发展的早期，软件配置项是以纸质文档的形式进行维护。这种方法存在许多问题：

1. 难以找出所需要的配置项
2. 难以确定更改了哪些配置项，是什么时候以及由谁更改的
3. 构造现有程序的新版本耗时，并且很可能出错
4. 实际上根本不可能详细描述各配置项或各配置项之间的复杂关系。



SCM中心存储库

如今，是在项目数据库或中心存储库（repository）中维护SCI的。

- 在软件工程历史早期，中心存储库只是一个人（程序员），程序员必须要记住所有与软件项目有关的信息的位置，还必须能够回想起那些从未记录下来的信息，并且能够重构那些已经失去的信息。
- 今天，中心存储库是一个“物件” - 一个做为软件工程信息聚集和存储中心的数据库，而人（软件工程师）的角色是通过与存储库集成在一起与中心存储库进行交互。



SCM中心存储库

中心存储库的作用

- SCM中心存储库是一 组机制和数据结构，它使软件团队可以有效地管理变更。通过保证完整性，信息共享和数据集成，它具有数据库管理系统的一般功能。此外，SCM中心存储库为软件工具的集成提供了中枢，它是软件过程流的核心。它能够使软件工程工作产品强制实施统一的结构和格式。
- 为了实现这些功能，用术语元模型（meta-model）来定义中心存储库。元模型决定了在中心存储库中信息如何存储、如何通过工具访问数据、软件工程师如何查看数据、维护数据完全性和完整性的能力如何，以及将现有的模型扩展以适应新需求时的容易程序如何等。

SCM中心存储库

一般特征和内容

中心存储库的特征和内容可以从两个方面来理解：

- 中心存储库存储什么
- 中心存储库提供什么特定服务

中心存储库存储的表示类型、文档和工作产品的详细分类如右图。

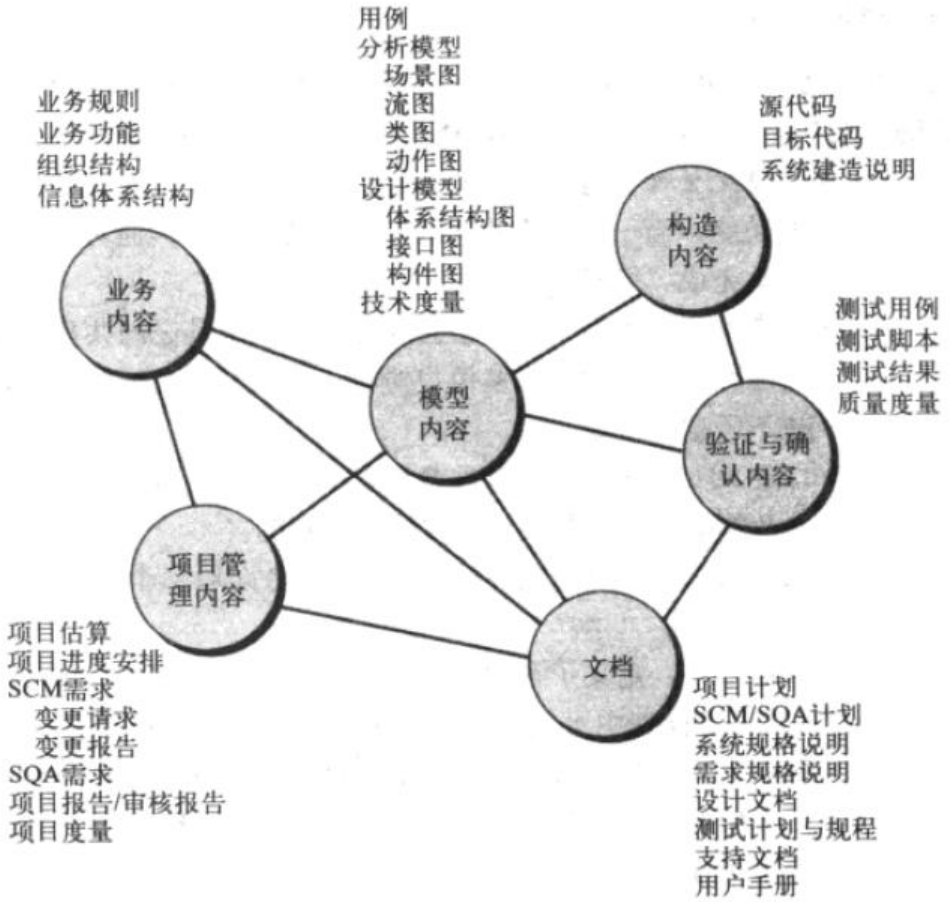


图8 - 3 中心存储库的内容

SCM中心存储库

一般特征和内容

一个健壮的中心存储库能够提供两种不同类型的服务：

- 期望从任何一个复杂的数据库管理系统得到相同的服务类型
- 特定于软件工程环境的服务类型

SCM中心存储库

一般特征和内容

作为软件工程团队的中心存储库，应该

- 集成或直接支持过程管理功能
- 支持在中心存储库中管理SCM功能特定规则和维护数据
- 提供与其他软件工程工具的接口
- 能够存储各种数据对象（如，文本、图形、视频、音频）



SCM中心存储库

SCM 特征

为了支持SCM，中心存储库必须具有下列特征的工具集：

- 版本控制
- 依赖性跟踪和变更管理
- 需求跟踪
- 配置管理
- 审核跟踪

SCM中心存储库

SCM 特征：版本控制

- 随着项目的进展，每个工作产品都可能有很多版本。中心存储库必须能保存所有这些版本，以便有效地管理产品发布，并允许开发者在测试和调试过程中可以返回到早先的版本。
- 中心存储库必须能控制各种类型的对象，包括文本、图形、位图、复杂文档及一些特殊对象，如屏幕、报告定义、目标文件、测试数据和结果等。在成熟的中心存储库中可以按任意粒度跟踪对象版本，例如，可以跟踪单个数据定义或一组模块。



SCM中心存储库

SCM 特征：依赖性跟踪和变更管理

- 中心存储库要管理所存储的配置对象之间的各种关系。这些关系包括：企业实体与过程之间的关系、应用系统设计各部分之间的关系、设计构件与企业信息体系结构之间的关系、设计元素与其他可交付工作产品之间的关系等。其中有些仅仅是关联关系，而有些是依赖关系或强制关系。
- 例如，如果修改了某UML类图，则中心存储库能够检测出是否有相关联的类、接口描述和代码构件也需要进行修改，并且能够提醒开发人员哪些SCI受到影响。

SCM中心存储库

SCM 特征：需求跟踪

- 这种特殊的功能依赖于相关管理，并可以跟踪由特定需求规格说明产生的所有设计构件、架构构件以及可交付产品（正向跟踪）。
- 此外，还能够用来辨别指定的工作产品是由哪个需求产生的（反向跟踪）



SCM中心存储库

SCM 特征：需求跟踪

- 这种特殊的功能依赖于相关管理，并可以跟踪由特定需求规格说明产生的所有设计构件、架构构件以及可交付产品（正向跟踪）。
- 此外，还能够用来辨别指定的工作产品是由哪个需求产生的（反向跟踪）



SCM中心存储库

SCM 特征：配置管理

- 配置管理设施能够跟踪表示特定项目里程碑或产品发布的一系列配置

SCM中心存储库

SCM 特征：审核跟踪

- 审核跟踪使我们能够了解变更是什么时候、什么原因以及由谁完成等信息。
- 变更的根源信息可以作为中心存储库中特定对象的属性进行存储。
- 每当设计元素进行了修改，中心存储库触发机制有助于提示开发人员或创建审核信息（如变更理由）的工具。



SCM 过程

SCM过程

软件配置管理过程中定义的一系列任务具有4个主要目标：

（1）统一标识软件配置项

（2）管理一个或多个软件配置项的变更

（3）便于构造应用系统的不同版本

（4）在配置随时间演化时，确保能够保持软件质量



SCM过程

如图，SCM任务可以看做是一个同心圆的层次结构。

软件配置项（SCI）在它们的有效生存期内都要从内到外经历各个层次，最终成为某应用程序或系统的一个或多个版本软件配置的组成部分。

当一个SCI进入某层时，该SCM过程层次所包含的活动可能适用，也可能不适用。例如，在创建一个新的SCI时，必须对其进行标识，但是，如果对该SCI没有任何变更请求，那就不必应用变更控制层的活动，而直接将该SCI指定给软件的特定的版本。

为了进行配置审核，就要维护SCI记录，并将这些记录报告给那些需要知道的人员。

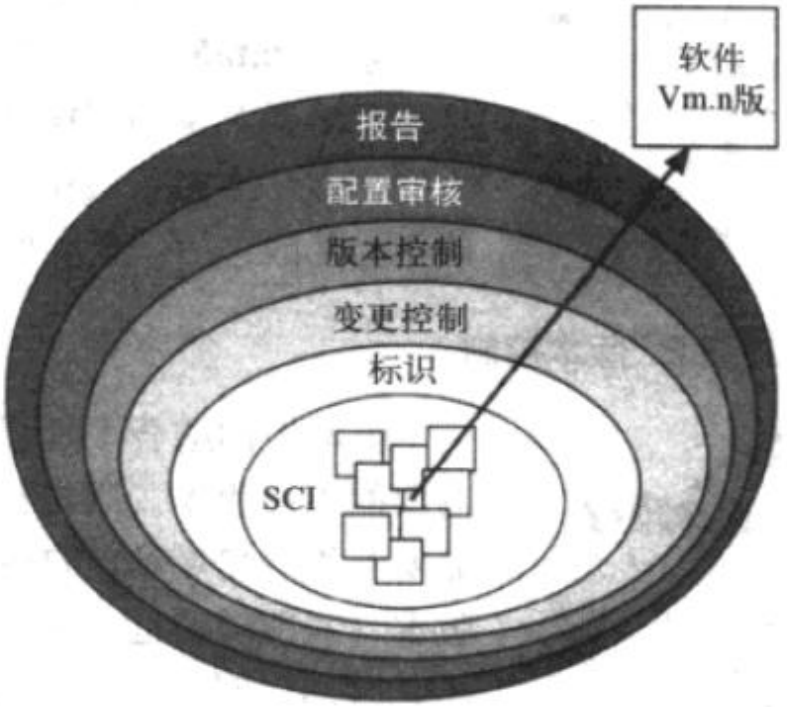


图8 - 4 SCM过程的层次

SCM过程

SCM过程：软件配置中的对象标识

为了控制和管理软件配置项，必须对每个配置项单独命名，然后用面向对象的方法进行组织。有两种类型的对象可以被标识：基本对象和聚合对象。

基本对象是软件工程师在分析、设计、编码或测试过程中创建的“信息单元(unit of text)”。例如，一个基本对象可以是需求规格说明的一个段落、设计模型的一个部件、一个构件的源代码或用于测试代码的一组测试用例。

聚合对象是基本对象和其他聚合对象的集合，在图8-2中的DesignSpecification就是一个聚合对象。在概念上，可将其视为一个已命名的（已标识的）指针列表，指向诸如 ArchitectureModel 和 DataModel 的聚合对象，以及诸如 ComponentN 和 UMLClassDiagramN的基本对象。

SCM过程

SCM过程：软件配置中的对象标识

每个对象均具有一组能够唯一地标识它的独特的特征：名称、描述、资源表及“实现”。

对象名称是能够清楚地标识对象的一个字符串；

对象描述是一个数据项的列表，它能够标识出该对象所表示的SCI类型（如，模型元素、程序、数据）、项目标识符以及变更和（或）版本信息；

资源是“对象提供的、处理的、参考的的实体，或是对象所需要的实体”。例如，数据类型、特定功能或者甚至是变量名都可以认为是对象资源；

“实现”是一个指针，对于基本对象，该指针指向其文本单元；对于聚集对象，该指针为空。

SCM过程

SCM过程：软件配置中的对象标识

每个对象均具有一组能够唯一地标识它的独特的特征：名称、描述、资源表及“实现”。

对象名称是能够清楚地标识对象的一个字符串；

对象描述是一个数据项的列表，它能够标识出该对象所表示的SCI类型（如，模型元素、程序、数据）、项目标识符以及变更和（或）版本信息；

资源是“对象提供的、处理的、参考的的实体，或是对象所需要的实体”。例如，数据类型、特定功能或者甚至是变量名都可以认为是对象资源；

“实现”是一个指针，对于基本对象，该指针指向其文本单元；对于聚集对象，该指针为空。

SCM过程

SCM过程：软件配置中的对象标识

标识配置对象时也可以考虑各个标识对象之间的关系，例如，使用下面的简单表示：

E-R diagram 1.4 〈part-of〉 data model；

Data model〈part-of〉 Design Specification；

这样，我们创建了一个SCI的层次结构。再如，

类图〈part-of〉需求模型

需求模型〈part-of〉需求规格说明

SCM过程

SCM过程：软件配置中的对象标识

在很多情况下，在对象层次结构中，对象是跨对象层次分支互相关联的。这些交叉的结构关系可以用下面的方式表示：

DataModel〈interrelated〉Data FlowModel；

DataModel〈interrelated〉TestCaseClassM；

第一种相互关系是存在于复合对象之间，而第二种相互关系则是存在于聚合对象（DataModel）和基本对象（TestCaseClassM）之间。

SCM过程

SCM过程：软件配置中的对象标识

在配置对象的标识过程中必须注意到，对象在整个软件过程中是不断演化的。在一个对象被确定为基线之前，它可能会变更很多次，甚至在已经被确定为基线之后，变更也可能会经常发生。

SCM过程

SCM过程：软件配置中的对象标识

在配置对象的标识过程中必须注意到，对象在整个软件过程中是不断演化的。在一个对象被确定为基线之前，它可能会变更很多次，甚至在已经被确定为基线之后，变更也可能会经常发生。

SCM过程

SCM过程：版本控制

- 版本控制结合了规程和工具，可以用来管理在软件过程中所创建的配置对象的不同版本。
- 版本控制系统实现或者直接集成了4个主要功能：
  (1)存储所有相关配置对象的项目数据库（中心存储库）
  (2)存储配置对象所有版本的版本管理功能
  (3)使用软件工程师能够收集所有相关配置对象和构造软件特定版本的制作功能
  (4)问题跟踪（也叫做错误跟踪）功能

SOFTWARE TOOLS

并发版本系统（CVS）

通过工具来实现版本控制对于实现高效变更管理是必须的。并发版本系统（Concurrent Version System, CVS）是在版本控制中普遍使用的工具。最初是为源代码设计的，但是可运用于任何文本文件。CVS系统的功能有：（1）建立简单的中心存储库；（2）以一个命名文件来维护文件的所有版本，仅仅存储原始文件的各个渐进版本之间的差异；（3）为每位开发者建立不同的工作目录以实现相互隔离，可避免同时对某个文件进行修改。当开发者完成各自的工作后，由CVS合并处理所有的修改。

要注意的是，CVS不是一个“构造”系统，即它不能构造软件的特定版本。CVS必须集成其他工具（例如，Makefile）才能完成这项工作。CVS也不能实现变更控制过程（如变更请求、变更报告和错误跟踪）。

虽然有上述局限性，CVS仍然“是一个优秀的、开源的、网络透明的版本控制系统，从单个开发者到大型、分散的团队都可以使用”[CVS07]。CVS的客户/服务器体系结构使用户可以从任何有Internet连接的地方访问文件，且其开源理念使其适用于大部分流行的平台。

可以免费获得Windows、Mac OS、LINUX和UNIX环境下的CVS，更详细的信息请查看[CVS07]。

SCM过程：变更控制

- 平衡问题
  太多的变更控制会给我们带来问题
  太少的变更控制又会给我们带来其他问题
- 对于大型的软件工程项目，不受控制的变更会迅速导致混乱。对于这种大型项目，变更控制应该将人为制定的规程与自动工具结合起来。



- 变更控制过程：
  (1)提交一个变更请求之后，要对其进行多个方面的评估：技术指标、潜在的副作用、对其他配置对象和系统功能的整体影响，以及变更的预计成本。评估的结果形成变更报告，由变更控制授权人（CCA）使用。
  (2)对每个被批准的变更，需要建立工程变更工单（ECO），ECO描述了将要进行的变更，必须要考虑的约束以及评审和审核的标准。
  (3)可以将要进行变更的对象放到 一个目录中，该目录只能由实施变更的软件工程师单独控制。
  (4)完成变更之后，版本控制系统可以更新原始文件。或者，可以将要进行变更的对象从项目数据库中“检出”，进行变更，并应用适当的SQA活动，然后，再将对象“检入”到数据库，并应用适当的版本控制机制构建该软件的下一个版本。



SCM过程

SCM过程：变更控制

- 以上版本控制机制与变更控制过程集成在一起，实现了变更管理的两个主要元素 - 访问控制和同步控制。
  访问控制负责管理哪个软件工程师有权限去访问和修改某个特定的配置对象
  同步控制协助保证两个不同的人员完成的并行变更不会被相互覆盖。

- 没有适当的防护措施，变更控制可能会阻碍进展，也可能会产生不必要的繁琐手续，大多数拥有控制机制的软件开发者通过许多控制环节来帮助避免变更控制过程的繁文缛节。

SCM过程

SCM过程：变更控制

- 在SCI成为基线之前，只需要进行非正式的变更控制。
  还在讨论之中的配置对象（SCI）的开发者可以进行任何变更，只要项目和技术需求证明变更是适当的。一旦配置对象经过正式技术评审并被批准，它就成为基线。

- 一旦SCI成为基线，就可以实现项目级变更控制了。
  这时，若要进行变更，开发者必须得到项目管理者的批准，如果该变更影响到其他SCI，则必须得到CCA的批准。
  在某些情况下，无需生成正式的变更请求，变更报告和ECO，但是，必须对每个变更进行评估，并对所有的变更进行跟踪和评审。

SCM过程

SCM过程：变更控制

- CCA在控制的第二层和第三层中扮演着主动的角色。
  CCA可以是一个人（如项目经理），也可以是很多人（如，来自软件、硬件、数据库工程、支持、市场等方面的代表），这取决于软件项目的规模和性质。
- CCA的作用是从全局的观点来评估变更对SCI之外事物的影响。
  变更将对硬件产生什么影响？变更将对性能产生什么影响？变更将怎样改变客户对产品的感觉？变更将对产品的质量和可靠性产生什么影响？还有很多其他问题都需要CCA来处理。

SCM过程

SCM过程：配置审核

- 标识、版本控制和变更控制帮助软件开发者维持秩序，否则情况可能将是混乱和不断变化的。
- 然而，即使是最优秀的控制机制也只能在ECO建立之后才可以跟踪变更。
- 通过技术评审和软件配置审核来保证变更的实现是正确的
  技术评审关注的是配置对象在修改后的技术正确性。评审者要评估SCI，以确定它与其他SCI是否一致，是否有遗漏，或是否具有潜在的副作用。除了那些非常微不足道的变更之外，应该对所有变更进行技术评审。

SCM过程

SCM过程：配置审核

- 作为技术评审的补充，软件配置审核针对在评审期间通常不被考虑的特征对配置对象进行评估。软件配置审核要解决以下问题：
  1. 在ECO中指定的变更已经完成了吗？引起任何额外的修改了吗？
  2. 是否已经进行了技术评审来评估技术正确性？
  3. 是否遵循了软件过程，是否正确地应用了软件标准？
  4. 在SCI中“显著标明”所做的变更了吗？是否说明了变更日期和变更者？配置对象的属性反映出该变更了吗？
  5. 是否遵循了SCM规程中标注变更、记录变更和报告变更的规程？
  6. 是否已经正确地更新了所有相关的SCI？

SCM过程：配置审核

- 在某些情况下，前述审核问题是作为技术评审的一部分来询问的。但是，当SCM是正式的活动时，配置审核将由质量保证小组单独进行。
  这种正式的配置审核还能够保证将正确的SCI（按版本）集成到特定的版本构造中，并且能够保证所有文档都是最新的，且与所构造的版本是一致的。

SCM过程

SCM过程：状态报告

- 配置状态报告（有时称为状态账目）是一项SCM任务，它解答下列问题：

1. 发生了什么事?
2. 是谁做的?
3. 是什么时候发生的?
4. 会影响到其他哪些事情?



SCM过程

状态报告

- 配置状态报告（Configuration Status Reporting, CSR）的信息流如变更控制过程示意图所示
  每当赋予SCI新的标识或更改其标识时，就会产生一个CSR条目；
  每当CCA批准一个变更（即创建一个ECO）时，就会产生一个CSR条目；
  每当进行配置审核时，其结果要作为CSR任务的一部分提出报告。
- CSR的结果可以放置到一个联机数据库或Web站点上，以便软件开发者或维护人员可以按照关键词分类来访问变更信息。
- 此外，定期生成的CSR报告使管理者和开发人员可以评估重要的变更。

SOFTWARE TOOLS

SCM的支持

目的：SCM工具可以支持22.3节所讨论的一个或多个过程活动。

机制：大部分最新的SCM工具都与一个中心存储库（一个数据库系统）相连，能够提供标识、版本控制和变更控制、审核及报告功能。

代表性工具：[⊖]

CCC/Harvest，由Computer Associates（www.cai.com）发行，是一个支持多种平台的SCM系统。

ClearCase，由Rational（www-306.ibm.com/software/awdtools/clearcase/index.html）开发，提供SCM的一系列功能。

Serena ChangeMan ZMF,由Serena（www.serena.com/US/products/zmf/index.aspx）发行，提供了适用于传统软件和WebApp的全套SCM工具。

SourceForge，由VA Software（sourceforge.net）发行，提供了版本控制、构造功能、问题/错误跟踪及其他管理功能。

SurroundSCM，由Seapine Software（www.seapine.com）开发，提供完整的变更管理功能。

Vesta，由Compac（www.vestasys.org）发行，是一个能够支持小型（<10 KLOC）和大型（10 000 KLOC）项目的没有版权限制的SCM系统。

商用SCM工具与环境的综合列表可以在www.cmtoday.com/yp/commercial.html找到。

小结

- 软件配置管理是应用于整个软件过程的普适性活动。SCM标识、控制、审核和报告修改总是发生在软件开发过程中及交付给客户之后。软件过程中产生的所有信息应该作为软件配置的一部分，要适当地对配置进行组织，才能进行有序的变更控制。
- 软件配置由一组相关的对象（也称为软件配置项）构成，这些对象是某些软件工程活动所产生的结果。除了文档、程序和数据外，用于创建软件的开发环境也应该属于配置管理。应该将所有的SCI存放在中心存储库中，中心存储库具有保证数据完整性的机制和数据结构，可以支持其他软件工具，支持软件团队所有成员之间的信息共享，还具有版本控制和变更控制功能。
- 一旦开发的配置对象通过了评审，它就成为基线。对基线对象的变更将导致对该对象新版本的创建。可以通过分析所有配置对象修订的历史记录来跟踪程序的演

小结

- 化过程。基本对象和复合对象可以形成对象池，通过对象池可以构建不同的版本。版本控制就是管理这些对象的一组规程和工具。
- 变更控制是一种过程活动，它能够在对配置对象进行变更时保证质量和一致性。变更控制过程从变更请求开始，然后决定是否拒绝该变更请求，最后，对将要变更的SCI进行可控制的更新。
- 配置审核是一种SQA活动，它有助于确保进行变更时仍能维护质量。状态报告为那些需要知道变更的人们提供了每次变更的信息。

思考题

1. 软件请求变更的起因是什么呢?
2. 每一个参与变更管理的人员的职责和应从事的活动是什么?
3. SCM中心存储库具有哪些功能?
4. SCM 过程应该回答什么问题?
5. 配置审核期间需要关注的主要问题是什么?



谢谢！